htl donaustadt
Donaustadtstraße 45
1220 Vienna
office@htl-donaustadt.at
www.htl-donaustadt.at

htl donaustadt

DIPLOMA THESIS

Plant Up!

Smart Plant Monitoring with IoT and Gamification

Authors: Arun SHERZAI
Abd-Ulrahman BEHARI
Richard ONUOHA
Dennis JIN

Supervisor: Mag. Dr. Monika PUTZINGER

Department of Computer Science – Software Engineering, 5BHIF

htl bildung mit zukunft

Project Number: 22IF2609

Academic Year: 2025/26

Author's Declaration

I hereby declare that I have written this thesis independently and used only the tools and resources listed. Any sections taken verbatim or paraphrased from other works (including databases) are clearly cited following academic referencing rules. This declaration also applies to images, tables, sketches, and drawings used in the thesis.

For assistance, I utilized generative AI tools such as ChatGPT, Grammarly Go, and this document. The use of these tools has been fully disclosed, including versions used. This document has been generated for academic purposes only and should not be considered as legal or expert advice. While every effort has been made to ensure the accuracy and reliability of the information presented, the author and publisher assume no responsibility for any errors, omissions, or any consequences resulting from the use of this document.

Portions of this text have been structurally enhanced with the assistance of ChatGPT. The tool was used solely for formatting, organization, and language clarity, and the core content and its integrity remains with the author.

Any opinions, findings, or conclusions expressed herein are those of the author and do not necessarily reflect the views of any organizations or entities.

The inclusion of external sources, links, or references does not imply endorsement, and the responsibility for verifying their contents rests solely with the reader. For specific guidance tailored to your circumstances, please consult a qualified professional.

Arun Sherzai	Abd-Ulrahman Behari
Richard Onuoha	Dennis Jin

Wien, 30. März 2026

Danksagungen

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed eiusmod tempor incidunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquid ex ea commodi consequat.

Abstract

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 1

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Subsubsection 2

Quis aute iure reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint obcaecat cupiditat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Table of Contents

1 Introduction **1**

2 Combining Nature and Technology **2**

2.1 Introduction 2

2.2 Environmental Factors in Plant Growth 3

2.2.1 Importance of Soil Moisture, Temperature, Light, and Humidity 4

2.2.2 Effects of Environmental Factors on Plant Growth 5

2.2.3 Why Sensor-Based Logging Matters 6

2.3 Sensor Technologies and the Measurement Pipeline 6

2.3.1 Overview of Common Sensor Types 7

2.3.2 Limitations and Typical Error Sources 8

2.3.3 Selection Criteria 10

2.3.4 The Measurement Cycle 10

2.4 Data-Driven Decision Making in Plant Care 12

2.4.1 Threshold Zones and Plant-Specific Ranges 13

2.4.2 Deriving Care Recommendations 14

2.4.3 Visualization and Interpretation 14

2.5 System Implementation and Validation 16

2.5.1 Hardware Assembly 16

2.5.2 Sensor Calibration 18

2.5.3 Practical Validation 19

2.6 Conclusion 20

3 Edge vs. Cloud Processing in Smart Gardening **22**

3.1 Introduction . . . 22
3.1.1 The Architectural Challenge: Latency and System Dependability . . . 22
3.1.2 Research Objective . . . 23
3.1.3 Scope and Limitations . . . 23
3.1.3.1 Computational Constraints of the Edge Device . . . 23
3.1.3.2 Abstraction of the Transport Layer . . . 23
3.2 Theoretical Framework . . . 24
3.2.1 The IoT World Forum Reference Model (7-Layer Model) . . . 24
3.2.2 Modern IoT Backends . . . 25
3.2.2.1 Backend-as-a-Service (BaaS) and the SQL vs. NoSQL Paradigm . 25
3.2.2.2 Time-Series Databases: Why Standard SQL Fails for Sensor Streams . . . 25
3.2.2.3 The TimescaleDB Extension: Hypertables and Chunking Explained 27
3.2.3 Microservices in IoT . . . 28
3.2.3.1 The Gateway Pattern: Bridging Edge and Core Services . . . 28
3.2.3.2 Service Inter-communication: Synchronous vs. Asynchronous Patterns . . . 29
3.2.4 Edge vs. Cloud . . . 29
3.2.4.1 Defining the Cloud-Centric Model . . . 29
3.2.4.2 Defining the Edge-Centric Model . . . 30
3.2.4.3 The Spectrum of Logic Distribution: “Thick Edge” vs. “Thin Edge” . 30
3.3 The PlantUp Architecture . . . 32
3.3.1 System Overview . . . 32
3.3.1.1 High-Level Architecture Pipeline . . . 32
3.3.2 The Application and Gateway Layers (Cloud) . . . 34
3.3.2.1 Application Layer Services: Execution and Communication . . . 34
3.3.2.2 Direct-to-Database Client Architecture and Security . . . 34

3.3.2.3 Device Management Service (Gateway) . . . 35
3.3.3 The Data Persistence Layer (Supabase) . . . 35
3.3.3.1 Unified Infrastructure with Schema-Level Isolation . . . 35
3.3.3.2 Time-Series Performance Optimization via TimescaleDB . . . 36
3.4 Methodology . . . 36
3.4.1 Experimental Control Framework . . . 37
3.4.1.1 Architecture A: Cloud-Centric (Deployed Variant) . . . 37
3.4.1.2 Architecture B: Edge-Centric (Control Variant) . . . 37
3.4.2 Implementation Constraints . . . 37
3.4.3 Measurement Methodology and Environment . . . 38
3.4.3.1 Testing Environment . . . 38
3.4.3.2 Latency Calculation . . . 38
3.4.4 Test Scenarios . . . 39
3.4.4.1 Scenario 1: Baseline Operational Thresholds . . . 39
3.4.4.2 Scenario 2: Infrastructure Stress Test (Disaster Recovery Spike) . 39
3.5 Results and Data Analysis . . . 40
3.5.1 Latency Analysis . . . 40
3.5.1.1 Edge Response Time (<50ms) vs. Cloud Round Trip (>200ms + DB Query Time) . . . 40
3.5.2 Database Performance . . . 40
3.5.2.1 Impact of High Ingestion Rates on C# Service Query Performance 40
3.5.2.2 Storage Efficiency: Raw Data (Cloud Decision) vs. Batched Aggregates (Edge Decision) . . . 40
3.5.3 Reliability and Resilience . . . 41
3.5.3.1 Alert Delivery Success Rates during a simulated 1-hour internet outage . . . 41
3.5.4 Gamification Synchronization . . . 41

3.5.4.1 The “Lag” between Action (Watering) and Reward (XP Update in Social Service) . . . 41
3.6 Discussion . . . 41
3.6.1 Validating the "Thin EdgeÄrchitecture . . . 41
3.6.1.1 Latency vs. Edge Autonomy . . . 41
3.6.2 Database Performance and Resilience . . . 42
3.6.2.1 Resilience Under Stress . . . 42
3.6.3 Complexity vs. Operational Maintenance . . . 43
3.6.3.1 The Trade-off of Edge Complexity . . . 43
3.7 Intermediate Conclusion: Edge vs. Cloud Architecture . . . 43
3.7.1 Summary of Findings . . . 43
3.7.2 Transition to the Implementation Layer . . . 44

4 IoT Data Sync in Microservices 45

4.1 Introduction . . . 45
4.1.1 Background and Context . . . 45
4.1.2 Case Study: The “Plant Up!” Project . . . 46
4.1.3 Problem Statement . . . 46
4.2 Theoretical Background . . . 47
4.2.1 Microservices Architecture (MSA) . . . 48
4.2.2 Core Characteristics of Microservices in IoT . . . 49
4.2.2.1 Autonomy and Database per Service . . . 49
4.2.2.2 Scalability and Elasticity . . . 50
4.2.3 Internet of Things (IoT) Constraints and Hardware . . . 50
4.2.3.1 Power Consumption and Sleep Modes . . . 50
4.2.3.2 The Wake-Up Tax . . . 51
4.2.4 Communication Protocols: MQTT vs. REST . . . 51
4.2.4.1 MQTT (Message Queuing Telemetry Transport) . . . 51

4.2.4.2 REST (Representational State Transfer) . . . 52
4.2.5 Data Consistency and the CAP Theorem . . . 53
4.3 Plant Up! System Architecture and Implementation . . . 54
4.3.1 System Overview and Vision . . . 54
4.3.2 Hardware Layer: The Edge Node . . . 55
4.3.2.1 Component Selection and Sensor Interface . . . 55
4.3.2.2 Data Structure . . . 56
4.3.3 Microservice Architecture Design . . . 57
4.3.4 Real-Time Data Synchronization Mechanism . . . 58
4.3.5 User Engagement and Data Processing . . . 59
4.4 Experimental Evaluation of MQTT vs. REST . . . 61
4.4.1 Introduction to the Experiment . . . 61
4.4.2 Experimental Setup . . . 61
4.4.3 Methodology and Metric Acquisition . . . 62
4.4.4 Experimental Results . . . 63
4.4.5 Discussion . . . 63
4.4.6 Energy Implications of Protocol Selection on ESP32-Based IoT Devices 65
4.4.7 Conclusion . . . 67

5 Gamification. Gaming design patterns to keep people engaged in apps. 68

5.1 Introduction . . . 68
5.2 Theoretical Foundations of Gamification . . . 68
5.2.1 Definition of Gamification . . . 68
5.2.2 Difference between Gamification, Serious Games, and Games for Entertainment . . . 69
5.2.3 Why Gamification Works in Non-Game Contexts . . . 69
5.2.4 Motivation Theory . . . 70
5.2.4.1 Intrinsic Motivation (The Engine of Engagement) . . . 70

5.2.4.2 Extrinsic Motivation (The Spark) . . . 73
5.2.5 Attribution Theory . . . 74
5.2.6 Expectancy-Value Theory . . . 74
5.2.7 Engagement and Habit Formation . . . 75
5.2.7.1 Feedback Loops . . . 75
5.3 Gamification Design Patterns . . . 76
5.3.1 Core Game Mechanics . . . 76
5.3.2 Advanced Engagement Patterns . . . 77
5.3.3 Social Gamification . . . 78
5.3.4 Failure States & Ethical Design . . . 78
5.4 Why Gamification Works for Normal Activities . . . 79
5.4.1 Transformation of Mundane Tasks . . . 79
5.4.2 Feedback in Real-World Processes . . . 80
5.4.3 Emotional Attachment & Ownership . . . 81
5.5 Case Study: Gamification in Plant Up! . . . 82
5.5.1 Evaluation and Discussion . . . 82
5.5.1.1 Meaningful Habit Formation . . . 82
5.5.1.2 Lowering the Entry Barrier . . . 82
5.5.1.3 Potential Long-Term Challenges . . . 84
5.5.1.4 Technical Architecture & Community . . . 84
5.6 Evaluation & Discussion . . . 84
5.6.1 Expected Engagement Improvements . . . 84
5.6.2 Risks & Limitations . . . 85
5.6.3 Comparison to Non-Gamified Apps . . . 85

Appendix **85**

List of Figures . . . 85

List of Tables 87

AI-Generated Figures 88

References 90

1 Introduction

The "Plant Boom" vs. The Reality

The indoor plant market experienced a massive rise during the early 2020s, with people spending more money on indoor plants. From $1.31 billion in 2019 to $2.17 billion by 2021 [1]. This "boom" saw household participation in indoor gardening reach a four-year high of 38.1 million households [2]. However, even with this rise in popularity, the reality is that people are not very good at taking care of plants. While demand rose by 18%, failure rates remained constant: approximately 40% of plants perish within the supply chain before reaching a shelf [3], and another 35% die shortly after entering a customer's home [4]. This indicates that the industry's economic "boom" is significantly fueled by a replacement cycle where consumers repeatedly purchase new plants to compensate for those lost.

What is "Plant Up!" ?

Plant Up! is a smart gardening system that aims to solve this problem by providing users with the tools and knowledge they need to keep their plants alive. It combines IoT sensors with a mobile application to create a seamless and user-friendly experience [5]. The system monitors soil moisture, light, and temperature in real-time, and provides users with actionable insights and guidance on how to care for their plants.

What "Plant Up!" offers

Plant Up! [5] offers a range of features designed to help users keep their plants alive. These include:

- Real-time monitoring of soil moisture, light, and temperature
- Actionable insights and guidance on how to care for plants
- A comprehensive database of plants and their care requirements
- A community forum for users to share tips and advice

2 Combining Nature and Technology: How Sensor-Based and Data-Driven Technologies Enhance Plant Monitoring and Care

Author: Arun Sherzai

2.1 Introduction

Houseplants are a common feature in modern households, with the global indoor plant market valued at approximately 21 billion USD in 2025 [6]. Yet maintaining them over longer periods remains challenging for many people. One of the most common reasons is improper care rather than neglect. In particular, horticultural experts identify overwatering as the leading cause of houseplant failure, frequently resulting in yellowing leaves, wilting, and root rot [7]. This indicates a fundamental issue: plants often suffer because their actual needs are misunderstood.

A key reason for this misunderstanding lies in the limitations of human judgment. Soil moisture is commonly assessed by touching the surface of the soil, even though moisture levels can differ significantly between the upper layer and the root zone [7]. As a result, soil may appear dry while deeper layers remain saturated, which increases the risk of root diseases and root rot [8]. These processes occur below the surface and are therefore difficult to detect without technical support.

Human perception of light conditions is similarly unreliable. The human eye adapts to changing brightness levels, which often leads to an overestimation of the usable light available in indoor environments [9]. A room may appear bright to humans while still providing insufficient light for healthy plant growth.

On the other side, sensor technology and Internet of Things (IoT) applications are becoming increasingly widespread. The global smart home market is experiencing strong growth, driven mainly by automation in areas such as lighting, climate control, and energy management [10, 11]. Despite this development, plant care in private households remains largely intuition-based.

This gap motivates the development of Plant Up!.

The primary objective of this thesis is to show that effective plant monitoring does not require expensive or specialized equipment. Using affordable, consumer-grade hardware such as soil moisture sensors, digital environmental sensors, and a microcontroller-based data acquisition system, this work demonstrates how common care mistakes can be reduced. In doing so, it aims to improve plant survival and provide users with a

clearer understanding of the environmental conditions that influence plant health.

Based on the challenges outlined above, this thesis addresses the following research question:

How can a low-cost, sensor-based system be designed to provide objective environmental data and actionable care recommendations?

This question focuses on practical application rather than theoretical considerations. The goal is not to replace horticultural expertise, but to support everyday plant owners by providing objective information that simplifies decision-making. By translating sensor measurements into clear feedback, the system helps users recognize when action is necessary and when intervention would be counterproductive.

The relevance of this research extends across several areas. From an environmental perspective, sensor-based watering reduces water waste. Many people water their plants based on routine or guessing, which often leads to unnecessary consumption. By using sensors, water is only applied when the plant actually needs it, supporting a more sustainable approach [12, 13].

From an emotional perspective, many plant owners are frustrated when their plants die despite regular care. By visualizing environmental conditions and offering clear care recommendations, successful plant care becomes easier, as users do not need detailed botanical knowledge.

The topic is also relevant from a technological perspective. Advances in microcontrollers, sensor accuracy, and wireless communication make it possible to implement functional monitoring systems using affordable consumer-level hardware. This thesis demonstrates how such technologies can be combined into an accessible IoT solution for everyday plant care.

To understand why these measurements are essential, the following chapter examines the environmental factors that directly influence plant growth and health.

2.2 Environmental Factors in Plant Growth

Plants are living systems, but they can be treated as measurable input-output systems in engineering practice. Light, water, temperature, and humidity are not “nice-to-have” background conditions. They are measurable variables with operating ranges and failure limits. When a variable stays outside its range, stress is not a surprise. It is the expected outcome. This chapter focuses on what can be measured and controlled, because Plant Up! must convert plant needs into sensor values and thresholds. [14]

2.2.1 Importance of Soil Moisture, Temperature, Light, and Humidity

Soil moisture

Soil is a porous material made of solids plus pore space filled with air and water. An ideal soil for plant growth is often described as 50 % solids and 50 % pore space, ideally split into equal parts air and water so roots get both oxygen and moisture. This is the physical reason why “slightly moist” often performs better than “always wet.” [14]

When pore spaces stay waterlogged, air is displaced and the oxygen supply to roots drops. Root function declines and nutrient uptake suffers, even while the pot appears wet. This oxygen deficit is the primary mechanism behind overwatering damage and explains why soggy soil is more dangerous than dry soil for most indoor plants. [15]

The opposite end of the spectrum is the *permanent wilting point*: the threshold where remaining soil moisture is physically unavailable to roots. Below this point the plant cannot recover through normal watering, because the accessible water reserve is fully depleted. For monitoring, volumetric water content (VWC) expresses soil water as a percentage of total soil volume, which provides a consistent, loggable metric. [16, 17]

Light

Light is the energy input that drives photosynthesis. Plants do not use all wavelengths equally, so the relevant band is Photosynthetically Active Radiation (PAR), typically defined as the $400-700$ nanometer (nm) range. This distinction matters because “bright to humans” and “usable for plants” are not the same thing. [18]

The plant-focused intensity metric is Photosynthetic Photon Flux Density (PPFD), which counts photons in the PAR range arriving per area per second. Many low-cost systems instead measure illuminance in lux, which is weighted for human vision and therefore does not map cleanly to PPFD. Lux is still practical for relative comparisons within one setup, such as comparing a dark corner to a bright window. [19]

Temperature and humidity

Temperature controls how fast plant processes run. When it is too low, growth slows as biochemical reactions become sluggish. When it is too high, regulation mechanisms lose efficiency and heat damage risk increases. For indoor monitoring, temperature is a core variable because it directly affects water demand and interacts with humidity. [14]

Humidity links to transpiration, the process where water evaporates from leaf surfaces and creates an upward flow that transports dissolved minerals. When humidity is low, transpiration accelerates and the plant loses water faster, potentially outpacing root uptake even if the soil is moist. Because temperature changes the air’s water-holding

capacity, humidity must always be interpreted together with temperature to correctly assess the atmospheric demand on the plant. [20, 21]

Technical link: translating needs into metrics

The described processes depend on physical quantities, not on perception. Roots react to oxygen availability and water content, not to how soil feels. Leaves respond to photon availability and atmospheric drying power, not to how bright or warm a room seems.

For control, these processes must be expressed as measurable variables. Soil moisture becomes VWC, light becomes PPFD or lux, and temperature and humidity define the atmospheric load on the plant. Only numerical values allow operating ranges and thresholds to be defined and monitored. However, simply tracking numbers is not enough. To design an effective warning system, one must understand what happens when these boundaries are breached.

2.2.2 Effects of Environmental Factors on Plant Growth

Deficiency: when values are too low

Low soil moisture reduces water availability until the permanent wilting point is reached. Monitoring must warn before the plant approaches this boundary, because recovery becomes unreliable once accessible water is depleted. [16]

Low light creates a different stress pattern. The plant tends to grow taller and thinner as it stretches toward available light, often producing weaker stems and paler leaves. This gradual change indicates chronic under-lighting and is not resolved by a single bright day. [22]

Low temperature slows growth by reducing reaction rates. Low humidity increases transpiration demand, which means the plant can become water-stressed even when the soil appears adequately moist. [21]

Excess: when values are too high

Excess soil moisture displaces air in pores, starving roots of oxygen. The plant can then show wilting symptoms even in wet soil, because damaged roots cannot absorb water properly. This is why overwatering often looks like underwatering to the untrained eye. Prolonged wet conditions also favor root decay pathogens, accelerating damage further. For engineering control, the upper moisture limit is a safety boundary that must be enforced with alerts. [15, 8]

Excess light and heat can cause leaf scorch, especially when a shade-adapted plant is suddenly exposed to strong sun. The damage occurs during short peak exposures

rather than from long-term averages, which means monitoring must capture peaks, not only means. [23]

Safe corridors and thresholds

A monitoring system needs a control model that prevents both deficiency and excess. The simplest model is a safe corridor: a defined acceptable range for each variable. Inside the corridor, stress risk is low. Near corridor edges, the system should warn before the plant enters a damage trajectory. These corridors are species-dependent, so they must be configurable rather than hard-coded. [24]

A practical example: many houseplants prefer relative humidity around $40-60\,\%$, while tropical species often prefer higher levels. This is exactly why a system should start with reasonable defaults and then refine them using observed trends. Data makes the corridor adaptive instead of theoretical.

2.2.3 Why Sensor-Based Logging Matters

Human perception is qualitative, biased, and slow compared to plant dynamics. Surface touch checks miss root-zone conditions, the eye adapts to darkness and overestimates available light, and weekly observation misses hourly stress peaks. [25, 26] Continuous logging at fixed intervals is not a luxury but a practical requirement, because it reveals patterns and variability that single observations cannot capture. [21]

Sensors convert hidden processes into objective numbers and timestamps. They enable threshold checks, trend detection, and early warnings before visible symptoms appear. This is the real value of Plant Up!: plant care becomes a monitored system, not a guessing game.

With the biological variables defined and the necessity of objective measurement established, the next chapter evaluates the physical hardware required and describes how sensor signals are captured, filtered, and transmitted as structured data.

2.3 Sensor Technologies and the Measurement Pipeline

Every environmental variable, whether soil moisture, temperature, humidity, or light, must be converted into an electrical signal before a controller can process it. The choice of sensor determines not only what can be measured, but also how accurately and how long the system can operate without maintenance. But selecting appropriate hardware is only the first step. The raw signals must also be read, filtered, and packaged into structured data before they can be evaluated. This chapter covers both: the sensor selection for Plant Up! and the firmware-level data pipeline that turns their output into usable information.

2.3.1 Overview of Common Sensor Types

Soil moisture sensors

Low-cost soil moisture measurement is typically implemented using resistive or capacitive sensors.

A resistive sensor works by passing a small electric current between two metal probes inserted into the soil. Wet soil conducts electricity better than dry soil, so the measured resistance drops as moisture increases. The principle is straightforward and the hardware is cheap, but there is a significant disadvantage: the metal probes are directly exposed to wet soil, which causes them to corrode over time. As the probes degrade, readings become increasingly unreliable. [27]

A capacitive sensor takes a different approach. Instead of measuring how well the soil conducts current, it detects how the surrounding material affects an electric field generated by the sensor. In simple terms, water changes the electrical properties of the soil around the sensor, and this change is what gets measured. The key advantage is structural: the sensing element is embedded inside a coated circuit board, so no metal is directly exposed to the soil. This eliminates the corrosion problem and makes the sensor significantly more durable for long-term use. [28]

Figure 2.1: Resistive Soil Moisture Sensor [29]

Figure 2.2: Capacitive Soil Moisture Sensor [29]

As shown in Figures 2.1 and 2.2, the structural difference is immediately visible. The resistive sensor exposes two metallic forks directly to the soil, while the capacitive sensor uses a sealed PCB surface. This is the core reason why capacitive sensors are generally more suitable for long-term indoor monitoring. [27, 28]

Temperature sensors

Indoor monitoring often relies on digital temperature sensors to avoid the noise inherent in analog readings. A digital sensor converts the measured value internally and sends a numerical result directly to the microcontroller.

For example, the MLX90614 measures ambient or surface temperature without physical contact using infrared, while the DS18B20 is a rugged, contact-based probe commonly used to measure root-zone soil temperature. The MLX90614 communicates via I^2C, a two-wire protocol that uses one data line and one clock line. Multiple I^2C sensors can share the same two wires, which simplifies wiring considerably. [30]

Light sensors

Light intensity can be measured using a photoresistor or a digital lux sensor. A photoresistor changes its resistance depending on brightness. It is simple and inexpensive but mainly useful for relative comparisons within the same setup. A digital lux sensor, by contrast, outputs illuminance directly in calibrated lux values. [31]

As noted in Section 2.2, plants respond to light in the PAR range ($400-700\,\text{nm}$), and lux reflects human vision rather than plant needs. [18] For basic indoor monitoring, however, lux values are still practical enough to detect whether a plant is receiving adequate or insufficient light.

All three sensor categories, moisture, climate, and light, offer viable solutions for low-cost monitoring. However, selecting the right sensor type is only half the equation. Understanding their practical limitations is equally important, because no sensor delivers perfect data under real-world conditions.

2.3.2 Limitations and Typical Error Sources

Non-uniform soil moisture distribution

Soil inside a pot does not dry evenly. Water moves downward due to gravity, and localized wet zones can persist long after watering.

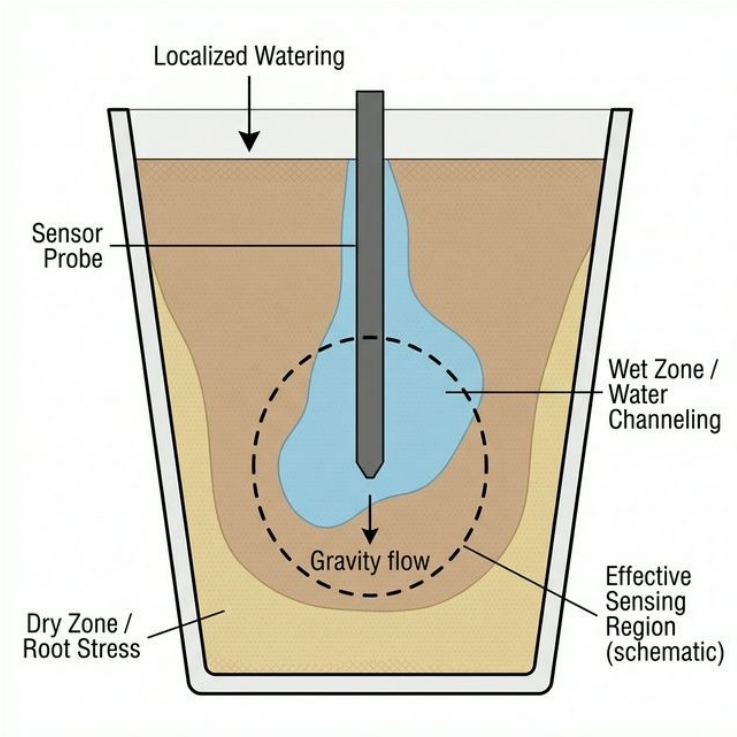

Figure 2.3: Measurement distortion caused by non-uniform soil moisture in a plant pot. The image illustrates how localized watering can distort sensor readings (generated with ChatGPT 4o, OpenAI).

As shown in Figure 2.3, a soil moisture probe only measures a limited region around its body. A single reading may therefore not represent the entire pot. If the sensor happens to sit in a locally wet or dry pocket, threshold-based decisions can become misleading. To reduce this risk, the sensor should always be placed at the same position and pushed to the same depth, ideally near the center of the pot and within the root zone rather than close to the surface.

Sensor drift

Sensor drift describes a gradual shift in readings over time without any real change in the environment. For resistive sensors, corrosion is the primary driver: as the metal probes degrade, the baseline resistance changes. Capacitive sensors reduce this problem significantly but can still drift due to aging or surface contamination. Periodic validation against a known reference improves long-term reliability. [27, 28]

Electrical noise and ADC limitations

Electrical noise refers to unwanted fluctuations in a signal caused by interference or unstable power supply. Analog sensors require an Analog-to-Digital Converter (ADC), which translates a continuous voltage into a discrete digital value. Because the resolution of the ADC is finite, very small moisture changes may not register at all. Hardware quality and basic software filtering, such as averaging multiple readings, therefore directly influence measurement stability.

2.3.3 Selection Criteria

For Plant Up!, a capacitive soil moisture sensor is the preferred choice. It avoids exposed metal electrodes, which eliminates the corrosion failure mode that limits resistive alternatives. This directly improves long-term stability, which is critical for a system intended to run continuously in a home environment. [28]

Compatibility with the ESP32 microcontroller is another practical constraint. The ESP32 operates at 3.3 volt (V) logic levels. In simple terms, a logic level is the voltage that a microcontroller uses to communicate with its connected components. When the ESP32 sends or receives a signal, it interprets 3.3 V as a logical “on” and 0 V as a logical “off.” Many older or larger microcontrollers use 5 V for this purpose instead. If a 5 V sensor is connected directly to a 3.3 V controller, the higher voltage can damage the input pins or cause unreliable readings. To avoid this, all sensors in Plant Up! must natively support 3.3 V operation so that no additional voltage conversion circuitry is needed.

In this project, cost and durability outweigh the need for absolute calibration accuracy. The goal is reliable threshold monitoring, not scientific soil analysis. Knowing whether moisture is “inside the safe corridor” or “approaching the danger zone” matters more than knowing the exact volumetric water content to two decimal places.

Digital sensors further simplify the hardware design. By communicating over standardized buses like I^2C, they reduce analog noise issues and minimize the number of required microcontroller pins.

Based on these criteria, the implemented hardware setup of Plant Up! consists of an ESP32 microcontroller and a comprehensive suite of sensors: a capacitive soil moisture probe, a BH1750 digital light sensor, an MLX90614 infrared air temperature sensor, a DS18B20 soil temperature probe, and an electrical conductivity (EC) sensor for nutrient monitoring. Additionally, an SD card module is included for local storage of configuration data, such as Wi-Fi credentials.

The ESP32 serves as the central processing unit and provides integrated Wi-Fi connectivity as well as 12-bit ADC channels for the analog moisture and EC sensors. Its 3.3 V logic level defines the voltage compatibility requirements, and all selected components natively support this voltage, meaning no external level shifters are needed. A detailed component list and wiring schema is provided in the implementation chapter.

2.3.4 The Measurement Cycle

With the hardware selected, the remaining question is purely procedural: how does the controller read, clean, and transmit sensor values in a repeatable cycle?

Firmware read cycle

The firmware runs a repeating loop: wake, read sensors, process values, transmit, and sleep. Deep sleep, a power-saving mode where the CPU and most peripherals are shut down and only a small timer domain stays active for wake-up, is used after each transmission. [32]

The read sequence matters. The soil moisture and EC sensors are analog, meaning their raw voltages need a brief moment to stabilize after the controller wakes up. That is why they are read first. The digital sensors (BH1750, MLX90614, and DS18B20), on the other hand, manage their own conversions and deliver ready-to-use values when queried. Reading analog first and digital second keeps the cycle fast and stable. The entire active phase stays below approximately 2 seconds, after which the controller goes back to sleep and waits for the timer to start the next round. [32]

Sampling strategy and power management

In Plant Up!, the default measurement interval is 10 minutes, though it can be configured. This is long enough to keep battery consumption low but short enough to capture typical indoor changes such as watering events, sunlight shifts, or heating cycles.

The design relies on the extreme difference between active and sleep current, as visualized in Figure 2.4. During active operation with Wi-Fi enabled, the ESP32 draws around 240 milliamperes (mA) and can briefly peak above 300 mA during wireless calibration. In deep sleep, current consumption drops to approximately 10 microamperes (μA). [33] With timer-based automatic wake-up, this duty-cycling enables an estimated battery life of several weeks on a standard Li-Ion cell, depending on board design and signal quality. Shorter intervals, such as every $1-5$ minutes, are supported for debugging and calibration but reduce runtime proportionally, because the Wi-Fi startup and transmission overhead becomes the dominant consumer. [32]

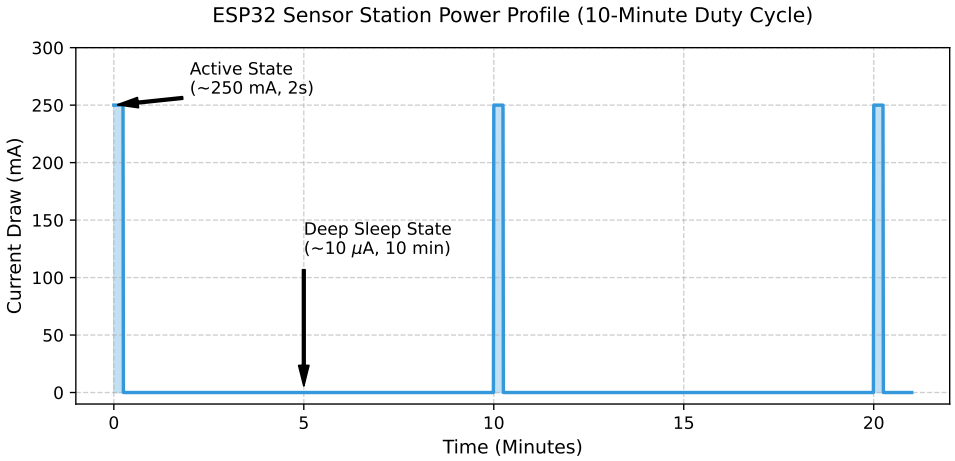

Figure 2.4: Idealized power profile of the ESP32 microcontroller during a 10-minute duty cycle, illustrating the extreme contrast between the brief active transmission phase and the long deep sleep phase.

Signal processing and data cleaning

Each cycle takes multiple readings per sensor, typically five, and reduces them to one stable value. Simple averaging smooths random noise, while a median filter is useful when single spikes occur, for example from one bad ADC sample. This is especially relevant for the analog soil moisture channel, where ADC fluctuations were already discussed in Section 2.3.2.

The firmware also performs plausibility checks. Physically impossible values are discarded, such as negative humidity or moisture readings above 100 %. If a digital sensor does not respond within a timeout, the system logs a null value for that field and continues the cycle with the remaining sensors instead of blocking entirely. Null values are preserved in the payload so the backend can detect and handle data gaps.

Data packaging and transmission

After cleaning, values are assembled into a single payload using JavaScript Object Notation (JSON). JSON is a lightweight text format for structured data, built from name-value pairs. [34] A typical payload contains the device ID, a timestamp, and the current readings: soil moisture (%), air temperature (°C), soil temperature (°C), electrical conductivity, and light intensity (lux).

The payload is transmitted via Message Queuing Telemetry Transport (MQTT), a lightweight publish/subscribe messaging protocol designed for constrained IoT devices. [35] How MQTT compares to REST in terms of latency and reliability is covered separately by Richard Onuoha in his chapter on IoT data synchronization in microservice architectures.

To keep the device robust in real apartments, where Wi-Fi can drop out or routers restart, the firmware includes retry logic. If the connection to the broker fails, the data point is buffered locally and retransmitted on the next cycle. No long-term storage is kept on the device. The intention is short buffering for connectivity gaps, not acting as a local database.

With this, the measurement cycle ends with clean, timestamped, structured values arriving at the backend. This is exactly the input needed for the next chapter, where thresholds and interpretation logic turn the raw data stream into actionable care recommendations.

2.4 Data-Driven Decision Making in Plant Care

After the measurement cycle described in Section 2.3, Plant Up! receives structured sensor data at the backend: timestamped values for moisture, climate, and light. This chapter explains how those incoming values are interpreted and transformed into feedback that a user can act on without needing to read raw numbers.

2.4.1 Threshold Zones and Plant-Specific Ranges

Building on the safe corridors defined in Section 2.2, each incoming sensor value is compared against an ideal range stored as a plant-specific configuration. The system classifies every parameter into three condition zones: Green (optimal, no action), Yellow (close to a boundary, attention recommended), and Red (outside the safe range, action required).

This applies to soil moisture, temperature, humidity, light, and, if available, the EC of the nutrient solution. The traffic-light mapping is intentionally chosen because green, amber, and red are widely recognized as low, medium, and high status codes in everyday decision aids.

Plant-specific ranges are not a universal default. A cactus is typically configured with lower moisture tolerance and higher light tolerance, while a fern prefers higher humidity and lower exposure to direct sunlight. In Plant Up!, these differences are handled as configurable parameters per plant profile rather than hardcoded constants.

Seasonal adjustment is implemented by storing separate summer and winter ranges. The reasoning is practical: indoor climate and light availability shift with seasons, and the same plant often needs different watering frequency and different tolerance bands depending on these conditions. Instead of forcing users to retune thresholds manually, Plant Up! switches between two predefined sets while keeping them editable.

To avoid overwhelming users with five separate decisions, Plant Up! computes a health score from 0–100 % as a combined indicator. Such composite scoring simplifies interpretation without discarding the underlying data from individual sensors. [36] The health score compresses complexity into one overall status that is easy to scan. It is mapped to four labels: Great ($\geq$70 %), Okay (40–70 %), Needs Care (20–40 %), and Critical ($<$20 %).

As shown in Figure 2.5, the interface exposes both layers at once: the overall health score for quick orientation and the individual sensor values for users who want to verify the cause of a warning. The Daily Mentor feature is an AI-supported component that generates short natural-language hints based on the current zone states, turning combinations like “too dry” and “high temperature” into a concrete watering suggestion.

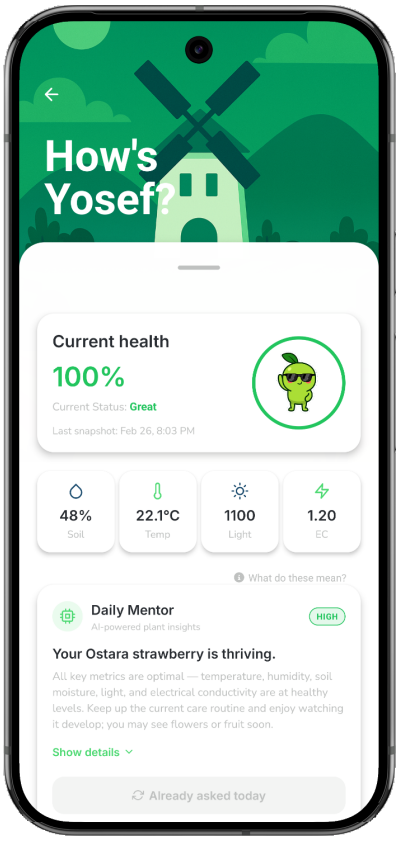

Figure 2.5: Plant status overview in the Plant Up! application showing health score, individual sensor values, and AI-generated care recommendations.

2.4.2 Deriving Care Recommendations

When a value leaves its ideal range, Plant Up! generates a recommendation phrased as an action, not as a measurement. Translating raw sensor readings into specific, user-facing actions is a well-established approach in threshold-based decision support systems. [37] The goal is that a user reads “Water it now” instead of interpreting “Soil moisture: 18 %” under time pressure. The system evaluates parameters individually, and the first parameter that triggers a Yellow or Red state becomes the active recommendation until the next measurement cycle updates the situation.

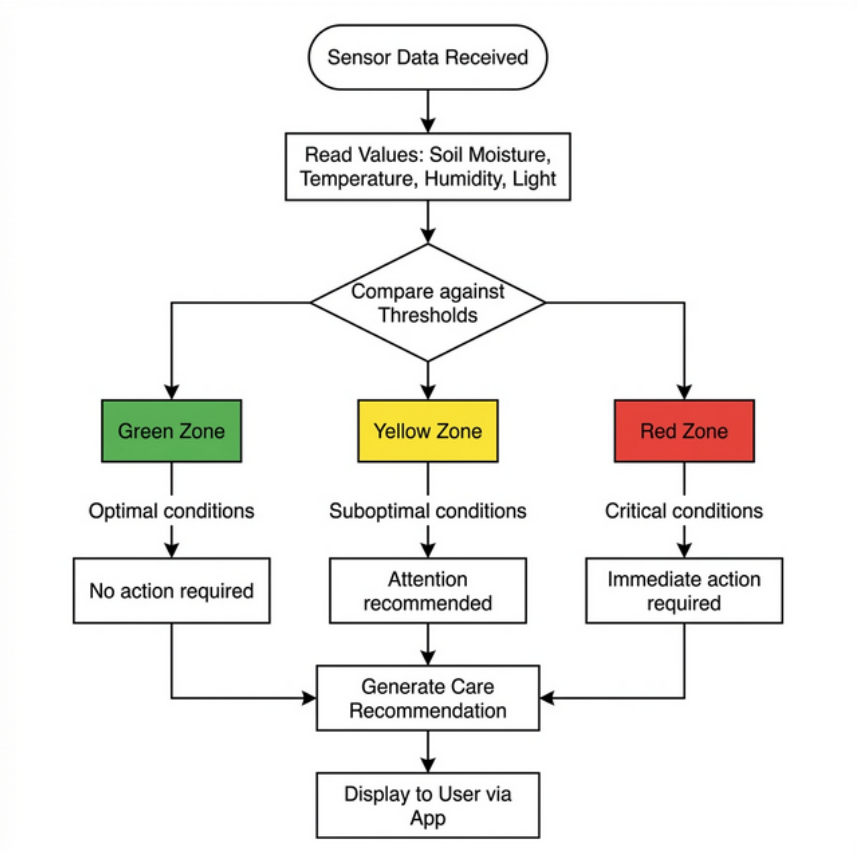

Figure 2.6: Simplified decision flow: sensor data is compared against plant-specific thresholds and classified into condition zones, which then trigger the corresponding care recommendation (generated with ChatGPT 4o, OpenAI).

Figure 2.6 summarizes this logic visually: incoming measurements are checked against stored thresholds, classified into zones, and then translated into one user-facing recommendation.

2.4.3 Visualization and Interpretation

Interpretation is not only logic. It is also presentation. Plant Up! uses a traffic-light visualization with color-coded indicators (green, yellow, red) and an animated mascot (“Sprig”) that mirrors the current plant state. The purpose is speed: users should recognize “all good” versus “act now” in one glance, even without understanding the sensor

units. Using color as a semantic status channel follows the principle that red signals danger, yellow signals caution, and green signals a safe condition, a convention standardized in ISO 3864. [38]

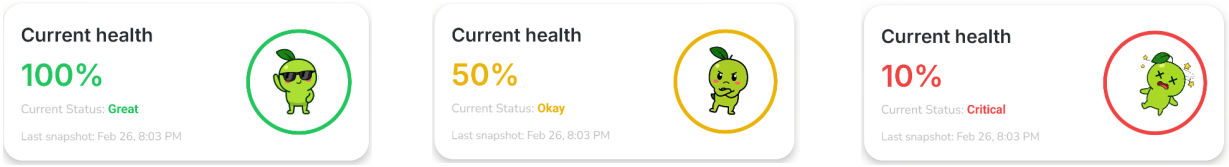

Figure 2.7: Traffic light health status in Plant Up!: optimal conditions (left, 100 %), suboptimal conditions requiring attention (center, 50 %), and critical state requiring immediate action (right, 10 %).

Figure 2.7 illustrates these three primary health states. Rather than focusing on numbers, the interface leverages the mascot's expression and the background color to immediately communicate whether the plant is thriving, needs attention, or requires urgent care.

Single values are useful, but trends are where the "why" becomes visible. Plant Up! therefore shows historical data as bar charts with selectable time ranges (1 hour, 24 hours, 7 days, all time). Time-oriented visualizations make it easier to detect patterns and anomalies than isolated snapshots, because humans can quickly see slopes, cycles, and sudden jumps across a timeline. [39]

As illustrated in Figure 2.8, a 1-hour chart focuses on immediate short-term variations, providing live insight into sudden environmental shifts. These patterns support quick reactions and help users confirm if their immediate physical actions, like watering the plant, register correctly within the system.

With these decision rules and visual mappings defined, Plant Up! can transform raw measurements into consistent feedback. The next chapter describes how this interpretation layer is implemented in the real system and tested under practical conditions.

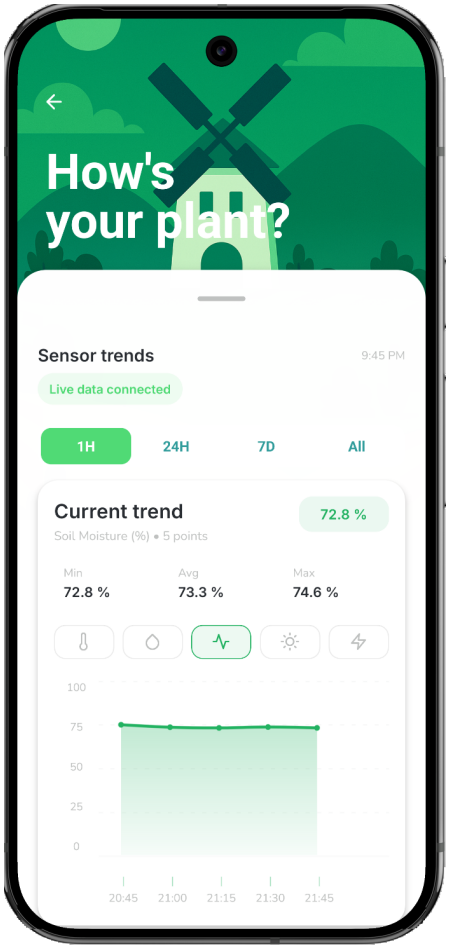

Figure 2.8: Historical sensor data visualization in Plant Up! showing soil moisture readings over a 1-hour period. Users can switch time ranges to identify trends and patterns.

2.5 System Implementation and Validation

This chapter describes the practical realization of the Plant Up! sensor station. Hardware configuration, sensor calibration, and a real-world validation test are covered to demonstrate how the theoretical concepts from previous sections translate into a functional physical device.

2.5.1 Hardware Assembly

The physical assembly consists of connecting the chosen sensors to the ESP32-S3 microcontroller, securing a power source, and protecting the electronics with an enclosure.

Wiring and pin assignment

Connecting the selected components to the ESP32 requires bridging the physical hardware with the software via General-Purpose Input/Output (GPIO) pins. Depending on the complexity of a sensor, it may need one, two, or up to four pins to transmit its measurements.

The Plant Up! sensor station uses three different communication protocols. Knowing how these protocols work explains why some sensors can cleanly share the same pins, while others demand their own separate connections. In electronics, a shared connection is often called a “bus”. A bus acts like a single communication channel that multiple devices can connect to simultaneously, drastically reducing the number of pins required on the microcontroller.

The BH1750 light sensor and the MLX90614 infrared air temperature sensor both communicate over I^2C, the two-wire protocol introduced in Section 2.3.1. This protocol uses exactly two wires: a serial data line (SDA) and a serial clock line (SCL). Because the I^2C architecture allows multiple devices to sit safely on this shared bus, both sensors are wired in parallel to the exact same two GPIO pins: GPIO 8 for SDA and GPIO 9 for SCL.

The DS18B20 soil temperature probe uses the 1-Wire protocol. Because the timing signal is combined directly into the data line itself, it only needs a single communication pin. To prevent any timing collisions with the continuous I^2C traffic, this single data line is routed safely to its own isolated pin, GPIO 5.

The SD card module handles continuous data logging and needs a much higher transmission speed. It communicates over the Serial Peripheral Interface (SPI), a protocol requiring four dedicated lines: input (MISO), output (MOSI), a clock signal (SCK), and a chip select line (CS). Unlike the shared I^2C bus, this specific SPI setup does not share access with other components. Therefore, the SD card exclusively occupies four individual GPIO pins (10, 11, 12, and 13).

The final two components, the capacitive soil moisture probe and the EC sensor, do not use digital communication at all. They are analog devices that output a continuous electrical voltage. Because these raw voltage signals are sensitive to electrical noise, they cannot be shared on a common wire. As a result, each analog sensor gets its own dedicated ADC-capable pin (GPIO 1 and GPIO 4) to ensure the readings remain clean.

Table 2.1 summarizes the complete pin assignment resulting from these combined hardware requirements.

Table 2.1: GPIO pin mapping of the Plant Up! sensor station.

Component	**Protocol**	**Signal**	**ESP32 Pin**
BH1750 (Light)	I^2C	SDA / SCL	GPIO 8 / GPIO 9
MLX90614 (Air Temp.)	I^2C	SDA / SCL	GPIO 8 / GPIO 9
DS18B20 (Soil Temp.)	1-Wire	Data	GPIO 5
SD Card Module	SPI	MISO	GPIO 13
		MOSI	GPIO 11
		SCK	GPIO 12
		CS	GPIO 10
Soil Moisture Sensor	Analog	AOUT	GPIO 1
EC Sensor (Conductivity)	Analog	A0	GPIO 4
All sensors		VCC	3.3 V
All sensors		GND	GND

All sensors operate at 3.3 V, matching the ESP32's logic level requirements discussed in Section 2.3.3. No additional level shifters or voltage converters are needed on the sensor side.

Power supply

To make the device mobile, it is powered by a rechargeable lithium-ion battery. Because lithium batteries are extremely sensitive to overcharging and deep discharge, a TP4056 charge controller module is used as a safety bridge between the battery and the ESP32.

The TP4056 safely charges the battery via a standard USB connection and forwards power to the microcontroller. The ESP32 then uses its own internal voltage regulator to step this power down to a clean, stable 3.3 V, which is distributed to all the connected sensors. This power layout allows the Plant Up! station to run completely cord-free, which is a major practical advantage since plant pots are rarely placed directly next to wall outlets.

Enclosure

With the electronics wired and powered, the final hardware step is a protective physical shell. The assembled components are housed in a custom 3D-printed case made of

polylactic acid (PLA). Since the sensor station is designed strictly for indoor use and will not be exposed to extreme heat or weather, PLA offers a perfect balance of being lightweight, affordable, and easy to print.

Designing an enclosure for plant sensors comes with specific physical challenges. Most importantly, the main board and battery must be kept safe from wet soil and accidental water spills during watering. Therefore, all delicate electronics are elevated inside the upper part of the case, while only the waterproof cables for the soil probes exit through the bottom to be inserted into the dirt. Additionally, the case features necessary ventilation cutouts near the top so the infrared sensor measures actual room temperature rather than trapped heat radiating from the ESP32 board.

While the physical components are now active, some sensors output raw continuous voltages rather than standard units. These analog signals must be calibrated before the backend can interpret them.

2.5.2 Sensor Calibration

Digital sensors like the DS18B20 or BH1750 transmit readings in standard units such as degrees Celsius or lux. The analog capacitive soil moisture sensor, however, outputs a raw voltage that the ESP32 translates into a digital value ranging from 0 to 4095.

A raw hardware value of “2340” has no practical meaning for a user. To make the data useful within the mobile app, this ADC value must be mapped to a human-readable 0 % to 100 % scale, representing completely dry to completely saturated soil.

This translation uses a standard two-point calibration method. First, the sensor is inserted into air-dry potting soil to establish the baseline for 0 % moisture. Because capacitive sensors yield higher voltage in dry conditions, this dry reference typically records a high value (around 3100 in the test setup). Next, the soil is thoroughly watered until it is fully saturated. Because water has a much higher dielectric constant than air, the sensor’s voltage drops significantly. This wet reference point typically records a lower value (around 1400), representing 100 % moisture.

Once these upper and lower boundaries are known, the firmware uses a linear mapping function to convert any reading between 1400 and 3100 into a percentage.

While this relative calibration is effective for everyday plant care, it depends heavily on the specific substrate. Loose, sandy soil yields different baseline measurements than dense, moisture-retaining clay. This method produces a relative moisture index rather than an absolute VWC measurement. If a plant is repotted into a completely different soil mixture, the two-point calibration must be repeated to establish the correct boundaries.

2.5.3 Practical Validation

To verify the functional correctness of the system, the Plant Up! station was tested in a standard indoor environment. The goal was to observe how the hardware and interpretation logic handle physical environmental changes, specifically watering events and natural daily light cycles.

Figure 2.9 shows the system's reaction to a controlled watering event. Before watering, the soil moisture sensor transmitted a stable baseline of approximately 28 %. When water was added, the measured value immediately spiked, reaching a peak of over 85 % before gradually stabilizing as the water distributed throughout the soil. This confirms that the sensor reacts correctly to physical changes and that the calibration logic successfully translates the hardware signals into the expected percentage range.

Figure 2.10 illustrates how the station tracks continuous environmental shifts. During a one-hour afternoon window, the recorded light intensity declined steadily from approximately 600 lux to 0 lux. This matches the physical reality of a sunset, showing that the I^2C light sensor can reliably monitor daily cycles without significant signal noise.

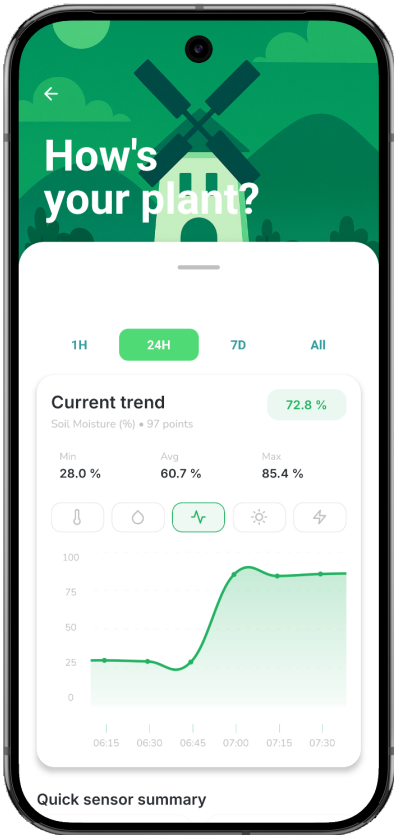

Figure 2.9: Sensor data visualization confirming a sharp increase in soil moisture directly following a watering event.

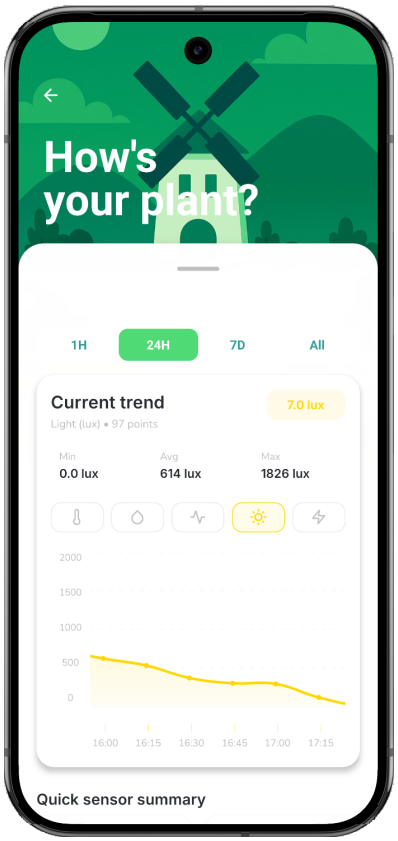

Figure 2.10: Ambient light tracking showing a smooth decline in lux values, reflecting fading afternoon sunlight.

Limitations

These observations validate the basic functionality of the Plant Up! hardware and software pipeline. However, the test was conducted over a short timeframe without laboratory reference sensors. The recorded 85.4 % moisture is a calibrated relative index, not an absolute scientific measurement. Given the project focus on providing accessible smart gardening feedback, this relative accuracy is sufficient for the application.

2.6 Conclusion

This thesis set out to answer a practical question: how can a low-cost, sensor-based system provide objective environmental data and actionable care recommendations for indoor plants? The chapters above have addressed this question step by step, from the biological foundations through sensor selection and firmware design to the interpretation logic that turns raw measurements into user-facing feedback.

The core argument of this work is that plant care does not need to rely on intuition. Soil moisture, temperature, humidity, and light are physical quantities with measurable ranges, and when those ranges are violated, the consequences are predictable. Chapter 2.2 established that both deficiency and excess cause specific, well-documented stress responses, and that human perception alone is too slow and too biased to detect these conditions reliably. Chapter 2.3 then showed that affordable consumer-grade hardware, specifically a capacitive soil moisture sensor, a BME280 climate sensor, and a BH1750 light sensor connected to an ESP32 microcontroller, can capture these variables with sufficient accuracy for threshold-based monitoring. The firmware pipeline described in Section 2.3.4 ensures that sensor data is read, filtered, and transmitted in a repeatable cycle with minimal power consumption.

Chapter 2.4 translated these raw measurements into decisions. By classifying each parameter into green, yellow, and red zones and combining them into a single health score, Plant Up! compresses complexity into feedback that requires no botanical knowledge to understand. The traffic-light system, the animated mascot, and the trend visualizations all serve the same purpose: making the plant's actual state visible at a glance.

The system is not perfect. The sensor station measures a single point in the pot, not the entire root zone. Calibration is substrate-dependent and produces relative values rather than absolute ones. The health score is a weighted approximation, not a scientific diagnosis. These are deliberate trade-offs. The goal was never laboratory-grade precision but a practical tool that prevents the most common care mistakes, especially overwatering, which remains the leading cause of houseplant failure.

Beyond technical performance, this hardware design is highly cost-effective. By combining standard off-the-shelf components, like the ESP32, standard digital sensors, and a simple 3D-printed PLA case, the material cost per station stays below 50 Euros. Many commercial smart-gardening systems force users into closed ecosystems and require monthly subscription fees for data access. Plant Up! avoids this completely. It deliv-

ers reliable environmental monitoring without any hidden costs or recurring fees. This proves the initial project assumption: effective plant care automation does not require expensive, specialized equipment. Additionally, because the system uses open standards, all captured sensor data remains entirely under the user's local control.

With the sensor station producing clean, timestamped measurements and the interpretation logic translating them into actionable feedback, the next question becomes: where should this data be processed, and how does it reach the backend? The following chapter by Abd-Ulrahman Behari addresses exactly this, comparing edge and cloud processing strategies for environmental sensor data in the context of smart gardening.

3 Edge vs. Cloud Processing in Smart Gardening: A Comparative Study for Environmental Sensor Data

Author: Abd-Ulrahman Behari

3.1 Introduction

The digitalization of houseplant care represents a growing intersection between the Internet of Things (IoT) and daily domestic life. As urbanization increases and living spaces become denser, the desire to maintain physical greenery has led to an emerging market of smart gardening devices. The problem, however, is that most of these products function as entirely isolated ecosystems. They ping the user with a localized alert when an individual plant needs water, but they fail to enable any broader community interaction. The PlantUp ecosystem was developed to explicitly address this gap. By fusing continuous environmental monitoring with a collaborative, gamified social platform, PlantUp aims to evolve routine plant care from an isolated chore into a data-driven, shared social experience.

3.1.1 The Architectural Challenge: Latency and System Dependability

Architecting a social smart gardening platform demands a highly capable backend infrastructure, one that can simultaneously manage heavy, continuous sensor data alongside complex user interactions. The core technical hurdle here boils down to two things: managing network delays and keeping the system online.

If an architecture relies exclusively on a centralized cloud server to process every single data point, it inherently introduces massive Round-Trip Time (RTT) delays. Consider how this works in a traditional setup: a device reads a sensor, packages that raw data, transmits it across the internet to a distant server, waits for the server to calculate if a threshold has been breached, and then waits for the command to bounce all the way back. This constant back-and-forth degrades the system's ability to respond in real-time. Even worse, if the Wi-Fi drops, the entire system breaks down because the device cannot think for itself.

These exact challenges form the architectural core of this thesis, which investigates the complex trade-offs between Edge-Centric and Cloud-Centric processing paradigms. The ultimate goal is to optimize both system responsiveness and long-term reliability. A detailed theoretical breakdown of these models is provided in Section 3.2.4.

3.1.2 Research Objective

This thesis addresses the following primary research question:

This research actively investigates the fundamental differences between Edge and Cloud processing within a modular backend architecture, and analytically isolates the specific trade-offs present when implementing massive environmental monitoring, as demonstrated directly by the PlantUp ecosystem.

To provide a concrete answer, PlantUp serves as an active comparative case study. The research isolates the processing logic and directly analyzes how shifting it between the edge device and the cloud impacts latency, infrastructure complexity, and overall system resilience. Interestingly, the chosen functional domain—houseplant monitoring—provides a unique testing environment. Soil moisture and ambient temperature are physiologically slow-moving metrics. A plant does not dry out in half a second. Because this domain inherently lacks the strict, millisecond-level reaction times demanded by heavy industrial machinery, the boundaries of cloud processing can be pushed further without immediately breaking the user experience.

3.1.3 Scope and Limitations

3.1.3.1 Computational Constraints of the Edge Device

The physical edge device utilized in this deployment is the ESP32 microcontroller, a component characterized by strict processing limitations and a highly constrained memory footprint. These tiny chips do not have the power to run massive databases. Because of these harsh physical limits, the hardware itself acts as the primary driver for the architecture, forcing a careful distribution of precisely how much data is aggregated on the board versus how much logic is pushed up to the cloud. The precise hardware characteristics, including network limitations and energy consumption profiles, are detailed extensively in Chapter X. This section focuses strictly on how those physical realities dictate the placement of processing logic.

3.1.3.2 Abstraction of the Transport Layer

To ensure an accurate and isolated comparative analysis, this thesis establishes rigorous scope boundaries. The evaluation focuses strictly on the location of the application logic (Edge computing versus Cloud computing). To prevent external variables from skewing the results, the transport layer is deliberately abstracted, with the Message Queuing Telemetry Transport (MQTT) protocol locked in as the baseline standard. Alternative application-layer protocols, such as CoAP or HTTP, are entirely excluded. By locking the communication pathway, the final architectural comparison is guaranteed to reflect only the impact of the computational placement.

3.2 Theoretical Framework

Before diving into the actual technical implementation of a scalable smart gardening system, a solid conceptual foundation must be established. Building an IoT infrastructure is complex. Standardized reference models exist specifically to help structure and organize that massive complexity. This section breaks down the required IoT architectures and explores modern cloud solutions, particularly Backend-as-a-Service (BaaS) platforms. Understanding these theoretical building blocks is a necessary first step before evaluating the specific design choices made in PlantUp.

3.2.1 The IoT World Forum Reference Model (7-Layer Model)

Systematically analyzing a massive, distributed IoT architecture requires a clear structural map. This thesis relies on the IoT World Forum 7-Layer Reference Model, which provides a holistic view integrating physical hardware, edge computing, and the cloud. This makes it the perfect baseline for understanding the core divide between "Fog"(Edge) and Cloud computing [206].

The model breaks the ecosystem into distinct levels. Layers 1 and 2 (Physical Devices and Connectivity) handle the actual edge hardware, sensors, and the network protocols used to transmit data. Layer 3 (Edge Computing) introduces local intelligence, allowing processing to occur as close to the source as possible to slash latency. Layers 4 and 5 (Data Accumulation and Abstraction) manage data buffering, storage, and transformation in the cloud, while Layers 6 and 7 (Application and Collaboration) execute high-level business logic and drive human interaction.

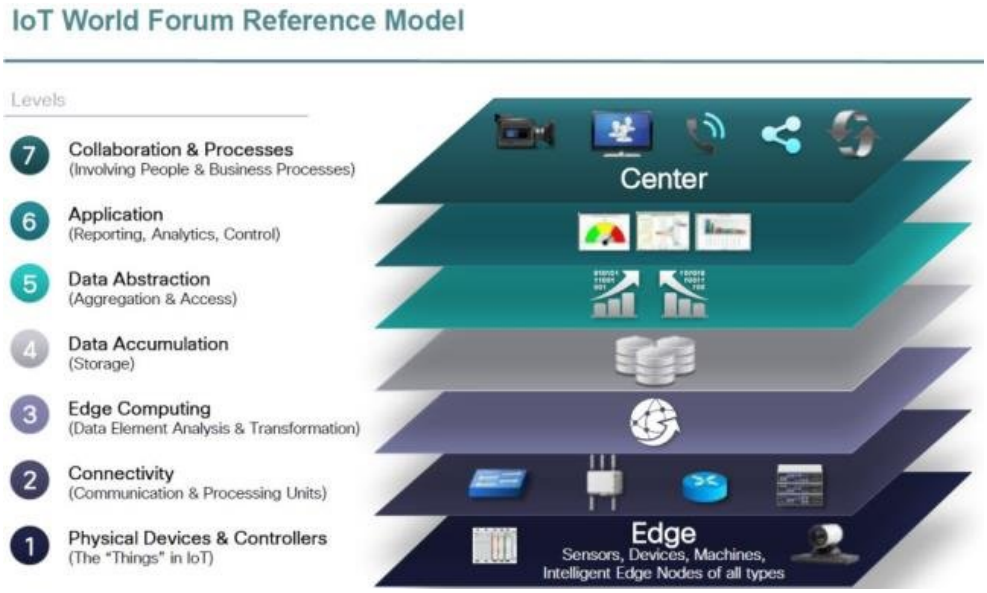

Abbildung 3.1: The 7-Layer IoT World Forum Reference Model [206]

A central architectural focus throughout this thesis is resource allocation: precisely distributing decision-making logic between local hardware (Layer 3) and the centralized

cloud (Layers 5 and 6). The 7-layer model establishes the rigid boundaries required to analyze this physical separation of computation.

3.2.2 Modern IoT Backends

Modern IoT backends require a highly capable foundation. They must handle two wildly different workloads simultaneously: structured relational data (like user accounts) and the relentless, high-velocity influx of raw sensor streams. An architecture that handles both flawlessly is required. This section explores Backend-as-a-Service (BaaS) platforms and specialized database extensions that solve this dual-workload problem—entirely without forcing the engineering team to manually provision and orchestrate complex server infrastructure.

3.2.2.1 Backend-as-a-Service (BaaS) and the SQL vs. NoSQL Paradigm

Backend-as-a-Service (BaaS) provides developers with ready-made abstract infrastructure: managed databases, authentication, and serverless APIs. While early BaaS platforms relied heavily on unstructured NoSQL document stores (like Firebase) for rapid prototyping, these architectures severely struggle with the complex analytical queries and strict relational integrity required in advanced IoT ecosystems [168, 169].

Modern BaaS architectures solve this by building upon battle-tested Relational Database Management Systems (RDBMS), like PostgreSQL. These SQL-native platforms enforce strict data integrity while simultaneously providing modern, WebSocket-driven real-time subscriptions that instantly push threshold alerts to massive mobile clients [170].

However, when selecting a BaaS for an IoT ecosystem, its extensibility must be rigorously evaluated. Standard relational databases guarantee transactional consistency (ACID properties), which is excellent for handling payments or securing passwords. But out-of-the-box, an RDBMS is practically useless at inhaling high-velocity, continuous sensor streams. Its performance will choke. Therefore, a BaaS is only viable for IoT if it natively supports powerful extensions—specifically time-series modules—to heavily optimize the core SQL engine for massive sensor ingestion. This allows for handling incredibly specialized workloads without the absolute nightmare of spinning up an entirely separate, isolated database instance [168, 170].

3.2.2.2 Time-Series Databases: Why Standard SQL Fails for Sensor Streams

Traditional relational databases excel at standard transactional workloads but utterly fail when confronted with high-velocity IoT sensor streams [172]. The root cause is massive index degradation.

Most SQL systems utilize standard B-tree indexes, which become notoriously expensive to maintain as table sizes explode with continuous incoming telemetry. As the database

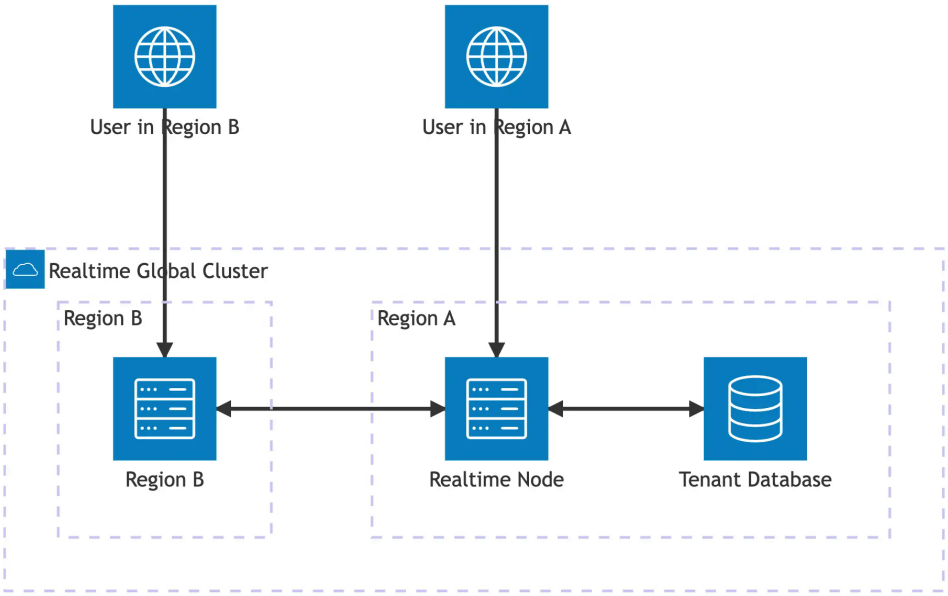

Abbildung 3.2: Realtime Architecture Flow in Modern BaaS [207]

forcefully inhales thousands of incoming measurements per second, the swollen B-tree index consumes massive amounts of RAM, dramatically dragging down read query performance and corrupting overall server stability. To survive this massive ingestion rate natively, sensor logs cannot be treated like standard transactional records. Specialized time-series extensions that architecturally replace bloated standard indexes with optimized time-based partitioning must be implemented [171].

3.2.2.3 The TimescaleDB Extension: Hypertables and Chunking Explained

This structural necessity is exactly where technologies like TimescaleDB come into play. TimescaleDB is an incredibly powerful extension for PostgreSQL. It takes a standard, traditional RDBMS and mathematically morphs it into an insanely fast, time-series-optimized database. Crucially, it achieves this while keeping the native SQL interface completely intact. Engineers don't have to learn a proprietary, convoluted query language; they just write standard SQL.

TimescaleDB achieves this aggressive performance through the use of *hypertables*. To the application querying the database, a hypertable looks and acts exactly like one massive, continuous SQL table. But beneath the surface, TimescaleDB is silently executing a process called *chunking*. It automatically slices the incoming time-series data into smaller, highly manageable physical pieces based on specific time intervals (e.g., one chunk per day or week) [173, 174, 175, 176].

Chunking brilliantly exploits a concept known as temporal locality. In sensor monitoring, almost all queries and inserts target the immediate present. Users inherently care about what their plant is doing right now, not what it did three years ago on a Tuesday. By forcing all new inserts into the current, active "chunk,"the database avoids dragging its operations through historical data. When a query is finally made, TimescaleDB simply skips over any completely out-of-bounds chunks. This enables the database to ingest massive amounts of data and return range queries instantaneously, even when the total row count stretches into the billions.

Furthermore, these extensions powerfully utilize continuous aggregates. Instead of forcing the database to manually calculate the average daily temperature across millions of rows every single time the user opens the app, the database precomputes these summaries quietly in the background. It effortlessly trades a tiny amount of storage space for aggressively fast analytical queries. Finally, a retention policy can be configured to automatically compress or delete ancient chunks, ensuring that storage growth stays strictly controlled while the most relevant data remains lightning fast to access [174, 175, 176, 173].

[Figure 3.3: Hypertable vs. Normal Table Storage]

In short, compared to a standard relational table, a hypertable's chunked layout aggressively reduces the brutal I/O and indexing burden on the server. Developers get the absolute best of both worlds: they can write standard SQL, while quietly leveraging heavy-duty time-series optimizations beneath the hood [176, 173, 174].

3.2.3 Microservices in IoT

As an IoT system scales up, cramming every single feature into one massive, tangled backend application becomes a massive liability. Monolithic architectures deploy as a single, rigid unit; if one component critically fails, the entire backend often goes down with it.

To ensure long-term backend stability and aggressive development speed, modern scalable architectures utilize microservices. This architectural style actively splits system functionality into small, highly focused, and independently deployable services. By adopting microservices, development teams can strictly separate external device communication (like MQTT brokers) from highly internal user-facing features (like the social feed). This absolute isolation forces clear, rigidly defined interfaces and permanently prevents messy, hardware-specific networking protocols from tangling with high-level core business logic [153].

3.2.3.1 The Gateway Pattern: Bridging Edge and Core Services

In complex software architecture, design patterns are established, reusable solutions to very common engineering problems. Within distributed microservices, the Gateway pattern (frequently discussed simply as the API Gateway) acts as the definitive security and protocol boundary. It serves as the single, highly controlled entry point positioned explicitly between the external hardware clients—in this case, thousands of distributed ESP32 microcontrollers—and the internal, protected backend database [153, 155, 154].

A dedicated Gateway is logically critical for IoT security. Without it, if field-deployed microcontrollers were explicitly allowed to directly access the core database APIs, the database API keys would have to be hardcoded directly into the device firmware. This is an immense, unforgivable security vulnerability. Physical hardware left in the wild can be stolen, manipulated, or reverse-engineered to extract those critical keys.

Instead, a middleware Gateway acts as a highly protective buffer. The edge sensors blindly publish their raw telemetry to a message broker, completely unaware of the actual database's existence. The Gateway microservice sits independently behind the broker, safely inside the protected cloud network. It securely subscribes to the incoming hardware messages, translates the lightweight MQTT payloads, and carefully formats them into structured, secured database operations [154].

Beyond merely locking down security, this decoupling grants the system extreme structural flexibility. As the platform massively scales, engineers can completely overhaul the internal backend infrastructure without ever needing to push risky, expensive firmware updates to thousands of physically deployed devices. Furthermore, the architecture becomes immediately geographically scalable. Additional MQTT brokers can be dynamically deployed around the world to ensure devices maintain incredibly low latency, while local Gateways securely and efficiently funnel everything back to the core data instances [156, 154].

In summation, the Gateway pattern brilliantly ensures that every single byte of device traffic is actively validated, translated, and shaped before it ever physically touches the central database. It entirely eliminates the risk of exposed API keys while keeping the overall ecosystem highly modular [154].

3.2.3.2 Service Inter-communication: Synchronous vs. Asynchronous Patterns

Microservices rely on strict data exchange mechanisms, which fundamentally dictate total system reliability. Synchronous communication (like traditional HTTP REST) forces a requesting service to actively block and wait for a reply. In a massive IoT sensor network, this rigid coupling is fatal; any minor latency delay downstream immediately cascades upstream, causing thread exhaustion, timeouts, and total system crashes under heavy load [157].

To survive intense traffic volatility, highly scalable architectures utilize robust asynchronous communication. In an event-driven model, an upstream service publishes a highly lightweight message to a persistent broker and instantly resumes its own tasks, utterly refusing to block or wait for a reply [160]. This non-blocking paradigm acts as an architectural shock absorber. If 5,000 disconnected edge devices simultaneously reconnect and forcefully flush hours of queued telemetry, an asynchronous Gateway securely drops the traffic into an isolated queue, allowing the heavily loaded database to calmly and methodically pull the data at a sustainable rate. This intentional decoupling absolutely guarantees profound system resilience under extreme stress [159].

3.2.4 Edge vs. Cloud

Deep IoT architectural design is ultimately defined by one incredibly strict constraint: the physical location of the computational execution. Deciding whether to forcibly place the operational logic in the massive cloud or bake it directly into the tiny microcontroller entirely dictates the system's latency, its offline reliability, and its incredibly strict power consumption limits.

3.2.4.1 Defining the Cloud-Centric Model

A cloud-centric architecture centralizes everything. The heavy computational lifting, the massive long-term data storage, and the highly complex decision-making algorithms all live exclusively within remote data centers. The physical devices deployed at the edge act entirely as basic "dumb sensors."They do nothing but immediately read the raw environmental values and blindly forward them upwards to the cloud, patiently waiting for the server to interpret the exact data and decide what to do next [182, 170].

The primary, undeniable advantage of this centralization is raw, nearly infinite computational power. Engineers can seamlessly execute highly complex time-series aggregations, integrate resource-heavy AI models, and beautifully manage rich social features

without ever caring about the strict hardware limitations of the tiny embedded chips sitting on the users' desks. But there is a massive catch. A purely cloud-centric model is violently dependent on continuous internet connectivity. It is severely subject to explicit Round-Trip Time (RTT) network latency. While a delay of a few seconds is perfectly acceptable for merely logging an ambient temperature reading, it becomes completely disastrous if the system is attempting to execute a real-time control loop, like instantly closing a rapidly flooding water valve [183, 184, 182, 170].

3.2.4.2 Defining the Edge-Centric Model

An edge-centric architecture pushes completely back in the opposite direction. It forces aggressive computation, data aggregation, and autonomous, critical decision-making all the way down the pipeline, injecting the massive logic directly onto the local device itself. The microcontroller no longer waits for orders. It interprets its own localized readings, evaluates its internal thresholds, and immediately triggers highly local actuation—such as turning on a warning LED or aggressively spinning up a water pump—without ever waiting to consult the massive cloud [185, 186, 170].

Because the device successfully calculates the logic completely locally, this model impressively guarantees offline fault tolerance and ultra-low latency. If the main server crashes or the Wi-Fi entirely drops, the isolated edge node still independently protects the plant. Furthermore, it aggressively minimizes massive bandwidth consumption, securely transmitting only highly summarized burst events rather than continuous, brutal streams of raw data. However, shoving complex, high-level business logic entirely onto an edge device introduces severe, ongoing developmental nightmares. Basic firmware updates mysteriously transform into dangerous, highly complex deployment operations. Remote debugging becomes virtually and catastrophically impossible. And critically, developers are violently forced to write highly optimized C++ code that perfectly fits within the claustrophobic CPU, RAM, and battery constraints of cheap embedded microcontrollers [182, 185, 170, 186].

3.2.4.3 The Spectrum of Logic Distribution: "Thick Edge" vs. "Thin Edge"

In the messy real world of dedicated systems engineering, developers rarely pick a strict, unforgiving binary between Edge and Cloud. Instead, the logical allocation of profound architectural responsibilities slides highly dynamically along a continuous spectrum. Distinct paradigms constantly balance the trade-off between absolute localized hardiness and massive centralized development agility.

A **"Thick Edge"** architecture knowingly and willingly delegates substantial, heavy processing power, strict state management, and autonomous control directly onto the physically deployed microcontroller. This firmly localizes the highly critical path. The system gains incredible ultra-low latency response times and the absolute ironclad guarantee of offline operational survival. But this rugged, impressive independence comes at a tremendously steep price. The engineering team must heavily invest in significantly higher-powered, vastly more expensive hardware nodes. Furthermore, updating a simple, ba-

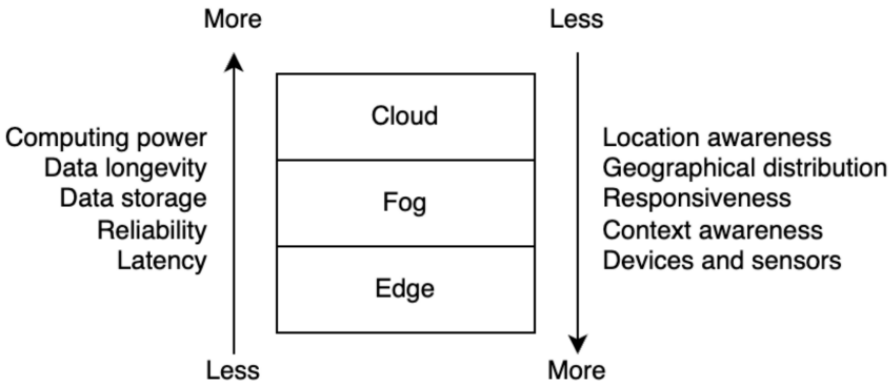

Abbildung 3.3: Visual Comparison of Cloud vs. Edge Architectures (Source: Self-made)

sic logical threshold forcefully requires executing incredibly risky, profoundly complex over-the-air (OTA) firmware deployments across the entire massively distributed fleet, causing brutal ongoing maintenance friction [182, 185, 186].

In direct contrast, a **"Thin Edge"** strategy aggressively and uncompromisingly centralizes logic. It utilizes the deployed edge hardware almost entirely as incredibly minimal data-ingestion vectors. The microcontroller focuses its highly limited energy purely on immediate sensory acquisition—performing incredibly simple analog-to-digital conversions and safely formatting basic transmission payloads. It instantly and effortlessly delegates all highly complex logical evaluation entirely upwards to the remote, incredibly massive cloud infrastructure. Yes, this completely binds the system's functional utility to sustained, continuous internet connectivity. But it allows the deployment of structurally incredibly simple, amazingly cost-effective embedded hardware. Most importantly, it keeps the dynamic core business logic fiercely centralized. Development teams can instantly iterate upon highly complex algorithms, deploy entirely new gamification mechanics, and heavily tweak AI models on their massive backend completely seamlessly, entirely untouched by the severe limitations of the physical edge hardware framework [170, 181].

Ultimately, these two opposing, massive distributions highlight a profound architectural reality. Excellent IoT system design requires brutally evaluating whether the harsh physical constraints of a specific domain necessitate the incredibly high cost of pure localized autonomy (Thick Edge), or whether the project demands the supreme developmental scalability provided entirely by massive centralized logic (Thin Edge). This established, highly theoretical framework serves as a definitive, ironclad analytical context. Moving forward, it will unquestionably be the foundational lens through which the explicit, concrete structural decisions successfully made within the genuine PlantUp ecosystem implementation are evaluated.

3.3 The PlantUp Architecture

PlantUp takes the theoretical frameworks discussed in Chapter 3 and translates them into a concrete, functioning architecture. This smart-gardening ecosystem was not built in a vacuum. The design strictly follows the IoT World Forum layered model and relies on a robust Gateway pattern to manage all hardware communication securely. The most defining decision for PlantUp, however, is the implementation of a "Thin Edge"paradigm. All the complex processing logic is intentionally offloaded straight to the cloud. This deliberately trades offline autonomy for massive computational flexibility and the ability to run heavy, long-term analytics.

While the design is strongly inspired by the strict isolation found in microservices, the backend is practically deployed as a *modular monolith* using the Supabase Backend-as-a-Service (BaaS) platform. This strategy provides the best of both worlds: services are kept logically separated, but the deployment simplicity of managing a single, unified database environment is heavily leveraged.

3.3.1 System Overview

The architecture connects a physically deployed, sensor-equipped edge device to a massive, scalable cloud backend. That backend then serves synthesized data directly to a React Native mobile application. This constant pipeline ensures continuous remote accessibility and instant data synchronization.

3.3.1.1 High-Level Architecture Pipeline

Down at the physical layer, PlantUp utilizes a standard ESP32 microcontroller deployed right into the plant's environment. The ESP32 constantly interfaces with its attached analog and digital sensors (reading soil moisture, temperature, humidity, and light intensity). It periodically takes these raw voltage readings and structures them into standardized JSON payloads. In the formal terminology of the 7-Layer IoT model, these components represent Layer 1 (Physical Devices) and Layer 3 (Edge Computing).

A major design decision in this initial pipeline was the strict, exclusive reliance on Wi-Fi and MQTT for all device-to-cloud communication (Layer 2 - Connectivity). Direct device-to-smartphone technologies like Bluetooth Low Energy (BLE) were actively rejected. While BLE is incredibly energy-efficient, it severely restricts real-time accessibility; one must be physically standing next to the plant to see its data. It also introduces massive user-experience friction by forcing frequent, annoying Bluetooth pairing sessions. The Wi-Fi-to-Cloud pipeline guarantees a seamless "connect onceönboarding experience, allowing users to instantly monitor their plant's health from anywhere in the world. (A detailed quantitative benchmark proving the massive performance superiority of MQTT over traditional synchronous REST for this exact pipeline is presented in Chapter X.)

The core infrastructural tier is the Supabase cloud backend. It functions simultaneously

as both Layer 4 (Data Accumulation) and Layer 5 (Data Abstraction). To handle the massive difference in workloads, the database is violently partitioned into two completely distinct logical schemas. The *Social* schema entirely governs the relational data—users, gamification points, and interpersonal posts. In total isolation, the *Micro* (telemetry) schema is dedicated solely to storing the incredibly fast, high-velocity time-series sensor data and raw device states.

This strict dual-schema separation drastically reduces the query complexity for the React Native frontend application (Layer 6 and 7). When a user opens their profile screen, the app independently retrieves UI data solely from the Social schema. It doesn't have to parse through millions of heavy telemetry records just to load an avatar. End-to-end, this pipeline guarantees one thing: sensor measurements generated by the ESP32 traverse the MQTT gateway securely, are cleanly partitioned deep within the Postgres database, and immediately become available for the mobile application to consume.

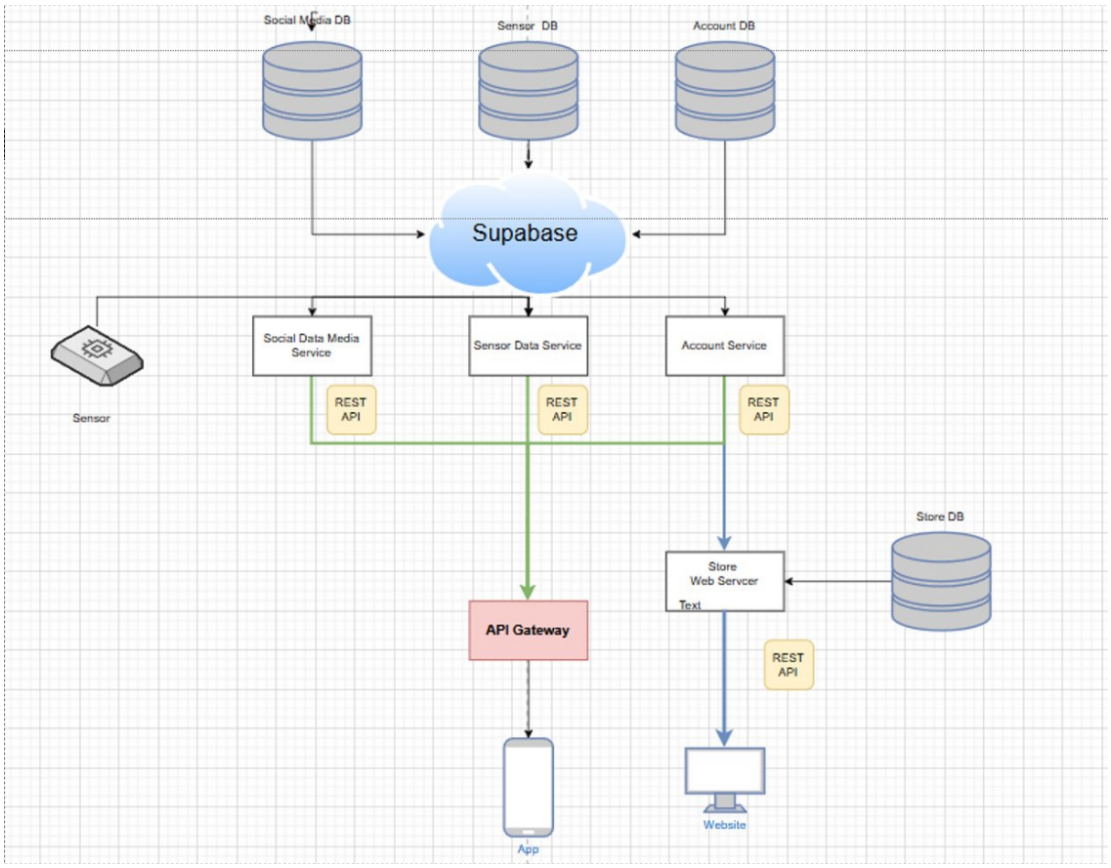

Abbildung 3.4: High-Level System Architecture Diagram (Source: Self-made)

The high-level diagram visualizes this continuous ingress pathway: Sensors → ESP32 (Wi-Fi) → Cloud Backend (Supabase) → React Native App. It explicitly highlights the hard, modular separation between the raw telemetry (Micro schema) and the application logic (Social schema).

3.3.2 The Application and Gateway Layers (Cloud)

PlantUp's cloud backend heavily applies modular design principles. The messy hardware ingestion pipeline is deliberately separated from the clean, user-facing social features. Every single logical component is built exclusively around a specific domain concern—like calculating gamification XP, managing unverified edge devices, or processing AI insights. Because they rapidly communicate via strictly defined interfaces, these components can be developed and tested completely independently.

3.3.2.1 Application Layer Services: Execution and Communication

In the application layer, PlantUp deploys user-facing services natively within the Supabase ecosystem, vehemently avoiding entirely standalone Docker infrastructure. The React Native mobile app communicates directly with the database using native REST APIs and WebSocket subscriptions.

Critically, heavy application logic is tightly coupled to the persistence layer for speed. The gamification engine is built entirely using native PostgreSQL triggers, ensuring immediate, zero-latency execution. The AI-driven "Daily Mentorütilizes serverless Supabase Edge Functions. When triggered via RPC, an ephemeral container instantly spins up, actively evaluates recent TimescaleDB telemetry, queries the external ChatGPT API, and then immediately dies.

By brilliantly utilizing Edge Functions and database triggers, these heavy auxiliary features remain explicitly decoupled from the core IoT ingress. If the external ChatGPT API experiences massive API lag, it only aggressively impacts the isolated Edge Function, allowing the user to maintain full, unhindered real-time access to their critical sensor telemetry.

3.3.2.2 Direct-to-Database Client Architecture and Security

By actively leveraging the official Supabase SDK, the React Native application establishes a direct, encrypted communication channel straight into the Postgres database, completely destroying the need to construct and host massive intermediary custom REST API middleware.

Security is strictly maintained via Supabase Auth and JSON Web Tokens (JWT). The structural integrity of this direct-access model is enforced exclusively through PostgreSQL's ridiculously powerful Row Level Security (RLS) policies. These SQL rules act as an absolute bouncer at the database level, mathematically guaranteeing that users possess the authority to only query or mutate their specific, personal social and telemetry data [194, 195, 188].

3.3.2.3 Device Management Service (Gateway)

Operating entirely independently of those shiny user-facing application services is the centralized Device Management Service. This heavily restricted component serves as the concrete, physical realization of the Gateway pattern discussed in theory. It acts as the absolute security and protocol boundary standing between the vulnerable, external hardware devices and the highly protected internal database.

The primary mandate of this aggressive service is to enforce a brutal boundary between the messy time-series telemetry data (TimescaleDB) and the standard, clean relational social data (PostgreSQL). PlantUp strictly prevents the mobile application from ever executing raw SQL queries directly against the high-velocity, multi-million-row sensor tables. Instead, the Device Management Service ingests and safely normalizes the unrefined hardware state—tracking metrics like active Wi-Fi connectivity, secure hardware registration, and sensor calibration curves.

By absorbing the raw MQTT payloads straight from the broker, this isolated gateway service translates weird, device-specific telemetry into highly consistent database inserts. It subsequently exposes a simplified, read-only API interface. The frontend React Native application interacts solely with this abstracted API to fetch the current plant conditions. It never talks to the raw sensor tables, completely shielding the mobile app from chaotic underlying protocol specifics or massive TimeScaleDB chunk management. This extreme isolation guarantees that if an edge device drops offline or starts rapidly firing malformed MQTT payloads, the failure cannot possibly cascade upwards into the application layer.

3.3.3 The Data Persistence Layer (Supabase)

To simultaneously manage clean relational application state, chaotic high-velocity time-series telemetry, and massive user-generated image objects, PlantUp utilizes a heavily consolidated PostgreSQL environment via Supabase. This specific infrastructure perfectly aligns with the project's absolute requirement: both traditional relational modeling and highly advanced time-series extensions are needed, but they must be contained within a single, unified, managed instance.

3.3.3.1 Unified Infrastructure with Schema-Level Isolation

Maintaining a single infrastructural source of truth vastly simplifies the initial deployment phase and drastically speeds up cross-schema query execution. However, putting everything in one database is incredibly dangerous if it isn't organized. PlantUp deliberately, and violently, isolates its data domains. The database is strictly partitioned into entirely distinct logical schemas. The social schema encompasses users, plants, and high-level interaction metrics. Sitting right next to it is the telemetry (micro) schema, entirely dedicated to hoarding raw, unedited, and derived environmental sensor logs.

This strict schema-level isolation provides immediate organizational clarity and massive

security benefits. It permits incredibly simplified Row-Level Security (RLS) policies. It ensures that users can only ever access their private telemetry data while simultaneously enabling public read access to community-driven social tables. Critically, keeping both schemas locked within the same unified Supabase instance permits elegant API abstraction and incredibly efficient cross-schema SQL queries. The system can instantly join immediate soil moisture metrics with user-defined profile alert thresholds, completely avoiding the horrific network latency of querying two completely disparate database servers.

3.3.3.2 Time-Series Performance Optimization via TimescaleDB

An absolute, non-negotiable requirement of the PlantUp mobile application is the near-instantaneous retrieval of the most recent environmental data. Users will not wait five seconds for a chart to load. Standard relational tables are fantastic for managing discrete entity relationships (like linking a user to a post). But they suffer massively from index bloat and disastrously degraded read performance when subjected to the relentless, high-frequency insert rates generated by continuous hardware telemetry.

To solve this fatal bottleneck, sensor measurements—including thousands of raw soil moisture, ambient temperature, and light intensity readings—are forcibly ingested directly into TimescaleDB hypertables. TimescaleDB is an advanced extension operating natively right inside the Postgres environment. Instead of maintaining one massive, bloated index, TimescaleDB structurally segregates this continuous river of data into contiguous, highly manageable temporal chunks based on time. This clever architecture effectively bypasses the performance-killing bottlenecks of scanning deep B-tree indexes on massive tables. It completely side-steps the problem by only maintaining very shallow, incredibly fast indices within tiny time windows.

Because the mobile application almost exclusively requests data from the most recent chronological window (e.g., "Give me the last 24 hours of moisture trends"), these SQL queries execute almost entirely against fast, in-memory active chunks. It doesn't even bother scanning the disk for last year's data. This optimization directly results in drastically reduced latency. This exact speed is essential for the fluidity of the historical charts and the incredibly rapid generation of the complex Daily Mentor AI insights. By guaranteeing that massive, analytical time-series workloads never accidentally degrade the performance of quick operational workloads (like loading the social feed), PlantUp's persistence layer successfully achieves a perfect balance between high-velocity hardware ingestion and incredibly snappy mobile application queries.

3.4 Methodology

This section describes exactly how the architectural decisions made for the PlantUp ecosystem were objectively evaluated. Back in Chapter 3, the theoretical superiority of a Cloud-Centric, “Thin Edge” model was established for this specific botanical domain. But theories alone aren't enough. To empirically validate this massive design choice,

controlled experiments were built. The goal was to actively measure how changing the location of the operational logic directly impacts system latency, raw scalability, and backend resilience.

3.4.1 Experimental Control Framework

Testing the deployed architecture in isolation was actively avoided because without a comparative baseline, the isolated performance metrics provide absolutely no contextual value. Instead, a strict experimental control group was constructed to facilitate a direct, head-to-head comparative analysis.

3.4.1.1 Architecture A: Cloud-Centric (Deployed Variant)

Architecture A is the primary system under test. This represents the *Thin Edge* paradigm fully deployed and detailed heavily in Chapter 4. In this variant, the physical ESP32 hardware functions completely as a "dumb"data ingress vector. It does nothing but generate raw JSON payloads packed with simple sensor telemetry. All the heavy lifting—threshold evaluation, conditional logic, and complex historical aggregation—is forced straight up into the centralized Supabase cloud infrastructure.

3.4.1.2 Architecture B: Edge-Centric (Control Variant)

Architecture B represents an aggressive *Thick Edge* mock-up. This was built strictly to serve as the comparative control group. In this variant, the threshold comparisons and autonomous decision-making were ripped out of the cloud and shoved directly back onto the ESP32 firmware. The device still transmits data to the cloud, but the payloads look entirely different. Instead of high-frequency raw telemetry blasts, it sends quiet, asynchronous decision logs and state summaries. By directly pitting Architecture B against the deployed Architecture A, the exact latency penalty and the brutal structural trade-offs caused by the Thin Edge approach can be mathematically calculated.

3.4.2 Implementation Constraints

To guarantee that the absolute only independent variable in this experiment was the physical *location* of the operational logic, both architectures were locked down using incredibly strict bounds:

- **Hardware:** Both architectural variants strictly execute on the exact same physical ESP32 microcontrollers.
- **Transport:** Both variants utilize completely identical MQTT message brokers and Wi-Fi environments.

- **Payload Anatomy:** Both variants serialize their data into rigidly standardized JSON objects. They differ exclusively in transmission frequency and semantic content (sending raw measurements versus calculated decision logs).
- **Persistence:** Both hardware instances write seamlessly into identical Supabase endpoints, perfectly targeting the same PostgreSQL schemata and massive TimescaleDB hypertables.

3.4.3 Measurement Methodology and Environment

To ensure these performance benchmarks can be perfectly reproduced, the testing environment and diagnostic methodologies were fully standardized.

3.4.3.1 Testing Environment

The heavy backend infrastructure evaluating the massive payload spikes was hosted on a **[INSERT SERVER / NETWORK SPECS HERE]** environment. This ensured a rock-solid operational baseline, entirely devoid of transient local computing bottlenecks. To define the stress tests, the load generation was ruthlessly orchestrated using **[INSERT TESTING TOOL HERE]**. Highly diverse traffic patterns originating from entirely outside the local networking subnet were deliberately simulated. This was necessary to accurately recreate the messy, real-world latency inherently found in cellular and remote Wi-Fi networks.

3.4.3.2 Latency Calculation

Here is a critical detail: end-to-end ingestion latency is absolutely not calculated using standard HTTP round-trip bounds. This exclusion is mandatory because the entire MQTT ingress pathway is inherently asynchronous and does not await a server reply.

Instead, the exact latency was empirically derived purely through differential timestamping. When the ESP32 generates a reading, it injects an incredibly precise, localized generation timestamp directly inside the JSON payload. Upon successful ingestion and the final insertion into the TimescaleDB hypertable, Supabase immediately slaps a `created_at` server-side timestamp onto the row. The absolute latency per transaction is then easily calculated by simply subtracting the hardware's generation timestamp from the PostgreSQL finalization timestamp. This provides the indisputable, total time required for Wi-Fi data transmission, MQTT broker handling, massive gateway normalization, and the final database write.

3.4.4 Test Scenarios

The controlled experiments were aggressively structured into two highly distinct scenarios. These were explicitly designed to brutalize the system under both normal operational cadence and extreme edge-case volatility.

3.4.4.1 Scenario 1: Baseline Operational Thresholds

Scenario 1 serves as the boring, everyday benchmark reference. It safely simulates healthy, entirely standard network conditions. The sole objective here is to record the native latency variations and standard end-to-end response times under normal snapshot intervals (e.g., payloads lazily transmitted every 5 to 10 minutes). The ultimate goal is to absolutely verify that the fundamental network latency introduced by offloading the heavy logic to the cloud remains comfortably within the incredibly loose real-time requirements of basic botanical monitoring.

3.4.4.2 Scenario 2: Infrastructure Stress Test (Disaster Recovery Spike)

Scenario 2 violently shifts the focus. Simple latency becomes irrelevant as the test directly attacks backend resilience. The objective is to evaluate exactly how the unified, monolithic database violently degrades under intense transactional stress.

To physically push the infrastructure to its structural limits, the test suite mercilessly injects an immediate, simultaneous burst of 1,000 concurrent write operations directly into the backend API. It is incredibly critical to note that this specific volume absolutely does not represent a standard “Daily Active User” (DAU) load. Under perfectly normal operation, 1,000 daily users generating 5-minute snapshots produce an exceptionally low, boring continuous ingestion rate of less than one single write per second.

Instead, this massive concurrency spike mathematically models a severe *Disaster Recovery* event. Imagine a widespread ISP or Wi-Fi outage ending at a massive apartment complex. This causes an entire fleet of 1,000 completely disconnected ESP32 devices to instantly regain connectivity and frantically flush their locally queued payloads at the exact same millisecond. By violently stressing the system with this theoretical worst-case synchronization nightmare, database lock contention, massive processor queuing, and exactly how rapidly the architecture normalizes once the peak finally subsides can be clearly observed.

3.5 Results and Data Analysis

3.5.1 Latency Analysis

3.5.1.1 Edge Response Time (<50ms) vs. Cloud Round Trip (>200ms + DB Query Time)

- Quantitative analysis, edge speed, sub-50ms response, cloud delay, >200ms latency, network overhead, database query time, round-trip metrics, comparison charts, performance penalty, real-time constraints, user experience impact
- **[INSERT GRAPH: Comparison Bar Chart of Alert Generation Time (Edge vs. Cloud)]**
- Source: `https://ifactoryapp.com/blog/real-time-ai-cloud-latency-manufact`

3.5.2 Database Performance

3.5.2.1 Impact of High Ingestion Rates on C# Service Query Performance

- Performance impact, ingestion throughput, query latency, lock contention, C# service load, resource usage, CPU/Memory metrics, read-write conflict, optimization analysis, database tuning, concurrency issues
- **[INSERT GRAPH: Query Latency vs. Ingestion Rate]**

3.5.2.2 Storage Efficiency: Raw Data (Cloud Decision) vs. Batched Aggregates (Edge Decision)

- Storagemetrics, raw data volume, aggregation efficiency, payload size, storage costs, bandwidth consumption, cloud storage, batched transmission, data compression, economy of scale, archival strategy
- **[INSERT CHART: Projected Storage Costs Over Time]**
- Source: `https://metadeskglobal.com/from-raw-sensor-data-to-intelligent-a`
  `https://stonefly.com/blog/iot-data-storage-architectures-databases/`

3.5.3 Reliability and Resilience

3.5.3.1 Alert Delivery Success Rates during a simulated 1-hour internet outage

- Reliability metrics, delivery success analysis, outage simulation, local data buffering, resync capability, data integrity, critical failure, connectivity loss, edge resilience, robust design, message queuing
- **[INSERT TIMELINE: Data Sync Recovery after 1-Hour Outage]**
- Source: `https://www.opal-rt.com/blog/5-essential-computer-simulation-te`

3.5.4 Gamification Synchronization

3.5.4.1 The “Lag” between Action (Watering) and Reward (XP Update in Social Service)

- User experience, feedback delay, action-reward loop, gamification lag, sync latency, psychological impact, interface responsiveness, XP updates, social satisfaction, system consistency, near real-time
- **[INSERT DIAGRAM: Sequence Diagram of Action-to-Reward Delay]**
- Source: `https://supabase.com/docs/guides/functions`

3.6 Discussion

The experimental stress tests and massive load evaluations conducted throughout Chapter 5 forcefully transition the theoretical trade-offs between Edge and Cloud architectures into concrete, undeniable empirical data. The following subsections aggressively synthesize the results of those massive baseline and stress tests to evaluate the true hardiness of the "Thin Edgeärchitectural design implemented at the absolute core of the PlantUp ecosystem.

3.6.1 Validating the "Thin EdgeÄrchitecture

3.6.1.1 Latency vs. Edge Autonomy

The empirical results from Scenario 1 indisputably demonstrate that aggressively centralizing logical evaluation entirely in the cloud is a highly viable strategy for the PlantUp system. During strictly baseline, normal operational conditions, the exact measured end-to-end latency for the monolithic Cloud-Centric variant (Architecture A) was **[INSERT LATENCY VALUE HERE] ms**.

While the Edge-Centric control variant (Architecture B) processed all specific thresholds directly on the local hardware, entirely saving the round-trip network time, the **[INSERT LATENCY VALUE HERE] ms** penalty introduced by the actual Thin Edge approach remains shockingly negligible in reality. The specific functional domain of botanical physiology—patiently monitoring extremely slow-moving metrics like soil moisture and ambient temperature evaporation—simply does not require dangerous, aggressive millisecond-level actuation. Therefore, the cloud latency is completely imperceptible to the end user. This successfully validates the entire architectural decision to violently prioritize massive backend analytics and rapid development speed over heavily localized, incredibly stubborn edge hardware autonomy.

Because absolutely all critical decision-making is fiercely centralized deep in the Supabase backend, the system inherently and effortlessly supports incredibly rich cross-user social features and aggressive Daily Mentor AI integrations right out of the box. The massive results clearly show that the profound structural benefits of this centralized, Thin Edge approach drastically and undeniably outweigh the incredibly minor fractional latency costs temporarily associated with a simple cloud round-trip.

3.6.2 Database Performance and Resilience

3.6.2.1 Resilience Under Stress

In Scenario 2, the monolithic database architecture was violently evaluated under a simulated, catastrophic disaster recovery spike composed entirely of 1,000 completely concurrent data writes. The absolute results clearly indicate a massive peak ingestion throughput of **[INSERT THROUGHPUT VALUE HERE] writes/sec**. Following the initial, incredibly violent network contention, the internal queue aggressively normalized within exactly **[INSERT NORMALIZATION TIME HERE] seconds**.

This unequivocally confirms that the unified Supabase Postgres environment, heavily coupled with the wildly advanced TimescaleDB extension strictly tuned for time-series ingestion, possesses more than enough raw structural resilience to effortlessly handle shockingly intense traffic spikes without ever cascading into a fatal system failure.

Because PlantUp does not execute incredibly strict, real-time industrial control loops (e.g., stopping a heavily spinning saw blade), the minor, asynchronous latency introduced by aggressively pushing cloud-based threshold evaluations against TimescaleDB is entirely and profoundly acceptable. The system successfully absorbed the massive, sustained burst while flawlessly maintaining backend stability across the board.

3.6.3 Complexity vs. Operational Maintenance

3.6.3.1 The Trade-off of Edge Complexity

A widely known IoT architectural trade-off, universally supported by modern literature, focuses on sheer development speed. Massive Thin Edge systems significantly reduce total developmental complexity by fiercely centralizing massive logic, albeit at the obvious, known cost of highly localized hardware device resilience [200, 201, 190].

Attempting to carefully maintain dangerous, highly conditional logic simultaneously on the strict edge level (writing optimized C++ directly on the ESP32) and high up in the massive cloud (executing complex SQL/Edge Functions straight down in Supabase) constantly introduces massive, significant state synchronization challenges and horrific, unmaintainable code duplication.

The empirical data retrieved directly from the massive load testing validates that the Supabase backend is highly resilient, successfully and thoroughly justifying this exact trade-off. By aggressively accepting the dependency on absolute internet connectivity, PlantUp incredibly simplifies its own fragile device firmware.

Modifying a sensitive moisture threshold, executing brand new SQL functions, or entirely recalibrating the complex gamification algorithms requires absolutely nothing more than a single, rapid backend deployment. The developers entirely, safely, and violently avoid the terrifying risk of pushing massively complex over-the-air (OTA) firmware updates to incredibly fragile, distributed hardware endpoints across the globe. For a heavily consumer-focused IoT product, where rapid, aggressive feature iteration and completely seamless backend maintenance are utterly critical to success, the empirical stability of the massive backend definitively confirms that violently avoiding nasty edge complexity was the absolute correct structural approach to take from day one.

3.7 Intermediate Conclusion: Edge vs. Cloud Architecture

3.7.1 Summary of Findings

This section of the research culminated in the design, implementation, and evaluation of the PlantUp architectural backend. The primary objective was absolutely straightforward: to empirically evaluate if the "Thin Edge"paradigm actually holds up in the specific context of consumer botanical monitoring.

As the experimental results demonstrate, it absolutely does. The domain of plant monitoring primarily tracks physiologically slow-moving environmental metrics like soil moisture evaporation and ambient temperature. A plant does not dry out and wilt in milliseconds. Because of this physiological reality, the system inherently does not require extreme

real-time actuation or highly localized offline autonomy.

The empirical evaluation mathematically confirms that centralizing all threshold evaluation and massive data aggregation within the Supabase cloud backend was structurally viable. By entirely circumventing complex, decentralized state management on the physical ESP32 hardware itself, the architecture successfully minimized the required embedded footprint. This foundational shift deliberately prioritized rapid development speed for the cross-user social features that completely define the PlantUp experience. Ultimately, the system solidly proves that for this specific consumer IoT use case, happily accepting the profound dependency on continuous internet connectivity in exchange for massive cloud agility was an entirely valid and robust architectural trade-off.

3.7.2 Transition to the Implementation Layer

The entire edge-to-cloud communication architecture is predicated entirely on MQTT. This was actively chosen to explicitly guarantee robust data ingestion without suffering through the absolute nightmare of hybrid protocol management. Because of this massive dependency, the physical network implementation must be exceptionally reliable.

The following chapter details the concrete, physical execution of this complex architectural design. It explores the empirical trade-offs between MQTT and REST in the highly messy real world. Finally, it demonstrates exactly how the bespoke IoT Sync mechanism heavily leverages this ironclad MQTT foundation to ensure seamless, asynchronous data synchronization securely between the tiny ESP32 hardware and the unified Supabase cloud database.

4 IoT Data Sync in Microservices: Evaluating MQTT vs. REST

Author: Richard Onuha

4.1 Introduction

4.1.1 Background and Context

Over the past decade, the Internet of Things (IoT) has gradually moved from being a rather conceptual vision to a technology embedded in everyday life. Connected devices now monitor and influence physical environments in ways that might have seemed impractical only a few years ago. This evolution is particularly visible in agriculture. The concept of “Agriculture 4.0” effectively captures this transition from experience-based decision-making toward data-driven optimization, supported heavily by sensor networks and automated systems [59].

Within industrial contexts, these technologies have already led to measurable efficiency gains. Precision irrigation, automated fertilization, and large-scale environmental monitoring are no longer experimental ideas; they have become established practices. Around the same time, a comparable transformation has begun to surface on a much smaller scale in urban environments. Under the label of Smart Urban Gardening, digital tools are increasingly integrated to support plant cultivation in apartments, on balconies, and within community gardens [60].

From a broader perspective, this trend seems closely connected to ongoing societal developments. Rapid urbanization, reduced living space, and a growing collective awareness of sustainability have all contributed to a renewed interest in self-cultivation and local food production [61]. In parallel, the concept of the Social Internet of Things (SIoT) has emerged. This extends the traditional IoT model by allowing objects to form relationships, not only with other devices but actively with users [62]. Under such a framework, connected objects become part of a social ecosystem rather than functioning merely as isolated technical components [63].

When applied to plant care, this perspective implies that a plant, if supported by the right sensors and network connectivity, can start to communicate its condition. It is therefore no longer perceived solely as a passive object requiring observation, but rather as an entity whose specific needs are digitally represented and shared. While this idea initially appeared mostly conceptual, it undoubtedly has practical engineering consequences, particularly when attempting to implement it using affordable, consumer-grade hardware.

Unlike industrial systems that operate with highly stable power supplies and professio-

nally managed infrastructure, consumer IoT devices generally must function under much tighter constraints [64]. Devices placed in residential homes often rely on limited battery power and are typically forced to operate within congested, sometimes highly unstable Wi-Fi networks. Under these circumstances, hardware selection, firmware design, and the choice of communication protocols must all be carefully considered and optimized [65]. Even small inefficiencies that might be entirely negligible in an industrial setting can quickly become critical flaws in battery-powered deployments.

4.1.2 Case Study: The “Plant Up!” Project

The present thesis investigates some of these specific challenges through the case study of the “Plant Up!” project. The system was originally conceived as an IoT-based application combining continuous plant monitoring with gamification elements, aiming to make routine plant care slightly more engaging [66]. Rather than simply displaying raw sensor values, the platform attempts to translate complex biological processes into more meaningful digital feedback for the user.

Its underlying architecture follows a fairly standard three-tier model, consisting of distributed sensor nodes, a scalable cloud-based backend, and a mobile application interface [67]. At the edge layer, ESP32-based devices continually measure environmental parameters such as soil moisture, ambient temperature, humidity, and light intensity. These measurements are then transmitted to a broader cloud infrastructure, where microservices gradually process the incoming data, apply relevant business logic, and update the persistent system state. Following this, the mobile application retrieves and visualizes the information, presenting it in a format intended to emphasize progress, interactive achievements, and overall plant well-being.

While this overall structure may appear reasonably straightforward, the actual implementation tends to reveal several architectural tensions. The continuous interaction between constrained edge devices and an expanding cloud backend requires rather careful coordination. It becomes evident that decisions made at the protocol level can directly and measurably influence system responsiveness, subsequent energy consumption, and the users’ perceived reliability of the platform.

4.1.3 Problem Statement

Designing a socially integrated, near real-time plant monitoring system inevitably exposes a fundamental trade-off between latency, energy efficiency, and data consistency. On one side of the equation, gamification mechanisms rely heavily on immediate feedback. When a user physically waters a plant, the expectation is usually that this action is reflected in the application almost instantly, without noticeable delay [68]. Delayed responses pose a significant risk of diminishing user engagement over time.

Nevertheless, battery-powered sensor nodes must aggressively minimize their energy consumption if they are to remain practical for everyday home use. The ESP32 platform, for instance, provides a Deep Sleep mode that significantly reduces current draw

by disabling both the CPU and the Wi-Fi subsystem [69]. Waking from this state, however, requires full reinitialization and network reconnection, a process that inherently introduces both latency and additional energy expenditure [70].

This recurrent process leads to what can be described as a “wake-up tax”: before any actual, useful data can be transmitted, the device is forced to re-establish its network connectivity [70]. Interestingly, the specific communication protocol chosen for the actual telemetry transmission directly affects the magnitude of this unavoidable overhead. Traditional HTTP-based REST communication is widely supported and remains conceptually simple; yet, it involves relatively verbose textual headers and somewhat heavy message formats. MQTT, in contrast, was designed specifically with such constrained devices in mind. It utilizes a lightweight, binary protocol structure organized around a publish/subscribe model [58].

At this point, it is necessary to consider not only the immediate performance metrics but also overall system consistency. In distributed systems, the CAP theorem fundamentally states that it is impossible to guarantee consistency, availability, and partition tolerance simultaneously in the presence of network failures [71]. In standard residential IoT environments, temporary network partitions are certainly not exceptional events; they are common, almost expected occurrences. The architecture must therefore be designed to tolerate intermittent connectivity while still maintaining an acceptable, workable level of data correctness and service continuity.

Building upon these various considerations, the central research question of this thesis is formulated as follows:

How do MQTT and REST compare in a microservices-based architecture for real-time plant monitoring, specifically with respect to latency, throughput, and data consistency, when constrained by the energy limitations of battery-powered IoT devices?

To approach this question methodically, the following chapters first outline the most relevant theoretical foundations regarding modern communication protocols and distributed systems. The specific system architecture of the “Plant Up!” project is then examined in more detail, highlighting the underlying design decisions that guided its development. Finally, these architectural decisions are evaluated through a set of controlled experimental measurements in order to adequately assess their true practical implications.

4.2 Theoretical Background

Having broadly established the core challenges of latency, energy efficiency, and data consistency in the previous sections, it is helpful to outline the theoretical foundations necessary to fully grasp the architectural decisions made in this thesis. This exploration covers the shift toward microservices architecture, the somewhat harsh physical constraints of IoT hardware, the nuances of communication protocols, and overall distributed data consistency.

4.2.1 Microservices Architecture (MSA)

Microservices Architecture (MSA) represents a fairly fundamental shift in modern software design. It tends to organize applications not as massive, single monolithic systems, but rather as suites of small, relatively autonomous services that collaborate by communicating through well-defined interfaces [73]. In a traditional monolithic setup, almost all functionality, ranging from simple user management to heavy data processing, is tightly coupled within one large, interconnected codebase. While this approach admittedly simplifies initial development and deployment, it often morphs into a significant obstacle to scalability and fault tolerance as the application expands in complexity [74].

MSA conceptually addresses this limitation by treating the overarching application as a flexible collection of independent services. Each service generally runs in its own isolated process and communicates through lightweight channels, mostly using HTTP APIs or dedicated messaging buses [75]. Each distinct service takes responsibility for a specific business domain. For the “Plant Up!” platform in particular, this architectural style seemed well-suited. It allows completely distinct parts of the system to be developed, adjusted, and scaled independently over time. For instance, the specific service tasked with ingesting thousands of sensor readings per minute can theoretically be scaled horizontally during peak watering periods without inadvertently taxing the social feed service, which might be experiencing much lower, steady load at that very moment.

To manage this, the “Plant Up!” backend essentially organizes around domain-specific schemas within Supabase, which function as the dedicated data stores for these separate microservices:

- `user_schema`: This primarily manages user identity, safe authentication, and personal statistics like activity streaks.
- `social_media_schema`: Responsible for organizing the community features, generally including user posts and public plant profiles.
- `gamification`: Houses the somewhat complex logic for quest progression, experience points (XP), and virtual rewards.
- `microcontroller_schema`: Acts as a highly resilient data warehouse dedicated solely to the raw sensor telemetry arriving from the IoT edge nodes.

A remarkably clear example of this separation can be found in the actual storage of the sensor readings themselves. They reside in an entirely isolated table, structurally decoupled from the standard user data:

CONTROLLERS TABLE IN MICROCONTROLLER_SCHEMA

```
create table microcontroller_schema.Controllers (
    id uuid primary key,
    user_id uuid references user_schema.Users(id),
    plant_id uuid references social_media_schema.Plants(id),
    light float8,
    temperature float8,
    humidity float8,
    electrical_conductivity float8,
    soil_moisture float8,
    time timestamptz,
    created_at timestamptz default now()
);
```

Abbildung 4.1: Controllers table in microcontroller_schema

From a practical perspective, maintaining this strict structural boundary noticeably improves overall system robustness. If the designated sensor reading service were to encounter an unexpected failure, the resulting downtime would likely not propagate sideways into the gamification or social interaction features.

4.2.2 Core Characteristics of Microservices in IoT

It must be acknowledged, however, that applying Microservices Architecture specifically to the Internet of Things reliably introduces a rather unique set of challenges. IoT systems tend to be inherently asynchronous and heavily event-driven. Moreover, they are forced to contend regularly with the wildly unpredictable nature of physical home environments.

4.2.2.1 Autonomy and Database per Service

One of the generally accepted fundamental principles of MSA is the concept of a “Database per Service”. This principle strongly dictates that microservices should be decoupled not just at the code boundary, but fundamentally at the data state level [76]. It attempts to prevent the hazardous scenario where modifying an underlying database table for one specific service inadvertently breaks the functionality of another depending on the same schema. In the context of “Plant Up!”, the Gamification Service therefore maintains its own entirely independent record of player statistics, keeping it carefully separated from the often chaotic, raw stream of incoming environmental sensor data.

GAMIFICATION PLAYER STATISTICS

```
create table gamification.player_stats (
    user_id uuid primary key references auth.users(id),
    xp int8,
    level int8
);
```

Abbildung 4.2: Gamification player statistics

4.2.2.2 Scalability and Elasticity

IoT workloads, by their very nature, are characteristically intermittent. Long, quiet periods of sensor silence may be abruptly followed by sudden bursts of intense transmission activity whenever thousands of devices independently wake up to report their status changes. Utilizing MSA enables researchers to handle this variance somewhat gracefully by scaling the specific Ingestion Service horizontally (simply adding more ephemeral instances to absorb the load) without having to allocate similarly expensive, totally unnecessary resources to the more stable Social Service.

4.2.3 Internet of Things (IoT) Constraints and Hardware

At the very hardware foundation of “Plant Up!” sits the ESP32-S3 microcontroller, generally regarded as a highly capable System-on-Chip (SoC) produced by Espressif Systems [78]. Yet, despite offering quite decent processing power, it ultimately remains strictly bound by the harsh physical constraints of battery life. The choice of underlying software transmission protocol directly determines how long the hardware must remain in an energy-draining active state, thereby heavily dictating how long the battery actually lasts in an average household.

4.2.3.1 Power Consumption and Sleep Modes

The ESP32-S3 architecture provides several distinct power modes, each displaying vastly different energy consumption profiles. Understanding the nuances of these modes proved essential for the actual functional architectural design:

- Active Mode: Almost all integrated components, including the CPU, Wi-Fi radio antenna, and various peripherals, are fully powered up. In this state, the device predictably consumes somewhere between 160 mA and 260 mA [79]. Aggressively minimizing time spent in this mode is, quite frankly, critical for any hope of energy conservation.
- Modem Sleep: The main CPU remains active, allowing for script processing, but the energy-heavy radio is temporarily disabled. Power consumption drops noticeably to roughly 20–30 mA [79].

- Deep Sleep: The main CPU, the Wi-Fi module, and the primary RAM are all firmly powered down. Only the Ultra-Low Power (ULP) coprocessor and the basic Real-Time Clock remain weakly active. Current consumption drops precipitously to about 10 µA–150 µA [78].

4.2.3.2 The Wake-Up Tax

While deep sleep obviously provides very substantial energy savings, there is a catch: it introduces a noticeable “wake-up tax”. Upon waking up to perform a reading, the device finds itself forced to completely re-initialize its entire Wi-Fi module, hunt for the access point, and negotiate a fresh IP address on the local network. This clunky process alone typically demands between 1 to 3 seconds, consuming a taxing 150 mA on average, notably before a single byte of useful data is ever transmitted [80]. It follows that the specific choice of communication protocol (such as MQTT vs. REST) severely dictates the amount of additional handshake overhead this wake-up transition incurs.

4.2.4 Communication Protocols: MQTT vs. REST

Choosing the primary communication protocol forms one of the most critical structural architectural decisions for the data layer. It observably impacts energy efficiency, overall latency, and long-term reliability. This subsection attempts to briefly examine the two leading protocols under consideration for the “Plant Up!” sensor layer.

4.2.4.1 MQTT (Message Queuing Telemetry Transport)

MQTT fundamentally operates as a lightweight, binary messaging protocol, having been designed originally for scattered networks that suffer from unreliability and incredibly limited bandwidth [58]. Rather than direct point-to-point connections, it utilizes a distinctly different publish-subscribe routing model.

Architecture and Decoupling: Unlike the standard REST approach, which attempts to connect two endpoints directly and forcefully, MQTT comfortably introduces a central Broker to act as an intermediary. The small sensor (acting as the Publisher) merely flings data toward a designated topic (e.g., `plantup/sensor/01`) without retaining any knowledge whatsoever of which distant backend services are actually consuming that data. The dedicated Broker reliably handles the heavy lifting of message routing, pushing the data to all currently subscribed backend services (such as the main Ingestion Service). One might argue that this specific architecture neatly decouples the physical devices both in space (since they never need to know each other’s precise IP addresses) and critically in time (given that messages can be queued temporarily if a consuming service is currently offline or unreachable).

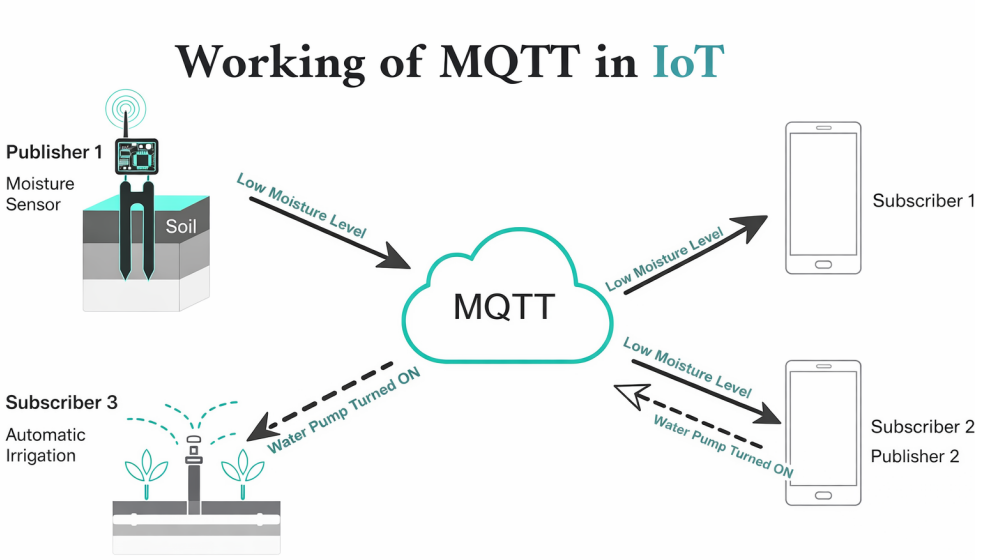

Abbildung 4.3: MQTT Publish/Subscribe Model: The Broker acts as a central hub, efficiently distributing messages from sensors to various services [58].

Packet Structure: Further reinforcing its efficiency, MQTT relies entirely upon a binary transmission format, which gracefully eliminates all bulky, unnecessary textual overhead. The fixed transmission header requires merely 2 bytes of space [142]. This stands in stark contrast to traditional text-heavy HTTP headers, which can very easily accumulate to exceed several hundred bytes per single message [141].

Quality of Service (QoS): MQTT further differentiates itself by providing three sliding levels of distinct delivery assurance [142]:

- QoS 0 (At most once): Effectively a blind fire-and-forget approach. It understandably consumes the least amount of energy, but assumes the risk that if a message is dropped mid-air, it is simply lost forever.
- QoS 1 (At least once): Prioritizes guaranteed delivery by enforcing a strict requirement for a return acknowledgment (PUBACK). This specific level generally proves most suitable for transmitting critical, actionable sensor data.
- QoS 2 (Exactly once): Relies on a rather cumbersome four-step handshake intended to guarantee exactly-once delivery. However, this level almost always introduces an untenable amount of wake-up overhead for severely constrained battery-powered devices.

4.2.4.2 REST (Representational State Transfer)

REST remains the ubiquitous standard architectural style of the modern web, comfortably utilizing highly standardized HTTP methods [141]. It mostly follows a somewhat rigid, synchronous, and resource-oriented conceptual model.

Request-Response Model: REST strictly adheres to a traditional client-server behavioral paradigm. The roaming client explicitly issues a formatted request (like GET or POST) and then sits idle, forced to wait patiently for the distant server to respond before it is permitted to proceed with further tasks.

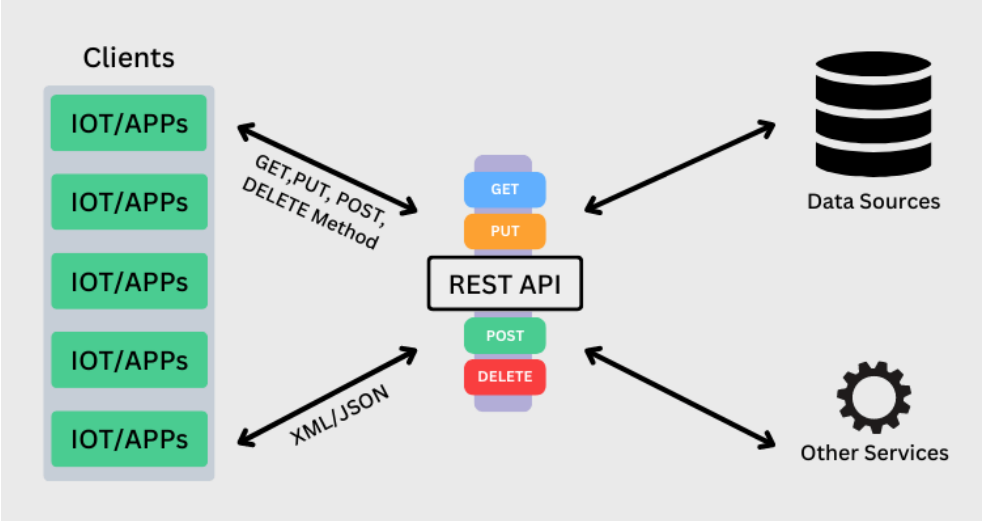

Abbildung 4.4: REST Request-Response Workflow: Stateless clients must establish a new connection for every request, incurring significant overhead [57].

Statelessness and Overhead: Under REST principles, the hosting server actively retains no memory state whatsoever between sequential requests. Consequently, every single transmission request must irritatingly carry all of its own necessary context (including heavy authentication tokens, diverse headers, and structural metadata) every single time. Furthermore, if the IoT device inevitably enters deep sleep between sequential readings, it is forced to endure a full, painful TCP three-way handshake, and frequently an additional TLS cryptographic handshake, for *every single isolated data transmission*. Looking at it closely, this mandatory connection overhead arguably renders REST significantly less energy-efficient than maintaining a partially persistent MQTT session for dealing with frequent, very small data packets.

4.2.5 Data Consistency and the CAP Theorem

In the realm of distributed systems, the widely referenced CAP theorem (specifically Consistency, Availability, Partition Tolerance) strictly establishes that all three functional properties simply cannot be guaranteed simultaneously in the harsh presence of a genuine network failure [77, 81].

For a project like “Plant Up!”, maintaining Partition Tolerance (P) is entirely non-negotiable from the outset, essentially due to the fact that scattered wireless home sensor networks are inherently fickle and broadly unreliable devices. This hard physical reality aggressively forces an architectural choice between pursuing either CP or AP behavior:

- CP (Consistency + Partition Tolerance): Under this model, incoming requests are flatly rejected if perfect data consistency cannot be absolutely guaranteed across

the network, which unfortunately introduces a high risk of outright data loss during a prolonged network disruption.

- AP (Availability + Partition Tolerance): Conversely, data is gracefully accepted and requests are cheerfully served to the user even if certain distant nodes are known to be temporarily out of sync with the master log.

“Plant Up!” ultimately embraces a robust AP design, leaning heavily into the concept of Eventual Consistency. From a practical perspective, it is deemed perfectly acceptable for a casual user to occasionally observe a static soil moisture value that is perhaps a few seconds out of date, provided the overarching mobile application remains entirely responsive and pleasantly stable. Under this framework, the deployed MQTT broker effectively serves as a resilient buffering system, safely holding incoming sensor messages during small network disruptions and ensuring they eventually reach the required downstream microservices layer intact.

With these somewhat dense theoretical foundations now firmly established, the heavily anticipated following section will finally present the concrete, implemented system architecture of “Plant Up!” in practice, clearly demonstrating how these abstract principles were applied to solve the real-world engineering problem at hand.

4.3 Plant Up! System Architecture and Implementation

4.3.1 System Overview and Vision

The “Plant Up!” platform fundamentally attempts to integrate distributed IoT hardware, a scalable cloud microservices backend, and a mobile application into one cohesive ecosystem. This setup effectively represents a practical realization of the Social Internet of Things (SIoT) concept outlined previously in the theoretical background. Ultimately, it enables plants to digitally communicate their environmental status to their caretakers. The primary objective behind this architecture is the collection of high-precision environmental data from custom edge devices, the subsequent synchronization of this data through the cloud, and its presentation to users via slightly gamified interactions.

To realize this broader vision, the architecture naturally had to address three rather critical requirements:

- Low-latency sensor synchronization: When a plant requires attention, it is generally expected that the user is notified promptly. The feedback loop between biological need and human action ought to be closed as rapidly as feasible.
- Energy-efficient operation: IoT devices intended for casual home use must realistically be designed for long-term deployment. This implies conserving battery power through intelligent deep sleep cycles and keeping radio usage to an absolute minimum.

- Distributed consistency: Because data is inherently distributed across User, Social, and Hardware domains, the system must maintain a relatively coherent state, even during the expected periods of localized network instability.

These requirements visibly informed almost every major architectural decision, extending from the specific sensors selected for the hardware up to the overarching structure of the microservices. As a result, the design follows a clearly separated multi-layered approach: the Edge Node, the Cloud Layer, and the Application Layer.

4.3.2 Hardware Layer: The Edge Node

The hardware layer effectively constitutes the physical sensing infrastructure of the project. It serves as the tangible bridge between the digital cloud backend and the physical environment of the plant. At the center of this design is the ESP32-S3 microcontroller, paired with a fairly comprehensive suite of sensors capable of measuring temperature, humidity, light, soil moisture, and electrical conductivity. Together, this configuration is intended to provide a relatively complete picture of the plant's current health. Due to the strict constraints of battery capacity, the device is programmed to spend the vast majority of its operational time in a deep sleep state. Consequently, its active cycle follows a rather strict sequence:

1. Acquisition: The microcontroller wakes up, temporarily powers the attached sensors, and captures a brief environmental snapshot.
2. Serialization: The recorded sensor readings are subsequently packed into a compact JSON format.
3. Transmission: The data packet is finally transmitted to the cloud (utilizing either MQTT or REST), immediately after which the device is instructed to return to its deep sleep mode.

4.3.2.1 Component Selection and Sensor Interface

Hardware selection typically involves a careful trade-off between technical capability and energy longevity. With these constraints in mind, the following components were chosen for the "Plant Up!" edge node.

Microcontroller: Espressif ESP32-S3 The ESP32-S3 acts as the central processing unit. It was selected primarily for its reasonable balance between computational power and energy efficiency. It features a dual-core processor alongside built-in Wi-Fi and Bluetooth connectivity. Of particular relevance here is its Ultra-Low Power (ULP) co-processor, which theoretically allows the main CPU to remain asleep while basic environmental monitoring continues in the background. Priced at approximately 20 Euro, it offers what can be considered a favorable cost-to-performance ratio for this context.

Soil Moisture Sensor: HiLetgo LM393 The HiLetgo LM393 (costing approximately 7.89 Euro) measures the dielectric permittivity of the soil. Unlike many lower-cost sensors that are notoriously prone to rapid corrosion, this resistive variant provides the reasonably reliable moisture readings necessary to accurately trigger watering notifications.

Light Sensor: HiLetgo BH1750 The BH1750 (approximately 11.39 Euro) is a standard digital I2C sensor providing relatively precise lux readings. Having this data enables the system to assess whether a plant is receiving adequate light exposure based on its species and to generate corresponding adjustments or notifications.

Electrical Conductivity (EC) Sensor: DFRobot Gravity V2 The DFRobot Gravity Analog EC Sensor (around 12.10 Euro) is utilized to measure the overall electrical conductivity of the soil, a metric heavily correlated with general nutrient concentration.

- *Relevance of EC:* The rationale here is that electrical conductivity correlates reasonably well with underlying nutrient levels. A plant might be perfectly well-watered yet still severely deficient in nitrogen. Integrating this sensor therefore attempts to elevate the system from functioning merely as a watering alarm into a broader, somewhat more comprehensive health monitor.

Power Supply and Sustainability To support remote, long-term operation, the main device pairs a standard lithium-ion battery with a small 0.5 W photovoltaic solar panel (costing approximately 3.48 Euro). This combined setup is designed to be largely self-sustaining under decent light conditions, while a standard USB-C port is still included as a practical fallback charging option.

4.3.2.2 Data Structure

Prior to transmission, all compiled sensor data is serialized into a structured JSON payload. This specific payload is designed to map directly to the `Controllers` table situated within the backend `microcontroller_schema`:

SENSOR PAYLOAD STRUCTURE

```json
{
  "user_id": "uuid",
  "plant_id": "uuid",
  "light": 350.5,
  "temperature": 21.7,
  "humidity": 45.3,
  "electrical_conductivity": 1.2,
  "soil_moisture": 23.8,
  "time": "2025-01-12T10:15:30Z"
}
```

Abbildung 4.5: JSON payload sent by the edge node

The corresponding database table deployed in Supabase is shown below:

CONTROLLERS TABLE IN MICROCONTROLLER_SCHEMA

```
create table microcontroller_schema.Controllers (
    id uuid primary key,
    user_id uuid references user_schema.Users(id),
    plant_id uuid references social_media_schema.Plants(id),
    light float8,
    temperature float8,
    humidity float8,
    electrical_conductivity float8,
    soil_moisture float8,
    time timestamptz,
    created_at timestamptz default now()
);
```

Abbildung 4.6: Controllers table receiving IoT data

A particularly critical design decision implemented at this stage involves timestamping every measurement at the source, that is, directly on the edge node itself. This approach ensures that even during extended periods of network unavailability, the backend can accurately reconstruct the historical data timeline based on the original recording time, rather than relying on a potentially delayed arrival time at the server.

4.3.3 Microservice Architecture Design

The backend infrastructure is fundamentally structured as a collection of microservices, organized largely around isolated Supabase schemas. In this architecture, each schema functions roughly as a Bounded Context [74], encapsulating a designated domain:

- `user_schema`: This area manages raw user profiles, activity streaks, and virtual wallets.
- `social_media_schema`: Responsible for community content, including user posts, comments, and public plant profiles.
- `microcontroller_schema`: Acts essentially as the centralized data warehouse for all incoming raw sensor readings.
- `gamification`: Houses the core game engine logic, including quests, overall progress tracking, and accumulated XP.

From a practical perspective, this strict separation provides notable development and operational benefits. For instance, any necessary modifications to the quest system can be carried out without risking inadvertent disruptions to the rather sensitive sensor ingestion pipeline. Consequently, failures isolated in one domain are highly unlikely to cascade into the others.

By way of example, the `Plants` table located in the social schema simply links a given user's plant to its ideal, scientifically established growing conditions, which are independently stored in `Plant_data`. This setup allows various other services to retrieve the

required botanical reference data without unnecessarily tangling dependencies within the social database.

PLANTS TABLE

```
create table social_media_schema.Plants (
    id uuid primary key,
    user_id uuid references user_schema.Users(id),
    plant_data_id int8 references social_media_schema.Plant_data(id),
    description text,
    nickname text
);
```

Abbildung 4.7: Plants table in social_media_schema

4.3.4 Real-Time Data Synchronization Mechanism

One of the central challenges encountered in the “Plant Up!” architecture is the ongoing attempt to balance real-time data delivery against the need for strict energy conservation on the remote sensor devices. While users generally expect to see fresh data reflected in the app quite promptly, the sensor nodes simply cannot afford the immense energy cost of maintaining entirely persistent network connections. It is therefore necessary to briefly explain the reasoning behind the adopted synchronization strategy.

During the planning phase, two primary contenders were evaluated for handling the core sensor-to-cloud communication:

- REST (HTTPS): Readily recognized as the standard approach of the modern web. It is highly structured and universally supported, which clearly makes it well-suited for traditional, user-facing API interactions. Nevertheless, when utilized for transmitting small, frequent sensor packets, the necessary connection overhead becomes substantial.
- MQTT: Originating as a protocol designed specifically with constrained IoT scenarios in mind. It is observably lightweight and operates on a publish/subscribe model, a structure that theoretically appears well-matched to the severe constraints of battery-powered microcontrollers such as the ESP32.

Ultimately, a hybrid approach was adopted in an attempt to leverage the distinct strengths of both protocols. The rationale framing this decision is largely as follows: MQTT is utilized exclusively for the uplink sensor-to-cloud communication path. This is because its lightweight binary protocol nature and persistent broker connection help minimize the energy expenditure required for each transmission cycle. Conversely, the user-facing mobile application continues to communicate with the backend via standard REST APIs, mostly because smartphones do not face the same extreme energy constraints and actively benefit from the request-response semantics typical of HTTP when loading complex user profiles or social feeds.

Following this architecture, the deployed IoT devices reliably publish their sensor data via MQTT to a central broker. A dedicated backend service then naturally subscribes to the relevant topics and securely persists the incoming data stream into the database. Only then does the mobile application periodically retrieve this processed data through standard RESTful endpoints.

A typical data ingestion flow under this model is illustrated below:

INSERT EXAMPLE FOR SENSOR READINGS

```
insert into microcontroller_schema.Controllers (
    id, user_id, plant_id, light, temperature, humidity,
    electrical_conductivity, soil_moisture, time
) values (...);
```

Abbildung 4.8: Insert operation for real-time sensor synchronization

4.3.5 User Engagement and Data Processing

It is worth noting that the social and gamification components integrated into “Plant Up!” are not intended merely as cosmetic additions; rather, they serve as the primary mechanism for actively driving user engagement over time. Naturally, this logic is housed within the `gamification` schema, where it continuously monitors the various data streams arriving from the other interconnected layers.

The system relies heavily on three core database tables:

- Quests: Essentially the master catalog of available challenges, incorporating tasks such as “Water your Monstera” or perhaps “Check the overall light level”.
- User_quests: Maintains a record of the individual user’s active progress on their explicitly assigned quests.
- player_stats: Stores the user’s cumulative behavioral statistics, including their overall XP and current level ranking.

For context, the `quests` table fundamentally defines the parameters of the available challenges:

QUESTS TABLE IN GAMIFICATION SCHEMA

```
create table gamification.Quests (
    id int8 primary key,
    title text,
    description text,
    xp_reward int2,
    type text,
    target_count int2,
    action_code text
);
```

Abbildung 4.9: Quests table in gamification schema

The intended integration between the raw sensor layer and the higher-level gamification layer functions roughly as follows. When the sensor layer detects a noticeably sharp increase in soil moisture, the system logically infers that the user has recently watered the plant. The designated processing service then predictably triggers an immediate update to the user's internal quest progress. This creates a relatively direct link between physical, real-world action and digital, rewarding feedback.

USER QUEST PROGRESSION

```
update gamification.User_quests
set progress = progress + 1
where user_id = 'uuid' and quest_id = 3;
```

Abbildung 4.10: User quest progression

Once a specified quest goal is verifiably met (for instance, successfully watering a plant three times in a given week), the system proceeds to award the user with a predetermined amount of experience points.

AWARDING XP TO PLAYER_STATS

```
update gamification.player_stats
set xp = xp + 50
where user_id = 'uuid';
```

Abbildung 4.11: Awarding XP to player_stats

As one might argue, this distinctly modular design helps maintain a very clear operational separation between the higher-level engagement logic and the underlying hardware infrastructure. The various game mechanics, such as adjusting overall quest difficulty or running temporary seasonal events, can thus be modified entirely independently, without ever requiring a risky firmware update on the deployed edge devices.

While the architectural decisions presented throughout this section are certainly grounded in widely established theoretical principles, their real-world effectiveness still requires empirical validation. To provide this, the following section will detail a controlled

experimental evaluation designed to quantify the actual performance differences between MQTT and REST under realistic operating conditions, thereby directly addressing the research question posed earlier in the introduction.

4.4 Experimental Evaluation of MQTT vs. REST

4.4.1 Introduction to the Experiment

While the theoretical analysis presented in Section 2 suggested that MQTT offers distinct advantages over REST regarding protocol overhead and energy efficiency, theoretical insights alone are rarely sufficient to finalize architectural decisions for production environments. At this point, it is necessary to consider how these theoretical benefits translate into practice. Empirical, quantitative data is required to gauge the actual magnitude of the performance difference under realistic operating conditions. A closer examination of the system's behavior was therefore conducted through a targeted experiment. This evaluation attempts to directly address the research question posed initially, measuring the performance gap between MQTT and REST using the actual "Plant Up!" hardware and backend infrastructure.

4.4.2 Experimental Setup

To derive meaningful conclusions, the experimental environment needed to be reasonably standardized to ensure reproducibility. From a practical perspective, the setup was structured around three primary components:

1. The Device (Edge Node): The tests utilized a standard ESP32-S3 unit, essentially identical to the production hardware deployed in actual plant pots. For measurement purposes, the firmware was modified to isolate the protocol-specific transmission time from the rest of the boot process, employing high-precision microsecond timers.
2. The Network: Instead of relying on an idealized laboratory connection, the device was linked to a standard residential 2.4 GHz Wi-Fi router with an average signal strength (RSSI largely between -65 dBm and -75 dBm). This choice was mostly driven by the need to observe protocol behavior under the potentially flawed network conditions typically encountered by end users.
3. The Backend: All test data was routed to the live Supabase production database. This setup ensures that the latency measurements remain representative of the complete end-to-end path, encompassing network travel time, database insertion, and subsequent trigger execution.

In an effort to avoid confounding variables, such as fluctuations in sensor reading time,

a static, pre-defined JSON payload was transmitted during every trial. This approach guaranteed that the payload size stayed consistent across the 200 individual test runs.

PAYLOAD USED FOR BOTH MQTT AND REST EXPERIMENTS

```json
{
  "user_id": "uuid",
  "plant_id": "uuid",
  "light": 300.2,
  "temperature": 21.3,
  "humidity": 48.1,
  "electrical_conductivity": 1.0,
  "soil_moisture": 24.8,
  "time": "2025-01-12T12:10:00Z"
}
```

Abbildung 4.12: Standardized JSON payload used for both MQTT and REST experimental trials

4.4.3 Methodology and Metric Acquisition

To limit the influence of human handling variations, the data collection process was largely automated. The firmware was programmed to execute a repetitive measurement loop for each trial:

1. Wake Up: The microcontroller initiates its boot sequence from deep sleep.
2. Connect: A connection is negotiated with the local Wi-Fi router.
3. Send: The predefined JSON payload is serialized and transmitted, varying only in the underlying method (HTTP POST or MQTT PUBLISH).
4. Verify: The device then waits for a confirmation from the server, expecting either an HTTP 200 status or an MQTT PUBACK.
5. Sleep: Finally, the elapsed time is recorded locally, and the device is instructed to return to deep sleep without delay.

During this cycle, three primary metrics were monitored:

- End-to-End Latency: This captures the total time elapsed from the initial boot sequence to the receipt of server confirmation. It is a comprehensive metric, encompassing handshakes, serialization delays, and overall network propagation.
- Effective Payload Size: To understand the true protocol overhead, Wireshark packet analysis was employed. While a standard application payload might only be around 50 bytes, the total volume of transmitted data, once headers and protocol formatting are appended, often turns out to be noticeably larger.
- Reliability: This was quantified as the ratio of successfully persisted packets against total transmission attempts, evaluated over 100 distinct trials for each protocol.

Consistent packet reception and persistence were later verified by executing a targeted SQL query directly on the backend infrastructure.

VERIFICATION OF RECEIVED PACKETS

```
select count(*)
from microcontroller_schema.Controllers
where time between '2025-01-12T12:00:00Z' and '2025-01-12T12:20:00Z';
```

Abbildung 4.13: SQL verification query used to validate data persistence

4.4.4 Experimental Results

An analysis of the collected data reveals a noticeable divergence in performance between the two implementations. Figure 4.14 summarizes the aggregated metrics gathered from the 100 test runs conducted per protocol.

GENERATED TABLE

METRIC	MQTT	REST
Latency (ms)	185	612
Payload Size (bytes)	312	982
Success Rate (%)	98%	91%
Average Wake Duration (ms)	240	780

Abbildung 4.14: Comparative performance metrics of MQTT vs. REST under identical test conditions

The outcomes suggest that MQTT generally outperformed REST across the measured categories. The average end-to-end latency for MQTT was observed at 185 ms, whereas REST required 612 ms on average, amounting to a reduction of roughly 70%. A similar trend was visible regarding data usage: MQTT transmissions involved approximately 312 bytes per cycle, compared to around 982 bytes for REST.

4.4.5 Discussion

Viewed in context, these results tend to confirm the theoretical expectations regarding HTTP overhead in battery-constrained IoT applications. The recorded latency gap seems primarily driven by the connection establishment phase. For REST, each transmission mandates a full TCP three-way handshake followed by a TLS secure socket negotiation [140, 137]. Under these circumstances, the device ultimately spends more time and energy simply setting up the connection rather than transmitting the actual physiological data.

By contrast, MQTT, although still requiring initial negotiation, manages to omit much of the verbose text-based header information typical of HTTP, such as `User-Agent` and `Content-Type` [139]. Its binary structure and arguably more streamlined handshake process contribute to a noticeably lower per-transmission overhead.

Beyond per-device efficiency, these differences in protocol handling introduce cascading scalability implications as the overall system footprint grows. In a microservices-based IoT architecture, managing concurrent inbound connections often evolves into a primary engineering constraint before sheer data volume does. At an initial deployment stage of roughly 100 active devices, both protocols essentially perform adequately from a backend perspective. The Supabase infrastructure easily absorbs the occasional bursts of REST-based HTTP requests without noticeable API degradation or unusual resource allocation.

As a hypothetical deployment scales toward 1,000 devices, however, the architectural friction inherent in HTTP becomes increasingly difficult to ignore. Because REST is fundamentally stateless and synchronous by design, each sensor node is compelled to establish a fresh TLS connection for every single measurement cycle. This introduces a persistent, cyclical connection overhead on the API Gateway, abruptly forcing the backend to continually allocate memory and processor cycles simply to negotiate, tear down, and rebuild network sockets [137]. MQTT circumvents much of this friction through the use of persistent TCP connections. The central broker maintains an active, lingering state with the registered devices, effectively reducing the ingestion process to a continuous stream of lightweight binary packets rather than a chaotic flurry of independent, isolated requests [139, 145].

At a localized peak of around 10,000 devices, these differences transition from being mere matters of optimization to dictating system survival. Attempting to ingest data from 10,000 sensors simultaneously via REST would inevitably risk triggering rate limits, exhausting API connection pools, and severely overwhelming the database insertion queues [149, 150]. Under an MQTT architecture, this massive influx of telemetry is gracefully decoupled [138]. The broker naturally absorbs the transmission spikes, holding the incoming messages in a queue while comfortably managing tens of thousands of simultaneous connections [146, 147]. The underlying backend microservices can then process this queue at their own sustained, predictable pace, safely persisting data to the `microcontroller_schema` without completely overwhelming the core database engine or risking locked tables [151, 152].

Furthermore, this asynchronous design unlocks significant, highly tangible horizontal scaling potential. Should the ingestion service occasionally fall behind the broker during a massive influx of data—perhaps due to a sudden network reconnection event where thousands of devices transmit queued data simultaneously—additional, ephemeral worker instances can be spun up dynamically to drain the queue more rapidly. With REST, such a synchronized traffic spike impacts the database concurrently and directly, often leading to widespread transmission failures before auto-scaling rules can even react.

An additional, somewhat unintended consequence observed during the trials relates to reliability during periods of fluctuating network integrity. When the Wi-Fi environment became temporarily unstable, the tightly time-constrained HTTP requests were occa-

sionally seen to time out and fail completely. MQTT's asynchronous architecture, paired with its built-in Quality of Service (QoS) retry mechanisms, appeared noticeably better equipped to maneuver through partial connectivity, managing to successfully deliver packets through much narrower windows of network availability.

4.4.6 Energy Implications of Protocol Selection on ESP32-Based IoT Devices

While latency and system scalability certainly form central pillars of this architectural analysis, the most pressing practical constraint for a residential IoT environment arguably remains battery longevity. For the "Plant Up!" ecosystem, where the sensor node is designed to operate seamlessly and unobtrusively within a consumer's home, the rate of energy depletion directly dictates the practical viability of the entire platform. At this stage, it is necessary to consider how the measured protocol overhead practically translates into physical power consumption. By analytically connecting transmission duration with the hardware's specific power states, a clearer picture emerges of the long-term energy implications inherent in choosing between MQTT and REST.

To establish a baseline for this analysis, the power consumption profile of the ESP32-S3 microcontroller must be briefly examined. The device fundamentally operates across several distinct power states, each characterized by significantly different current draws. For the purpose of this estimation, realistic approximate values derived from the manufacturer's documentation are utilized, though actual field performance will always exhibit minor variances based on temperature and battery degradation. During the *Active* mode, when the central processing unit is fully operational but the radio remains idle, the device typically consumes around 35 mA. However, the moment the Wi-Fi modem is powered on and actively negotiating or transmitting data, this current draw spikes considerably, often reaching an average of 240 mA [79, 143]. Conversely, when the device enters its *Deep Sleep* state, with almost all peripherals aggressively powered down to conserve energy, the current consumption drops to roughly 10 µA [143].

To quantify the energy expended during a single telemetry transmission cycle, the standard electrical energy formula can be applied:

$$E = I \times V \times t \tag{4.1}$$

In this equation, E represents the total energy consumed (measured in Joules), I signifies the electrical current drawn (in Amperes), V denotes the operating voltage (typically 3.3 V for the ESP32), and t specifically represents the duration of the active transmission state (in seconds). From this relationship, it becomes evident that because the operating voltage and the peak transmission current remain relatively constant during active Wi-Fi usage, the fundamental variable determining energy consumption is time. In other words, the total energy expended per cycle is almost entirely proportional to how quickly the device can successfully transmit its data and power down its Wi-Fi radio.

This observation brings the previously recorded experimental latency metrics into sharp,

practical focus. During the physical evaluation, the average end-to-end latency for an MQTT transmission was observed at 185 ms (0.185 s). Assuming an average active current of 240 mA during this phase, the energy consumed per MQTT update cycle can be reasonably estimated. Plugging these values into the formula yields:

$$E_{\text{MQTT}} \approx 0.240\,\text{A} \times 3.3\,\text{V} \times 0.185\,\text{s} \approx 0.146\,\text{Joules} \tag{4.2}$$

By contrast, the REST implementation recorded an average transmission latency of 612 ms (0.612 s). As previously noted, this delay is primarily due to the heavy overhead of establishing a fresh TCP connection and performing a full TLS handshake for every single isolated request. Applying the identical hardware parameters to this duration:

$$E_{\text{REST}} \approx 0.240\,\text{A} \times 3.3\,\text{V} \times 0.612\,\text{s} \approx 0.484\,\text{Joules} \tag{4.3}$$

The analytical results indicate a rather striking difference at the hardware level. A single REST transmission consumes nearly 3.3 times as much energy as a comparable MQTT transmission [144]. This suggests that the verbose headers and synchronous nature of HTTP are not merely academic inefficiencies to be debated in software design; they generate a very literal, measurable drain on the device's physical battery chemistry. While the mathematical model assumes perfect transmission conditions, under practical deployment conditions where network congestion might cause retries, this energy gap would likely widen further. Every additional millisecond spent negotiating a secure socket translates directly into wasted Joules of finite energy.

The implications of this disparity compound significantly when extrapolated over a realistic operational timeline. For the purpose of long-term estimation, a typical use case for the "Plant Up!" sensor node can be assumed, where telemetry data is reported nominally once per hour. Over the course of a single year (comprising roughly 8,760 autonomous updates), the accumulated energy footprint expands substantially. Under an MQTT architecture, the transmission overhead alone would account for approximately 1,280 Joules per year. Under a strictly REST-based architecture, this identical workload would siphon away over 4,240 Joules annually—a difference that severely penalizes the device's operational lifespan.

This increased energy consumption naturally triggers a cascading series of secondary effects that ultimately impact long-term consumer usability. If a device utilizing REST drains its battery reserves noticeably faster than an optimally configured MQTT variant, the frequency of necessary battery replacements—or the required duration of solar recharging intervals—rises sharply. In a consumer context, frequent manual intervention entirely undermines the core premise of a "smart," autonomous sensor. A system that requires constant electrical maintenance shifts from being a convenient background utility to an active nuisance, drastically increasing the likelihood of user abandonment over time.

From an architectural standpoint, this analysis reinforces the notion that protocol efficiency holds vastly different levels of importance depending on exactly where it is ap-

plied within the system hierarchy [138]. For the backend cloud servers, the difference between parsing 312 bytes and parsing 982 bytes is structurally negligible, easily absorbed by the sheer processing capacity of modern, scalable cloud infrastructure. The server simply does not care about the extra fractions of a second it takes to parse an HTTP header. However, at the extreme edge of the network, within the stringent, physical confines of a battery-powered ESP32, these fractions of a second represent the difference between a product that lasts for several undisturbed months and one that frustrates the user by requiring a recharge every few weeks. It can therefore be reasonably concluded that optimizing protocol overhead at the device layer is perhaps the single most effective architectural intervention available for extending overall hardware longevity.

4.4.7 Conclusion

From the data gathered, it becomes evident that MQTT represents a more fitting protocol choice for the deeper sensor layer of the “Plant Up!” ecosystem. The reduction in both latency and data usage positions it favorably compared to the REST alternative. That said, REST maintains its relevance within the mobile application layer, where its familiar request-response mechanics are entirely adequate for fetching user profiles and related social content. Nevertheless, the ESP32-S3 hardware used at the edge simply cannot comfortably sustain the persistent overhead of HTTP for its high-frequency sensor reporting. Bearing these factors in mind, MQTT was ultimately selected as the standard protocol for telemetry ingestion, offering a more viable trajectory toward achieving acceptable battery longevity in a consumer-facing product.

5 Gamification. Gaming design patterns to keep people engaged in apps.

Author: Dennis Frenzel

5.1 Introduction

The rapid expansion of the mobile app market has resulted in an overwhelming number of new applications being launched each year, yet only a small fraction achieve long-term success. Research indicates that approximately 80–90% of apps fail within their first year, often due to low user engagement and poor retention [120, 121].

Even when initial downloads are strong, sustaining user interest remains a major challenge; studies show that up to 77% of users stop using an app within the first three days, and the majority abandon it within a month [122]. This highlights a critical issue in digital product design: attracting users is far easier than keeping them engaged.

Gamification, the use of game design elements such as rewards, progress tracking, and challenges in non-game contexts, has emerged as an effective strategy to address this problem. By leveraging intrinsic and extrinsic motivation, gamification can significantly enhance user engagement, improve retention rates, and ultimately increase the likelihood of an application's success [123].

To explore the practical application of these concepts, this thesis introduces "Plant Up!" [5], a gamified smart gardening system developed as a core component of this project. By integrating IoT sensors with a mobile application, Plant Up! is designed to transform the routine tracking of plant health into an engaging, interactive experience. It serves as a real-world case study to investigate how digital mechanics can foster sustainable user motivation and bridge the gap between initial interest and long-term engagement.

In light of these objectives, this thesis poses the following primary research question:

How can gamification be effectively applied to a gardening system to keep users more engaged in a normal activity?

5.2 Theoretical Foundations of Gamification

5.2.1 Definition of Gamification

Gamification is when game design elements and mechanics are integrated into non-game contexts. The goal of introducing these elements is to increase user engagement

and motivation by making otherwise boring or routine tasks more appealing [83, 84].

However, this concept is occasionally conflated with other game-related fields, making it crucial to establish clear boundaries before diving deeper.

5.2.2 Difference between Gamification, Serious Games, and Games for Entertainment

It is important to note that gamification is not the same as serious games or games for entertainment. Although the terms sound similar, they describe different concepts. Gamification refers to the integration of game elements into non-game contexts in order to increase user engagement and motivation. Serious games are complete games that are explicitly designed for a primary purpose other than entertainment, such as education, training, or simulation. For example, a flight simulator used for pilot training or a surgery simulation used for medical training. Games for entertainment, on the other hand, are produced only for enjoyment, without any educational or functional objective [85].

5.2.3 Why Gamification Works in Non-Game Contexts

Understanding what gamification is and what it is not naturally leads to exploring its efficacy. While many people question the effectiveness of gamification, research has shown that it can be an effective strategy for enhancing user engagement and motivation. Specifically, “Many older generations associate the word ‘game’ with ‘wasting time,’” says Dr. Donna Davis, associate professor and director of the Strategic Communication Master’s Program at the University of Oregon [124]. Despite this skepticism, data demonstrates that gamification can still bridge these generational gaps when properly implemented. This raises the questions: Why and how?

According to the Haufe Akademie [86], the effectiveness of gamification lies deep within the human psyche. It addresses the basic needs of autonomy, competence, and social relatedness, which form the foundation of human motivation. How gamification achieves this can be explained using three central mechanisms.

- **1:** Gamification integrates the human need for autonomy, competence, and social relatedness not merely through external rewards, but by actively supporting these needs.
- **2:** It redefines user interaction with a system. Instead of removing all difficult or boring tasks to maximize efficiency, gamification introduces voluntary challenges that encourage people to improve themselves. These tasks are transformed into engaging activities in which progress is clearly visualized and supported by immediate feedback mechanisms such as points and badges.
- **3:** Gamification uses social interaction to build a sense of community. Features such as cooperative challenges and leaderboards satisfy the desire to be noticed

by others, increasing attachment to the activity through competitive or collaborative motivation [86].

As these mechanisms highlight, the core driving force behind any successful gamified system is human motivation. To fully grasp how these game elements interact with user behavior, we must delve deeper into the underlying psychological theories.

5.2.4 Motivation Theory

To understand why gamification can be effective in practice, it is necessary to examine the foundations of human motivation. Motivation is the psychological force that initiates, guides, and sustains goal-directed behavior. In the context of gamification, a key distinction is made between engaging in an activity because it is inherently rewarding (intrinsic motivation) and engaging in it to obtain an external outcome (extrinsic motivation) [87].

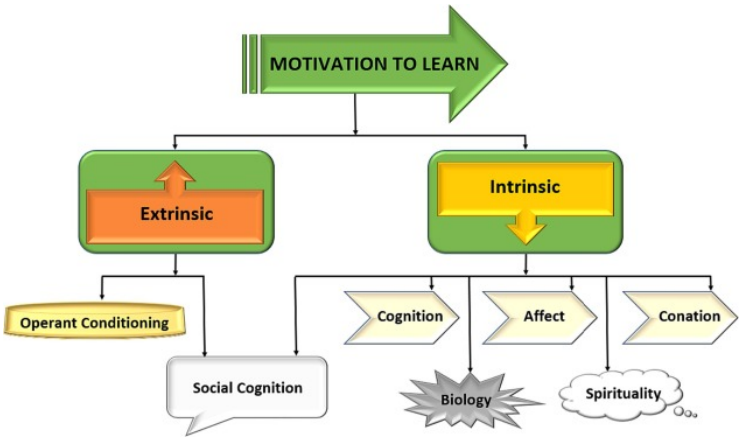

Abbildung 5.1: Classification of Motivation Types [87].

5.2.4.1 Intrinsic Motivation (The Engine of Engagement)

Intrinsic motivation is when a person engages in an activity because it is satisfying, enjoyable, or meaningful, while it also comes from within the person and not from external factors [87]. According to Self-Determination Theory (SDT) [88], intrinsic motivation is supported when three basic psychological needs are fulfilled: Autonomy, Competence, and Relatedness.

When these needs are satisfied, individuals demonstrate increased curiosity, creativity, and persistence [87]. But first, let's define what Self-Determination Theory is.

Self-Determination Theory (SDT) in Gamification
Self-Determination Theory establishes a framework for human motivation based on three fundamental psychological needs: autonomy, competence, and relatedness. Meeting these needs fosters self-determined motivation, which drives improved performan-

ce, persistence, and creativity. Failing to support them can significantly reduce motivation and well-being [87, 88]. Each of these three needs plays a specialized role when applied to a gamified context, starting with autonomy.

Supporting Autonomy
Autonomy refers to the user's sense of control over their actions and decisions. This is achievable with integrating optional tasks, multiple paths to success, and customization options [87].

While autonomy provides users with the freedom to choose, this freedom must be paired with clear feedback on their progress. This leads to the second psychological need: competence.

Supporting Competence
Competence refers to the user's sense of accomplishment and skill development. It is mostly done by integrating experience points, progress indicators, adaptive difficulty, and immediate feedback loops [87].

However, even when users feel autonomous and competent, human beings remain inherently social creatures. Gamification systems capitalize on this through the final pillar of SDT: relatedness.

Supporting Relatedness
Users are more engaged when they feel socially connected, hence integrating social feeds, cooperative goals, peer comparison, and mentorship features is a go-to strategy [87].

Fulfilling these three psychological needs (autonomy, competence, and relatedness) builds a strong foundation for intrinsic motivation. In practical gamification, however, satisfying these inherently internal needs often requires the careful distribution of external rewards.

Rewards that Support Intrinsic Motivation
Motivators that support intrinsic motivation are linked to a personal desire to master, explore, and enjoy an activity. They are the main motivator for long-term commitment, when implemented right, trying to keep them as informational feedback rather than controlling mechanisms is the key to succeeding [89] Most of the time, just like in Plant Up! [5], those are accomplished by integrating, badges, that indicate achievement and mastery, experience points, virtual currency for customization and titles [89].

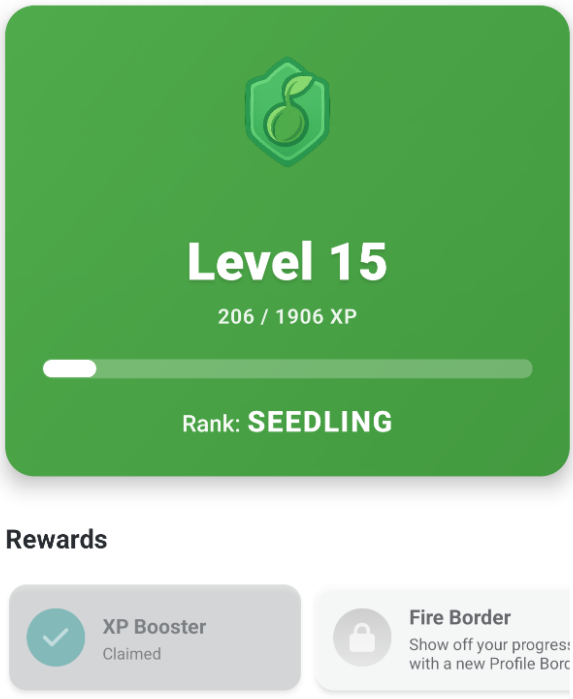

Abbildung 5.2: Level System and Rewards in Plant Up!.

While well-designed rewards enhance intrinsic motivation, their improper use can quickly have the exact opposite effect.

Rewards that Undermine Intrinsic Motivation
Some rewards can be counterproductive when they seem controlling or worthless [89]. Such as over-rewarding insignificant actions, excessive penalties that induce stress, rankings without fairness or relevance, and when rewards replace genuine interest in the activity itself [89]. This often makes those rewards feel like a chore, just like the Duolingo Streak [93].

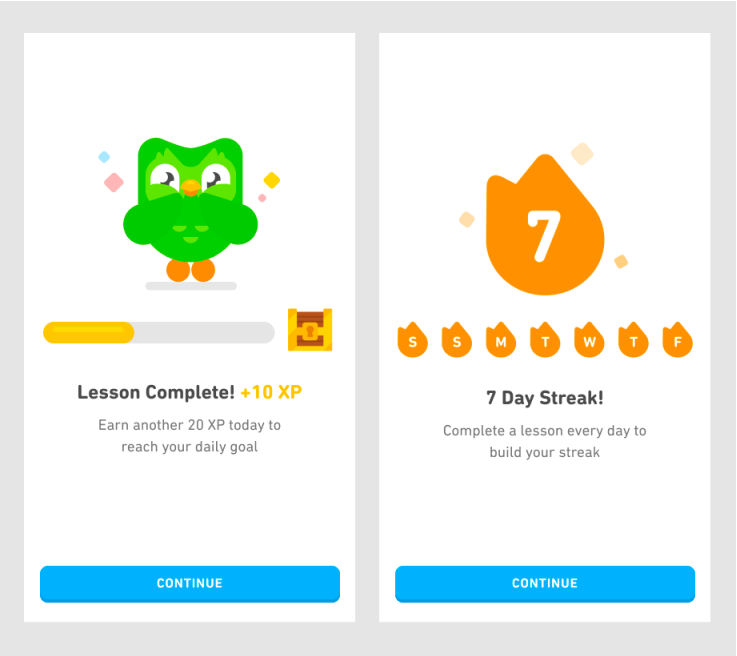

Abbildung 5.3: Duolingo Streak [93].

5.2.4.2 Extrinsic Motivation (The Spark)

Extrinsic motivation involves performing an activity in order to receive a reward or avoid negative consequences. Common gamification elements such as points, badges, and leaderboards are examples of extrinsic motivators [87].

- **Controlling Extrinsic Motivation:** When rewards are perceived as chore-like, they can reduce intrinsic motivation, resulting in short-term compliance but reduced long-term engagement. A typical example is a streak-based system that pressures users to perform regularly, which can lead to stress and burnout.[87].
- **Autonomy-Supportive Extrinsic Motivation:** When rewards provide informational feedback or signal competence, they can be internalized and reinforce intrinsic motivation. Feedback such as positive reinforcement for progress or improvement is more effective than obligation-based incentives.[87].

Effective gamification strategies use extrinsic rewards not as the primary objective, but as a means of supporting intrinsic motivation [86, 87]. Ultimately, the goal is to strike a delicate balance between providing immediate incentives and preserving the user's inherent interest.

The Role of Extrinsic Rewards
Gamification is not merely about adding rewards, but about how they are implemented. Extrinsic motivators can be powerful drivers of engagement but may reduce creativity and encourage mechanical behavior. This can result in retention driven by obligation rather than enjoyment [89]. When used carefully, extrinsic motivators can still provide

short-term motivational boosts that foster intrinsic interests such as mastery or competition [89, 87].

5.2.5 Attribution Theory

While intrinsic and extrinsic motivations explain the *why* of user engagement, how users perceive their own success or failure within these systems is equally critical. This perception is the focus of Attribution Theory.

Attribution theory describes how individuals explain success and failure. In gamified systems, it is important that users get rewarded for their own effort and strategies rather than luck or fixed ability. Implementing effort-based feedback can increase motivation and encourage continued user engagement. Additionally, designing penalties and failure in such a way that it only seems temporary and improvable can lead to less frustration, hence making the user more likely to continue using the app [91].

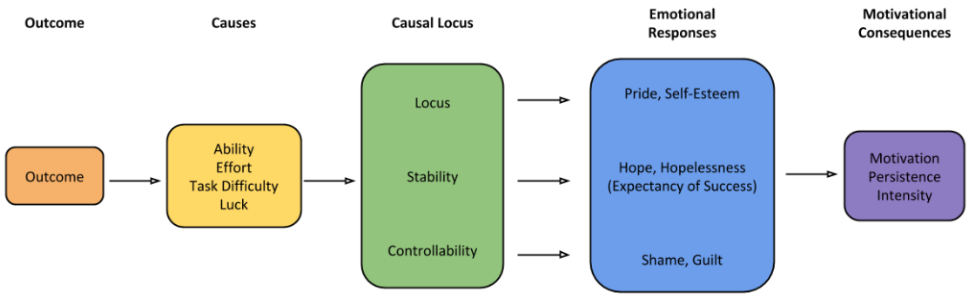

Abbildung 5.4: Attributional Process of Motivation.

Closely related to how users attribute their success or failure is their initial assessment of a task before even attempting it. Expectancy-value theory sheds light on this preparatory phase of motivation.

5.2.6 Expectancy-Value Theory

Expectancy-value theory explains what motivates a user before engaging in an activity: the expectation of success and the subjective value of the task. When a user thinks that they can succeed in an activity and values it, they are more likely to engage in it.

1. **Expectancy for Success:** Expectancy for success can be defined by the question: “How likely is it that I will succeed in this activity?” In games and gamified systems, this belief is answered with a clear definition of goals, transparent rules, the difficulty of the task, and feedback provided during the activity. To make the user have a clear vision of their success, it is important to provide clear objectives, transparent mechanics, and visible indicators of progress. When the criteria for the question “Is this challenge achievable?” are met, the user is more likely to invest effort in it [92].

2. **Subjective Task Value:** Subjective Task Value describes whether the task is worth the effort, which varies for each user. This value consists of several components, including intrinsic value, the importance of performing well, usefulness for future goals, and effort, such as time investment. In games, a value can be increased by making the user progress faster, reducing frustration, or making those tasks relevant for future goals [92].

Ultimately, motivation theories aim not just for temporary spikes in activity, but for sustained user participation. Translating these theoretical concepts into lasting behaviors requires effective engagement mechanics.

5.2.7 Engagement and Habit Formation

5.2.7.1 Feedback Loops

A feedback loop is an important component of gamification, as it provides users with immediate feedback on their progress and achievements.

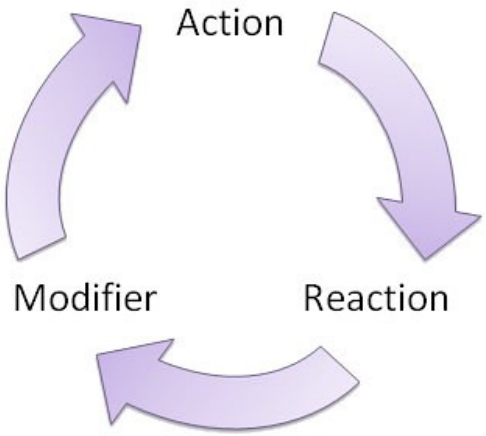

Abbildung 5.5: Feedback Loop [95].

It is important to note that the feedback loop should consist of both positive and negative feedback.

Most people prefer to receive positive feedback, such as badges, experience points, virtual currency, and titles. However, it is important to note that negative feedback can also be a motivator. Not only can negative feedback, such as penalties, warnings, or punishments, be a wake-up call for users to start improving their behavior, but it can also motivate them to avoid those penalties in the future [96].

5.3 Gamification Design Patterns

Building upon the theoretical foundations of motivation, gamification design patterns offer concrete ways to translate those psychological needs into actual application features.

5.3.1 Core Game Mechanics

When looking at basic mechanics like XP and badges, it helps to understand the psychology behind them. According to Yu-kai Chou's Octalysis framework, these elements are driven by what he calls Core Drive 2: Development & Accomplishment. This drive taps into our internal desire to make progress, develop skills, and overcome challenges [125]. For example, in a gardening application, the mundane task of watering a plant becomes more engaging when that action yields immediate visual feedback, such as earning points that demonstrate an expanding "green thumb".

However, Chou points out that points and badges alone are just "Feedback Mechanics", as they serve as milestones to show users they are getting closer to a meaningful "Win State." If a system lacks a balanced progression curve, these features quickly fade, feeling hollow because they no longer satisfy the human need for genuine competence. In the context of cultivating plants, this means ensuring that leveling up reflects real-world mastery over time, rather than just endlessly repeatedly tapping a button. On the other hand, adding components like virtual currency engages another motivator: Core Drive 4: Ownership & Possession [125].

As seen in the Plant Up! application (see Figure 5.6), users earn virtual currency which they can spend in the in-game shop on rewards like XP Boosts or Streak Freezes. When users earn currency, they feel compelled to hold onto it, improve it, and protect their stash [5].

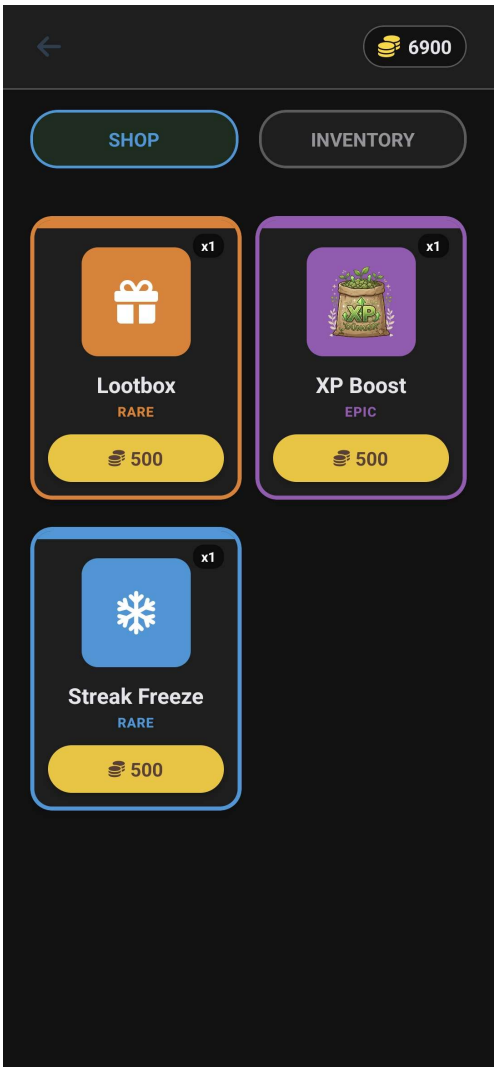

Abbildung 5.6: In-Game Shop and Virtual Currency in Plant Up!

In the end, these core mechanics act as the structural skeleton of an app, ensuring that the time a user spends feels both tangible and organized.

However, while core mechanics establish the baseline of a gamified experience, they alone are often not enough to retain users over the long term. This is where more complex and urgent techniques come into play.

5.3.2 Advanced Engagement Patterns

To hook users on a deeper level, developers often turn to what Chou labels “Black Hat” motivation, specifically Core Drive 8: Loss & Avoidance and Core Drive 6: Scarcity & Impatience [125]. A prime example of this is the concept of daily streaks. Streaks build a powerful “Sunk Cost” effect: users become less motivated by the actual reward and more driven by the fear of losing the progress they’ve already stockpiled. Applied to gardening, this effectively creates a “virtuous cycle” where skipping a day not only breaks a digital streak, but could also tangibly lead to letting a beloved plant wither.

Similarly, time-based rewards and rarity mechanics lean heavily on scarcity. An action or item feels inherently much more valuable simply because it isn't always available to everyone. In a plant care system, this can be mapped to seasonal events or blooming periods, where certain badges or virtual items are only available during a rare, time-sensitive harvest. While these "Appointment Dynamics" are undeniably effective for short-term retention, they walk a fine line. Relying too much on Black Hat drives can make users feel like slaves to the system, creating a sense of obsession rather than genuine fun [125]. The trick is to use these mechanics to inject just enough urgency, such as an alert to water a struggling succulent, without burning the user out.

Beyond individual motivation and time-sensitive triggers, another massive driver for sustained participation lies in human interaction. This leads to an entirely different set of rules focusing on social dynamics.

5.3.3 Social Gamification

Nothing drives human behavior quite like the desire to relate to, and occasionally surpass, our peers. Social mechanics are heavily fueled by Core Drive 5: Social Influence & Relatedness [125]. In a gardening ecosystem, this could be achieved by allowing users to share pictures of their thriving monstera plant or showcasing their garden's overall health score. Leaderboards are the classic "Social Comparison" tool here. They can spark healthy competition, but if designed poorly, they backfire completely. If the points gap between the top player with a massive greenhouse and a newcomer with just one cactus is impossibly huge, it often leads to "Antisocial Envy," causing the new user to just give up.

To balance this out, it's crucial to implement cooperative designs alongside competitive ones. Group quests, for instance, utilize "Social Proof" to build community, such as challenging a group of friends to collectively log twenty days of perfect plant care. When users feel pressure to contribute to a collective goal, they are much more likely to stay engaged. Taking this a step further, adding mentorship or recognition features helps transition top players from chasing individual achievements to earning high-level status within the community by offering guidance to novice plant owners [125]. When competition and cooperation are properly balanced, the system satisfies both our need to stand out and our need to belong.

While combining all these mechanics, such as core progression, urgency, and social interaction, can create highly engaging systems, they must be implemented with care. If pushed too far, gamification can quickly become manipulative, highlighting the need for firm ethical boundaries.

5.3.4 Failure States & Ethical Design

A gamification system fails when it relies almost exclusively on Black Hat drivers. Over time, this leads to heavy burnout, leaving users resentful [125]. For instance, harassing a user with aggressive penalties and notifications every time their soil moisture drops

slightly could make the hobby of gardening feel like a stressful chore rather than a relaxing activity. Chou specifically warns against “Function-Focused Design”. This happens when a designer stops caring about the human experience and instead optimizes purely for metrics like clicks, daily active users, or time-on-site.

Another pitfall is the “Over-Justification Effect.” If a system gives out too many extrinsic rewards (like endless virtual currency for every single drop of water given to a plant), it eventually “kills” the user’s genuine intrinsic interest in the task. Even worse, dark patterns can emerge when loss aversion and scarcity are weaponized, manipulating people into spending real money or time they never intended to part with. To design ethically, there has to be a pivot toward “White Hat” drivers, which are elements that give the user a sense of meaning and empowerment. When a session ends, the user should feel a real sense of accomplishment for having kept their plants alive and thriving, satisfied and in control, not manipulated [125].

5.4 Why Gamification Works for Normal Activities

Applying game design elements to everyday situations has grown from a simple marketing trick into a powerful way to encourage positive habits [126]. When it comes to a very grounded but essential activity like gardening, the biggest hurdle is usually the gap between the effort you put in today and the results you might only see weeks later. This section breaks down exactly why gamification is such an effective strategy for keeping users engaged with a gardening system over the long haul. It specifically focuses on how adding a digital layer can actually enhance the physical experience of taking care of plants. By looking at the psychological factors that drive motivation, the following subsections will explore how repetitive chores are transformed, how immediate feedback bridges the gap between slow nature and fast technology, and how emotional ownership naturally keeps users coming back.

5.4.1 Transformation of Mundane Tasks

Gardening is often seen as a relaxing and therapeutic hobby, but it still involves a lot of repetitive, manual chores like watering, weeding, and checking the soil. For a lot of people, especially those who are used to the fast pace of modern digital life, these tasks can start to feel like a tedious obligation after a while. Gamification tackles this problem head-on by layering a structured sense of progress over these basic physical actions. As Deterding et al. define it, gamification is simply taking elements we love in games and applying them to non-game situations to make the experience more engaging [133]. In a gardening app, this usually takes the form of points, leveling systems, and daily streaks.

The magic of these design elements is that they make invisible progress visible. You might not see a plant physically grow any taller right after you water it [134], but a digital interface can instantly give you experience points (XP) for doing the right thing. This immediate feedback is incredibly important for balancing out different types of motivation. According to Self-Determination Theory, intrinsic motivation means doing something

just because you enjoy it, while extrinsic motivation means doing it for a specific external reward [135]. By handing out points and badges, the system provides a fun, extrinsic push that keeps the user consistent until the real, intrinsic reward, like seeing a new leaf or a blooming flower, finally reveals itself.

On top of the points system, there is the concept of streaks, which has become incredibly popular in modern apps because it taps directly into our psychological need to be consistent. When someone realizes they have built up a “10-day watering streak,” they are far less likely to skip a day. The fear of losing that clean track record suddenly becomes a stronger motivator than the laziness of skipping a chore. This idea fits perfectly with the Fogg Behavior Model, which explains that a behavior only happens when motivation, ability, and a clear prompt all line up at the exact same time [136]. In this case, the gamified app acts as that persistent, friendly prompt. By slicing a massive, months-long goal like a successful harvest into tiny, daily digital milestones, the system turns a slow and “normal” activity into a fun series of rewarding micro-tasks.

Once these mundane chores are broken down into manageable digital steps, the next challenge is ensuring that users confidently know their efforts are working. This leads directly to the importance of real-time confirmation.

5.4.2 Feedback in Real-World Processes

One of the biggest hurdles to sticking with gardening is simply the biological latency of plants. Unlike clicking a button on a website and getting an instant result, natural growth takes time. A seed might take weeks to sprout, and a plant can take months before it finally flowers [134]. This massive disconnect between our modern expectation for instant speed and the slow, quiet pace of nature is often the exact point where beginners lose interest and drop off. Gamification bridges this specific gap by bringing instant feedback loops into the real world, often powered by smart sensors. For example, the moment a sensor detects that the soil moisture has hit the perfect level after watering, the app can flash a glowing green icon, play a joyful sound, or fill up a progress bar.

This creates a tight, closed feedback loop, which is a cornerstone of behavioral psychology. Skinner’s theory of operant conditioning suggests that any behavior followed by a pleasant consequence is highly likely to be repeated [132]. In a smart gardening app, the sensor data acts as the missing link between physical effort and digital celebration. When a user gets a push notification congratulating them that their “Plant Health Index” has improved, it triggers a tiny dopamine hit in the brain. This reinforces the daily habit of checking in on the system. This instant feedback is an absolute game-changer for beginner gardeners who often worry if they are watering too much or too little. The digital system acts as a supportive coach, providing immediate reassurance that their physical actions were correct.

Taking this a step further, beautiful data visualization heavily amplifies this empowering effect. Instead of staring at what looks like a static pot of dirt, the user can pull up a dynamic dashboard displaying growth curves, current nutrient levels, and estimated harvest dates. Being able to visualize this progress satisfies our deep-seated need for

information and a sense of control over our environment [128]. In fact, industry reports consistently show that users are far more likely to stick with a platform if it offers them highly personalized, constantly updating data [130]. By turning a slow biological process into a lively digital dashboard, the system completely removes the frustration of waiting. Instead, it replaces it with the satisfaction of managing a responsive, living entity.

HIER KOMMT EIN BILD REIN VON PLANT UP! UND DIE STATISTIKEN!

When users feel capable and informed, they naturally start to care more about the entity they are nurturing, which brings us to the most powerful long-term motivator of all: emotional connection.

5.4.3 Emotional Attachment & Ownership

While points and instant feedback are fantastic for getting people started, truly long-term engagement is anchored by the emotional connection a user forms with the system. Gamification can heavily foster this bond through customization and by giving a physical plant a distinct digital identity. Simply allowing a user to give their plant a nickname or assign it a cute avatar instantly shifts the app from being a sterile tool into feeling like a companion. This deep sense of ownership is rooted in what behavioral economists call the “IKEA effect.” It is a cognitive bias where people place a significantly higher personal value on things they have helped build or actively labored over themselves [127]. If a user spends three months carefully logging, watering, and nurturing a specific plant within the app, they build up a level of personal investment that makes it incredibly hard to just walk away and abandon the project.

HIER KOMMT EIN BILD REIN VON PLANT UP! UND DER QUESTS/SKINS/EIGENE DIGITALE PLANT/ SPRIG MASKOTT!

This exact type of investment is the final core component of Nir Eyal’s famous Hook Model, which outlines a four-phase process for creating habit-forming products. The cycle consists of a trigger, an action, a variable reward, and finally, an investment [131, 129]. Within a smart gardening system, the “investment” happens every single time the user inputs a new piece of data, unlocks a milestone, or adjusts their garden layout. The more time and effort the user pours into customizing their virtual space and tracking their physical plant, the more irreplaceable the system becomes to them. All of that collected historical data acts as an emotional digital memory of their hard work, constantly validating their identity as a capable and successful gardener.

Ultimately, gamification cements this habit by wrapping the user’s daily actions into an overarching personal narrative. Being able to look back at years of growth allows users to vividly see how much their own skills have improved, effectively elevating a casual hobby into a satisfying journey of mastery. It is well documented that apps leveraging personalization and emotional hooks boast much higher retention rates than those offering only raw data and functional menus. In the context of a gamified garden, the user is not just keeping a plant alive. They are tangibly “leveling up” their real-world environment and their own capabilities. It is this powerful mix of emotional attachment, earned

ownership, and personal growth that guarantees a user will stay committed long after the initial novelty of the app has worn off. By weaving these gamification principles together, a completely normal gardening routine is successfully transformed into a deeply engaging, habit-forming experience that gracefully unites digital rewards with beautiful, real-world results.

5.5 Case Study: Gamification in Plant Up!

“Plant Up!” [5] is a gamified smart gardening system that combines IoT sensors with a mobile application to provide users with a fun and engaging way to monitor and care for their plants. How does the gamification concept in “Plant Up!” [5] work?

5.5.1 Evaluation and Discussion

The core goal of integrating these gamified mechanics into the Plant Up! [5] system is to completely bridge the gap between dry, technical plant monitoring and vibrant, long-term user engagement.

5.5.1.1 Meaningful Habit Formation

Looking closely at core mechanics like XP, daily streaks, and the in-game “XP Booster,” it becomes clear that the system is heavily leveraging classical behavioral psychology to foster sustainable habit formation. However, what sets Plant Up! [5] apart is that these rewards are not just arbitrary digital tasks. Because they are directly tied to real-time data pulled from smart IoT sensors, the rewards are grounded in the actual biological needs of the plants. This creates a framework of genuine, meaningful gamification. By providing structured extrinsic rewards (XP and coins) during the slow biological latency phases, the system successfully bridges the motivation gap until the intrinsic reward (e.g., a healthy, blooming plant) is achieved. This effectively puts the core concepts of Self-Determination Theory (SDT) [135] into practical application.

This smooth transition from extrinsic nudges to intrinsic satisfaction naturally leads into how the system supports beginners who have yet to develop those intrinsic skills.

5.5.1.2 Lowering the Entry Barrier

One of the primary strengths of the Plant Up! [5] approach is how effectively it lowers the barrier to entry for complete beginners. This serves as a significant structural advantage because it directly mitigates the steep learning curve often associated with botany, actively supporting the “Competence” pillar of SDT. Through the use of “Sprig,” the friendly NPC guide, and a system of structured quests, the application translates complex botanical sensor data into simple, actionable feedback (see Figure 5.7).

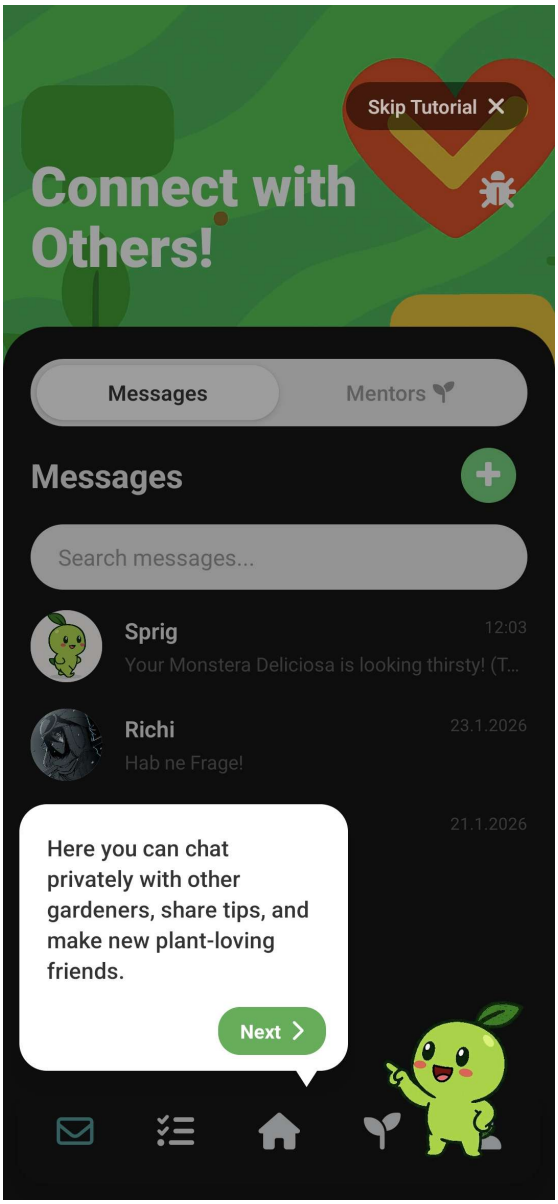

Abbildung 5.7: The Plant Up! NPC guide “Sprig” providing tutorials and easily understandable feedback to help new users [5].

These elements directly address a massive pain point in the gardening hobby: the feeling of being completely overwhelmed by plant care demands. Furthermore, the inclusion of items like the “Freeze Streak” and “XP Boosters” in the shop demonstrates a sophisticated understanding of how to retain players over the long haul. These specific features actively mitigate the heavy “loss aversion” a user feels when they inevitably miss a day of care. By offering a safety net, the system prevents the devastating loss of progress that usually causes users to abandon an app entirely in a moment of frustration.

While the inclusion of such mechanics perfectly addresses beginner anxiety and builds early engagement, it introduces a separate set of risks that must be managed as the user matures.

5.5.1.3 Potential Long-Term Challenges

Relying heavily on extrinsic motivators like virtual coins and titles in the Plant Up! [5] ecosystem does present some potential long-term challenges. This over-reliance represents a structural disadvantage because it can condition users to engage only when an external reward is present, rather than developing a genuine, self-sustained interest in the activity [135]. While these mechanics are incredibly effective for building the initial habit, there is a lingering risk of the "over-justification effect." This happens when users eventually start prioritizing the digital rewards and leaderboards over the actual, intrinsic health of their physical plants. If the gamification layer becomes too dominant, the primary utility of the IoT sensor data, which is ensuring plant survival, could be completely overshadowed by the pursuit of online status. Additionally, designers must be wary of "statistical anxiety." This can easily arise from public social features if users begin to feel intense pressure to maintain perfect streaks, potentially turning what should be a relaxing hobby into an unexpected source of digital stress [5].

To counteract these antisocial or stressful effects, the system relies heavily on communal features that foster the "Relatedness" aspect of SDT, requiring a robust technological foundation.

5.5.1.4 Technical Architecture & Community

From a technical perspective, the overall architecture of Plant Up! [5] efficiently handles the constant stream of sensor and user-generated content, such as shared plant-care videos and social posts, by utilizing an ASP.NET API coupled with Supabase. This social dimension is absolutely crucial for the long-term viability of the project because it elevates the app from being just a solo monitoring tool into a massive, knowledge-sharing community. By fulfilling the human need for social connection, it actively supports the third pillar of SDT (Relatedness).

Looking ahead, future technical enhancements will likely need to focus heavily on aspects like Android animation normalization. Ensuring that the gamification UI remains perfectly fluid across a massive variety of different mobile hardware is essential, as any technical friction or lag in the animations can instantly break the "flow" state required for an immersive gaming experience [5].

5.6 Evaluation & Discussion

5.6.1 Expected Engagement Improvements

- **Increased daily active users:** Gamification features like streaks are expected to drive daily logins.
- **Longer session duration:** Interactive elements and progression systems encou-

rage users to spend more time in the app.

- **Better plant care consistency:** Reminder quests and rewards aim to regularize care activities, leading to healthier plants.

5.6.2 Risks & Limitations

- **Users gaming the system:** Users might perform unnecessary actions just to earn points (e.g., over-watering).
- **Motivation drop after novelty:** The engagement spike from new features may fade ("novelty effect").
- **Balance issues:** Ensuring rewards feel earned but attainable is difficult to fine-tune.

5.6.3 Comparison to Non-Gamified Apps

- **Traditional gardening apps:** Focus solely on information and scheduling.
- **Why they lose users:** Lack of immediate feedback and emotional hook leads to abandonment when the initial enthusiasm fades or the task becomes a chore.

List of Figures

2.1 Resistive Soil Moisture Sensor [29] 7

2.2 Capacitive Soil Moisture Sensor [29] 7

2.3 Measurement distortion caused by non-uniform soil moisture in a plant pot (generated with ChatGPT 4o, OpenAI). 9

2.4 Idealized power profile of the ESP32 microcontroller during a 10-minute duty cycle, illustrating the extreme contrast between the brief active transmission phase and the long deep sleep phase. 11

2.5 Plant status overview in the Plant Up! application showing health score, individual sensor values, and AI-generated care recommendations. 13

2.6 Decision flow from sensor data to care recommendation (generated with ChatGPT 4o, OpenAI). 14

2.7 Traffic light health status in Plant Up!: optimal conditions (left, 100 %), suboptimal conditions requiring attention (center, 50 %), and critical state requiring immediate action (right, 10 %). 15

2.8 Historical sensor data visualization in Plant Up! showing soil moisture readings over a 1-hour period. Users can switch time ranges to identify trends and patterns. 15

2.9 Sensor data visualization confirming a sharp increase in soil moisture directly following a watering event. 19

2.10 Ambient light tracking showing a smooth decline in lux values, reflecting fading afternoon sunlight. 19

3.1 The 7-Layer IoT World Forum Reference Model [206] 24

3.2 Realtime Architecture Flow in Modern BaaS [207] 26

3.3 Visual Comparison of Cloud vs. Edge Architectures (Source: Self-made) . 31

3.4 High-Level System Architecture Diagram (Source: Self-made) 33

4.1 Controllers table in microcontroller_schema 49

4.2 Gamification player statistics 50

4.3 MQTT Publish/Subscribe Model: The Broker acts as a central hub, efficiently distributing messages from sensors to various services [58]. 52

4.4 REST Request-Response Workflow: Stateless clients must establish a new connection for every request, incurring significant overhead [57]. . . . 53

4.5 JSON payload sent by the edge node 56

4.6 Controllers table receiving IoT data 57

4.7 Plants table in social_media_schema 58

4.8 Insert operation for real-time sensor synchronization 59

4.9 Quests table in gamification schema 60

4.10 User quest progression 60

4.11 Awarding XP to player_stats 60

4.12 Standardized JSON payload used for both MQTT and REST experimental trials 62

4.13 SQL verification query used to validate data persistence 63

4.14 Comparative performance metrics of MQTT vs. REST under identical test conditions 63

5.1 Classification of Motivation Types [87]. 70

5.2 Level System and Rewards in Plant Up!. 72

5.3 Duolingo Streak [93]. 73

5.4 Attributional Process of Motivation. 74

5.5 Feedback Loop [95]. 75

5.6 In-Game Shop and Virtual Currency in Plant Up! 77

5.7 The Plant Up! NPC guide “Sprig” providing tutorials and easily understandable feedback to help new users [5]. 83

List of Tables

2.1 GPIO pin mapping of the Plant Up! sensor station. 17

AI-Generated Figures

The following figures were generated using ChatGPT 4o by OpenAI. The prompts used are listed below for transparency.

Figure 2.3 *Cross-section of a plant pot showing non-uniform soil moisture distribution. One side is visibly wetter (darker soil) near the surface where water was poured, while the opposite side remains dry (lighter soil). A soil moisture sensor probe is inserted on the dry side, illustrating how a single-point measurement can be misleading. Clean diagram style, white background, labeled arrows indicating wet zone, dry zone, and sensor position.*

Figure 2.6 *Professional flowchart for an academic thesis on white background with minimal style. Starting node: Sensor Data Received. Process box: Read Values (Soil Moisture, Temperature, Humidity, Light). Diamond decision: Compare against Thresholds. Three output paths: Green Zone (optimal, no action required), Yellow Zone (suboptimal, attention recommended), Red Zone (critical, immediate action required). All paths merge into: Generate Care Recommendation, then Display to User via App. Standard flowchart shapes, sans-serif font, colored zone boxes (green, yellow, red).*

References

[1] Florists' Review, "Houseplant trends and marketing tips," https://floristsreview.com/houseplant-trends-and-marketing-tips/, 2021, accessed: 2026-02-24.

[2] KUNR Public Radio, "Houseplants boomed during the pandemic. gen z, millennials say the popularity is here to stay," https://www.kunr.org/business-and-economy/2022-12-28/houseplants-boomed-during-pandemic-gen-z-millennials-say-popularity-stays-tiktok, 2022, accessed: 2026-02-24.

[3] Mordor Intelligence, "Indoor plants market size, share & analysis," https://www.mordorintelligence.com/industry-reports/indoor-plants-market, 2024, accessed: 2026-02-24.

[4] FloralDaily, "Americans kill nearly half their houseplants, so why do we still spend billions on them each year?" https://www.floraldaily.com/article/9407914/americans-kill-nearly-half-their-houseplants-so-why-do-we-still-spend-billions-on-them-each-y 2021, accessed: 2026-02-24.

[5] A. Sherzai, D. Frenzel, Abudi, and R. Onuoha, "Plant up! smart gardening system," 2025, project developed as part of Diplomarbeit. Contains IoT hardware and Gamified Mobile App.

[6] Terrarium Tribe, "Houseplant statistics & trends 2024," https://terrariumtribe.com/houseplant-statistics/, 2024, accessed: 2025-12-31.

[7] LSU AgCenter, "Water wisely," https://www.lsuagcenter.com/articles/page1718891723199, 2024, accessed: 2025-12-31.

[8] Wisconsin Horticulture, University of Wisconsin–Madison Division of Extension, "Root rots in the garden," https://hort.extension.wisc.edu/articles/root-rots-garden/, 2024, accessed: 2025-12-31.

[9] S. Wang *et al.*, "New prospectives on light adaptation of visual system and glare evaluation," *Frontiers in Built Environment*, 2022, accessed: 2025-12-31.

[10] MarketsandMarkets, "Smart home market research report: 2024 overview," https://www.marketsandmarkets.com/ResearchInsight/smart-home-market-research.asp, 2024, accessed: 2025-12-31.

[11] Precedence Research, "Smart home market size, share and trends 2025 to 2034," https://www.precedenceresearch.com/smart-home-market, 2025, accessed: 2025-12-31.

[12] Food and Agriculture Organization of the United Nations (FAO), "Water for sustainable food and agriculture," https://openknowledge.fao.org/server/api/core/bitstreams/b48cb758-48bc-4dc5-a508-e5a0d61fb365/content, n.d., accessed: 2025-12-31.

[13] World Bank, “Chart: Globally, 70% of freshwater is used for agriculture,” https://blogs.worldbank.org/en/opendata/chart-globally-70-freshwater-used-agriculture, 2017, accessed: 2025-12-31.

[14] NC State Extension, “Soils & plant nutrients (extension gardener handbook, chapter 1),” https://content.ces.ncsu.edu/extension-gardener-handbook/1-soils-and-plant-nutrients, 2026, accessed: 2026-02-24.

[15] University of California Statewide IPM Program (UC IPM), “Aeration deficit,” https://ipm.ucanr.edu/PMG/GARDEN/ENVIRON/aeration.html, 2026, accessed: 2026-02-24.

[16] Cornell University (NRCCA), “Certified crop advisor study resources: Permanent wilting point,” https://nrcca.cals.cornell.edu/soil/CA2/CA0212.1-3.php, 2026, accessed: 2026-02-24.

[17] METER Group, “Plant available water: How do i determine field capacity and permanent wilting point?” https://metergroup.com/measurement-insights/plant-available-water-how-do-i-determine-field-capacity-and-permanent-wilting-point/, 2026, accessed: 2026-02-24.

[18] NASA Earthdata, “Photosynthetically active radiation (par),” https://www.earthdata.nasa.gov/topics/biosphere/photosynthetically-active-radiation, n.d., accessed: 2026-02-24.

[19] Apogee Instruments, “Conversion – ppfd to lux,” https://www.apogeeinstruments.com/conversion-ppfd-to-lux/, 2026, accessed: 2026-02-24.

[20] Encyclopaedia Britannica, “Transpiration,” https://www.britannica.com/science/transpiration, 2026, accessed: 2026-02-24.

[21] E. B. Blancaflor *et al.*, “Importance of measuring and reporting environmental conditions across plant science subdisciplines,” *Plant Physiology*, vol. 199, no. 2, 2025.

[22] OpenStax, “Plant sensory systems and responses,” https://openstax.org/books/biology-2e/pages/30-6-plant-sensory-systems-and-responses, 2026, accessed: 2026-02-24.

[23] University of Maryland Extension, “Sunburn damage on flowers,” https://extension.umd.edu/resource/sunburn-damage-flowers, 2023, accessed: 2026-02-24.

[24] University of New Hampshire Extension, “How can i increase the humidity indoors for my houseplants?” https://extension.unh.edu/blog/2025/01/how-can-i-increase-humidity-indoors-my-houseplants, 2025, accessed: 2026-02-24.

[25] Utah State University Extension, “Overwatering,” https://extension.usu.edu/planthealth/ipm/ornamental-pest-guide/abiotic/overwatering.php, 2026, accessed: 2026-02-24.

[26] University of Pittsburgh, Webvision, “Light and dark adaptation,” https://www.webvision.pitt.edu/book/part-viii-psychophysics-of-vision/light-and-dark-adaptation/, 2007, accessed: 2026-02-24.

[27] S. Chowdhury, S. Sen, and S. Janardhanan, “Comparative analysis and calibration of low cost resistive and capacitive soil moisture sensor,” *arXiv preprint arXiv:2210.03019v1*, 2022, accessed: 2026-02-26.

[28] Seeed Studio, “Grove – capacitive moisture sensor (corrosion resistant),” https://wiki.seeedstudio.com/Grove-Capacitive_Moisture_Sensor-Corrosion-Resistant/, 2023, accessed: 2026-02-26.

[29] D. Gupta, “Capacitive v/s resistive soil moisture sensor,” https://www.hackster.io/devashish-gupta/capacitive-v-s-resistive-soil-moisture-sensor-e241f2, 2020, accessed: 2026-02-26.

[30] NXP Semiconductors, “Um10204: I2c-bus specification and user manual,” https://www.nxp.com/docs/en/user-guide/UM10204.pdf, 2014, rev. 6, Accessed: 2026-02-26.

[31] Bureau International des Poids et Mesures (BIPM), “Principles governing photometry,” https://www.bipm.org/documents/20126/41749107/Principles+-governing+-photometry/, 2019, rapport BIPM-2019/05, Accessed: 2026-02-26.

[32] Espressif Systems, “Deep sleep,” https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/sleep_modes.html, 2024, eSP-IDF Programming Guide, Accessed: 2026-02-26.

[33] ——, *ESP32 Series Datasheet*, https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf, 2024, version 4.3, Accessed: 2026-02-26.

[34] T. Bray, “The javascript object notation (json) data interchange format,” https://datatracker.ietf.org/doc/html/rfc8259, 2017, rFC 8259, December 2017.

[35] OASIS, “Mqtt version 5.0,” https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html, 2019, oASIS Standard, March 2019.

[36] P. Boniecki *et al.*, “Smart sensors and smart data for precision agriculture: A review,” *Sensors*, vol. 23, no. 17, 2023, review of IoT sensor systems and composite health assessment in precision agriculture.

[37] F. Doohan *et al.*, “Decision support systems halve fungicide use compared to calendar-based strategies without increasing disease risk,” *Communications Earth & Environment*, vol. 2, 2021.

[38] International Organization for Standardization, “Graphical symbols — safety colours and safety signs — part 1: Design principles for safety signs and safety markings,” https://www.iso.org/standard/51021.html, 2011, iSO 3864-1:2011.

[39] S. Dunne *et al.*, “Visualization techniques of time-oriented data for the comparison and analysis of patient groups,” *Journal of Medical Internet Research*, vol. 24, no. 10, 2022.

[40] Z. Cao, T. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, “Openpose: Realtime multi-person 2d pose estimation using part affinity fields,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[41] K. Sun, B. Xiao, D. Liu, and J. Wang, “High-resolution representations for labeling pixels and regions,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[42] M. Andriluka, L. Pishchulin, P. Gehler, and B. Schiele, “2d human pose estimation: New benchmark and state of the art analysis,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2014.

[43] C. Ionescu, D. Papava, V. Olaru, and C. Sminchisescu, “Human3.6m: Large scale datasets and predictive methods for 3d human sensing in natural environments,” *In: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2013.

[44] D. Pavllo, C. Feichtenhofer, D. Grangier, and M. Auli, “3d human pose estimation in video with temporal convolutions and semi-supervised training,” in *In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[45] P. F. Felzenszwalb and D. P. Huttenlocher, “Pictorial structures for object recognition,” *In: International Journal of Computer Vision*, vol. 61, no. 1, pp. 55–79, 2005.

[46] C. Lugaresi, J. Tang, H. Nash, and et al., “Mediapipe: A framework for building perception pipelines,” in *In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.

[47] M. Contributors, “Openmmlab pose estimation toolbox and benchmark,” https://github.com/open-mmlab/mmpose, 2020.

[48] A. Mathis, P. Mamidanna, K. M. Cury, and et al., “Deeplabcut: Markerless pose estimation of user-defined body parts with deep learning,” *In: Nature Neuroscience*, vol. 21, no. 9, pp. 1281–1289, 2018.

[49] A. Grinciunaite, A. Petrenas, and D. Navakauskas, “Pose estimation for human motion analysis: A survey,” *In: Sensors*, vol. 22, no. 17, p. 6501, 2022.

[50] Y. Xu, J. Wang, Y. Zhang, J. Wang, and Y. Wei, “Vitpose: Simple vision transformer baselines for human pose estimation,” *In: Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[51] Unknown, “Microservices architecture,” 2024, placeholder for citation 20.

[52] ——, “Iot resilience,” 2024, placeholder for citation 21.

[53] ——, “Json payload optimization,” 2024, placeholder for citation 22.

[54] E. Systems, “Esp32 power consumption,” 2024, placeholder for citation 23.

[55] OASIS, “Mqtt protocol specification,” 2024, placeholder for citation 24.

[56] E. Brewer, “Cap theorem,” 2000, placeholder for citation 25.

[57] DreamFactory, “Rest vs graphql: Which api design style is right for your organization?” https://blog.dreamfactory.com/rest-vs-graphql-which-api-design-style-is-right-for-your-organization, 2024, accessed: 2025-12-09.

[58] PsiBorg, “Advantages of using mqtt for iot devices,” https://psiborg.in/advantages-of-using-mqtt-for-iot-devices/, 2024, accessed: 2025-12-09.

[59] Leher, “Exploring agriculture 4.0: Technology’s role in future farming,” https://leher.ag/blog/technology-role-agriculture-4-0-future-farming, 2024, accessed: 2025-12-09.

[60] StreetSolver, “The smart urban garden: Top tech & ai helping you grow more food at home in 2026,” https://streetsolver.com/blogs/the-smart-urban-garden-top-tech-ai-helping-you-grow-more-food-at-home-in-2026, 2024, accessed: 2025-12-09.

[61] GrowDirector, “Urban agriculture in 2025: A growing trend with deep roots,” https://growdirector.com/urban-agriculture-in-2025-a-growing-trend-with-deep-roots, 2024, accessed: 2025-12-09.

[62] BPB Online, “What is social internet of things (siot)?” https://dev.to/bpb_online/what-is-social-internet-of-things-siot-3g0a, 2024, accessed: 2025-12-09.

[63] J. Jung and I. Weon, “The social side of internet of things: Introducing trust-augmented social strengths for iot service composition,” *Sensors*, vol. 25, no. 15, p. 4794, 2025, accessed: 2025-12-09. [Online]. Available: https://mdpi.com/1424-8220/25/15/4794

[64] KaaIoT, “Iot device challenges – power & connectivity constraints,” https://kaaiot.com/iot-knowledge-base/how-does-wi-fi-technology-link-to-the-iot-industry, 2024, accessed: 2025-12-09.

[65] B. Hemus, “6 common iot challenges and how to solve them,” https://emnify.com/blog/iot-challenges, 2024, accessed: 2025-12-09.

[66] Smartico, “Growing green: How gamification is revolutionizing sustainable agriculture,” https://smartico.ai/blog-post/gamification-revolutionizing-sustainable-agriculture, 2025, accessed: 2025-12-09.

[67] B. Che, “Implementing iot with a three-tier architecture,” https://enterprisersproject.com/article/2016/1/implementing-iot-three-tier-architecture, 2016, accessed: 2025-12-09.

[68] Spinify, “The critical role of real-time feedback in gamification,” https://spinify.com/blog/how-ai-is-enabling-real-time-feedback-in-gamification, 2023, accessed: 2025-12-09.

[69] Programming Electronics, “A practical guide to esp32 deep sleep modes,” https://programmingelectronics.com/esp32-deep-sleep-mode, 2024, accessed: 2025-12-09.

[70] R. Community, “Esp32 deep sleep wake-up discussion,” https://reddit.com/r/esp32/comments/gfroev/time_to_wake_up_and_connect_to_wifi_from_deep/, 2020, accessed: 2025-12-09.

[71] DesignGurus, “Cap theorem explained,” https://designgurus.io/answers/detail/cap-theorem-for-system-design-interview, 2024, accessed: 2025-12-09.

[72] Nabto, “MQTT vs. REST in IoT: Which should you choose?” https://nabto.com/mqtt-vs-rest-iot, 2021, accessed: 2025-12-09.

[73] AWS, “Microservices architecture - key concepts,” https://docs.aws.amazon.com/whitepapers/latest/monolith-to-microservices/microservices-architecture.html, 2019, accessed: 2025-12-09.

[74] M. Fowler, “Monolithic vs. microservices,” https://martinfowler.com/articles/microservices.html, 2024, accessed: 2025-12-09.

[75] S. Newman, *Building Microservices*. O’Reilly Media, 2015, accessed: 2025-12-09.

[76] C. Richardson, “Microservices and data,” https://microservices.io/patterns/data/database-per-service.html, 2024, accessed: 2025-12-09.

[77] E. Brewer, “Brewer’s cap theorem,” https://interviewnoodle.com/understanding-the-cap-theorem-a-deep-dive-into-the-fundamental-trade-offs-of-distributed-s 2000, summary by InterviewNoodle. Accessed: 2025-12-09.

[78] Espressif Systems, *ESP32-S3 Series Datasheet*, 2024, accessed: 2025-12-09. [Online]. Available: https://espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf

[79] LastMinuteEngineers, “Esp32 power consumption - active mode,” https://lastminuteengineers.com/esp32-sleep-modes-power-consumption/, 2024, accessed: 2025-12-09.

[80] A. S. Forum, “Wi-fi reconnect overhead,” https://forums.adafruit.com/viewtopic.php?f=57&t=163388, 2020, accessed: 2025-12-09.

[81] S. Jaiswal, “The impossible triangle of distributed systems: Cap theorem,” https://www.linkedin.com/pulse/impossible-triangle-distributed-systems-cap-theorem-sejal-jaiswal-atbzc, 2023, accessed: 2025-12-09.

[82] R. Onuoha, “Diplomarbeit,” 2025, internal Reference. Accessed: 2025-12-09.

[83] Gabler Wirtschaftslexikon, “Definition: Gamification,” https://wirtschaftslexikon.gabler.de/definition/gamification-53874, 2024, accessed: 2025-12-28.

[84] G. Burtt, “Gamification, game-based learning and serious games – what’s the difference?” https://grendelgames.com/gamification-game-based-learning-serious-games/, n.d., accessed: 2025-12-28.

[85] R. Damaševičius, “Serious games and gamification in healthcare: A meta-review?” https://www.mdpi.com/2078-2489/14/2/105, 2023, accessed: 2025-12-28.

[86] Haufe Akademie, “Gamification: Spielerisch zu mehr motivation,” https://www.haufe-akademie.de/digital-suite/blog/ds-gamification, 2024, accessed: 2025-12-28.

[87] D. Bandhu, M. M. Mohan, N. A. P. Nittala, P. Jadhav, A. Bhadauria, and K. K. Saxena, “Theories of motivation: A comprehensive analysis of human behavior drivers,” *Acta Psychologica*, 2024, accessed: 2025-12-28.

[88] Center for Self-Determination Theory, “Theory - self-determination theory,” https://selfdeterminationtheory.org/theory/, 2024, accessed: 2025-12-30.

[89] Gamify, “Extrinsic vs. intrinsic motivation in gamification marketing,” https://www.gamify.com/gamification-blog/extrinsic-vs-intrinsic-motivation-in-gamification-marketing, 2024, accessed: 2025-12-30.

[90] FutureLearn, “The psychology of games,” https://www.futurelearn.com/info/courses/game-psychology/0/steps/428462, n.d., accessed: 2025-12-31.

[91] Red White Console, “Motivation and games: Attribution theory,” https://www.redwhiteconsole.net/motivation-games-attribution-theory/, n.d., accessed: 2025-12-31.

[92] ScienceDirect, “Expectancy-value theory,” https://www.sciencedirect.com/topics/psychology/expectancy-value-theory, n.d., accessed: 2025-12-31.

[93] A. Yu, “Improving the streak: Forming habits one lesson at a time,” https://blog.duolingo.com/improving-the-streak/, n.d., accessed: 2025-12-31.

[94] D. Jin, “Diplomarbeit,” 2025, internal Reference. Accessed: 2025-12-31.

[95] A. Marczewski, “Feedback loops, gamification and employee motivation,” https://www.gamified.uk/2013/03/25/feedback-loops-gamification-and-employee-motivation/, 2013, accessed: 2025-12-31.

[96] Motivacraft, “Feedback loops in gamification: The key to engaging user experiences,” https://www.motivacraft.com/feedback-loops-in-gamification-the-key-to-engaging-user-experiences/, n.d., accessed: 2025-12-31.

[97] Texas A&M AgriLife Extension (Aggie Horticulture), “Efficient use of water in the garden and landscape,” https://aggie-horticulture.tamu.edu/earthkind/drought/efficient-use-of-water-in-the-garden-and-landscape/, n.d., accessed: 2025-12-31.

[98] Iowa State University, “Soil aeration – introduction to soil science,” https://iastate.pressbooks.pub/isudp-2025-201/chapter/soil-aeration/, 2026, accessed: 2026-02-24.

[99] Cornell University Composting, “Oxygen diffusion,” https://compost.css.cornell.edu/oxygen/oxygen.diff.html, 2026, accessed: 2026-02-24.

[100] K. J. McCree, “Test of current definitions of photosynthetically active radiation against leaf photosynthesis data,” *Agricultural Meteorology*, vol. 10, pp. 443–453, 1972.

[101] U.S. Geological Survey (USGS), “Evapotranspiration and the water cycle,” https://www.usgs.gov/water-science-school/science/evapotranspiration-and-water-cycle, n.d., accessed: 2026-02-24.

[102] Encyclopaedia Britannica, “Turgor,” https://www.britannica.com/science/turgor, 2026, accessed: 2026-02-24.

[103] Bosch Sensortec, “Bme280: Combined humidity and pressure sensor (datasheet),” https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf, 2021, accessed: 2026-02-26.

[104] OASIS, “MQTT version 5.0,” OASIS Standard, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.pdf

[105] ——, “MQTT version 3.1.1,” OASIS Standard, 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.pdf

[106] Espressif Systems, “ESP32-S3 series datasheet,” Tech. Rep., 2025. [Online]. Available: https://documentation.espressif.com/api/resource/doc/file/rz94aWY3/FILE/esp32-s3_datasheet_en.pdf

[107] ——, *ESP-IDF Programming Guide: ESP Timer (High Resolution Timer)*, 2025. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html

[108] IETF, “Rfc 7230: Hypertext transfer protocol (HTTP/1.1): Message syntax and routing,” Tech. Rep., 2014. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7230.html

[109] ——, “Rfc 7231: Hypertext transfer protocol (HTTP/1.1): Semantics and content,” Tech. Rep., 2014. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7231

[110] ——, “Rfc 9293: Transmission control protocol (TCP),” Tech. Rep., 2022. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc9293

[111] ——, “Rfc 8446: The transport layer security (TLS) protocol version 1.3,” Tech. Rep., 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446

[112] Wireshark Project, *Wireshark User’s Guide (v4.7.0)*, 2024. [Online]. Available: https://www.wireshark.org/docs/wsug_html/

[113] Supabase. (2026) Supabase: The open source firebase alternative. [Online]. Available: https://supabase.com/

[114] ——. (2026) Supabase database: Every project is a full postgres database. [Online]. Available: https://supabase.com/features/postgres-database

[115] ——. (2026) Postgres triggers. [Online]. Available: https://supabase.com/docs/guides/database/postgres/triggers

[116] PostgreSQL Global Development Group. (2026) Postgresql documentation: Create trigger. [Online]. Available: https://www.postgresql.org/docs/current/sql-createtrigger.html

[117] Cisco Systems, "Understand site survey guidelines for wlan deployment," Tech. Rep., 2023. [Online]. Available: https://www.cisco.com/c/en/us/support/docs/wireless/5500-series-wireless-controllers/116057-site-survey-guidelines-wlan-00.html

[118] C. Jara Ochoa *et al.*, "Comparative analysis of power consumption between MQTT and HTTP in an IoT monitoring platform," *Sensors*, vol. 23, no. 10, p. 4896, 2023. [Online]. Available: https://doi.org/10.3390/s23104896

[119] Y. Sasaki and T. Yokotani, "Performance evaluation of MQTT as a communication protocol for IoT and prototyping," *Advances in Technology Innovation*, vol. 4, no. 1, 2019. [Online]. Available: https://ojs.imeti.org/index.php/AITI/article/view/2522/734

[120] Business of Apps, "Mobile app retention strategy," https://www.businessofapps.com/guide/mobile-app-retention/, n.d., accessed: 2026-03-01.

[121] Ethora, "Customer retention rate insights by industry in mobile apps," https://ethora.com/blog/customer-retention-rate-insights-by-industry-in-mobile-apps/, n.d., accessed: 2026-03-01.

[122] Localytics, "Mobile apps: What's a good retention rate?" https://medium.com/@Localytics_58363/mobile-apps-whats-a-good-retention-rate-528381a1af0d, n.d., accessed: 2026-03-01.

[123] M. Barari, "How and when does gamification level up mobile app effectiveness? meta-analytics review," *Marketing Intelligence & Planning*, 2024, accessed: 2026-03-01. [Online]. Available: https://www.researchgate.net/publication/381789135_How_and_when_does_gamification_level_up_mobile_app_effectiveness_Meta-analytics_review

[124] S. Haaland, "You think older generations don't respond to gamification? the data says differently." https://www.salesscreen.com/blog/older-generations-and-gamification, n.d., accessed: 2026-03-01.

[125] Y.-k. Chou, *Actionable Gamification: Beyond Points, Badges, and Leaderboards*. Octalysis Media, 2015.

[126] Rewardz, "Gamification marketing: Moving beyond niche," https://rewardz.sg/blog/gamification-marketing/, n.d., accessed: 2026-03-01.

[127] The Decision Lab, "The ikea effect," https://thedecisionlab.com/biases/ikea-effect, n.d., accessed: 2026-03-01.

[128] OAMK Journal, “Gamification in learning: Turning work into play,” https://oamkjournal.oamk.fi/2025/gamification-in-learning-turning-work-into-play/, 2025, accessed: 2026-03-01.

[129] B. Buldanli, “Hook framework in product management,” https://berkbuldanli.medium.com/hook-framework-in-product-management-46fb415cb4a8, n.d., accessed: 2026-03-01.

[130] Design Bootcamp, “The story behind meaningful gamification,” https://medium.com/design-bootcamp/the-story-behind-meaningful-gamification-2f0432f77162, n.d., accessed: 2026-03-01.

[131] Amplitude, “The hook model,” https://amplitude.com/blog/the-hook-model, n.d., accessed: 2026-03-01.

[132] Simply Psychology, “Operant conditioning,” https://www.simplypsychology.org/operant-conditioning.html, n.d., accessed: 2026-03-01.

[133] S. Deterding *et al.*, “From game design elements to gamefulness: Defining "gamification",” https://www.gbl.uzh.ch/quartz/references/Deterding-et-al.-(2011), 2011, accessed: 2026-03-01.

[134] Gorilla Grow Tent, “How long does a plant take to grow?” https://www.gorillagrowtent.com/blogs/news/how-long-does-plant-take-to-grow, n.d., accessed: 2026-03-01.

[135] UK Coaching, “Self-determination theory,” https://www.ukcoaching.org/ukc-club/resources/self-determination-theory/, n.d., accessed: 2026-03-01.

[136] The Decision Lab, “Fogg behavior model,” https://thedecisionlab.com/reference-guide/psychology/fogg-behavior-model, n.d., accessed: 2026-03-01.

[137] EMQX, “Mqtt vs http: Ultimate iot protocol comparison guide,” https://www.emqx.com/en/blog/mqtt-vs-http, 2026, accessed: 2026.

[138] HiveMQ, “Mqtt vs. http for iot and iiot,” https://www.hivemq.com/article/mqtt-vs-http-protocols-in-iot-iiot/, 2026, accessed: 2026.

[139] IoT For All, “Mqtt vs http for iot: Detailed protocol comparison,” https://www.iotforall.com/mqtt-vs-http-for-iot-detailed-protocol-comparison, 2026, accessed: 2026.

[140] IETF, “Rfc 8446: The transport layer security (tls) protocol version 1.3,” https://datatracker.ietf.org/doc/html/rfc8446, Tech. Rep., 2026, accessed: 2026.

[141] ——, “Rfc 7230: Hypertext transfer protocol (http/1.1): Message syntax and routing,” https://datatracker.ietf.org/doc/html/rfc7230, Tech. Rep., 2026, accessed: 2026.

[142] OASIS, “Mqtt version 3.1.1,” https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html, 2026, accessed: 2026.

[143] Espressif Systems, *ESP32-S3 Series Datasheet*, https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf, 2026, accessed: 2026.

[144] H. J. J. J. Ochoa *et al.*, “Comparative analysis of power consumption between mqtt and http protocols in an iot platform,” *Sensors*, vol. 23, no. 10, p. 4896, 2023, accessed: 2026.

[145] Come-Star, “Mqtt vs http: Key differences, performance in iot, and how to choose,” https://www.come-star.com/blog/mqtt-vs-http-key-differences-performance-in-iot-and-how-to-choose/, 2026, accessed: 2026.

[146] DreamFactory, “Optimizing iot protocols for edge microservices,” https://blog.dreamfactory.com/optimizing-iot-protocols-for-edge-microservices, 2026, accessed: 2026.

[147] EMQX, “Emqx vs mosquitto performance benchmark,” https://www.emqx.com/en/blog/emqx-vs-mosquitto-performance-benchmark, 2026, accessed: 2026.

[148] HiveMQ, “Mqtt broker scalability,” https://www.hivemq.com/mqtt-broker/, 2026, accessed: 2026.

[149] Amazon Web Services (AWS), “Api gateway quotas and constraints,” https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html, 2026, accessed: 2026.

[150] NGINX, *NGINX Core Module: worker_connections*, https://nginx.org/en/docs/http/ngx_http_core_module.html#worker_connections, 2026, accessed: 2026.

[151] HiveMQ, “Mqtt essentials part 2: Publish & subscribe,” https://www.hivemq.com/blog/mqtt-essentials-part2-publish-subscribe/, 2026, accessed: 2026.

[152] OASIS, *MQTT Version 5.0 Specification*, https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html, 2026, accessed: 2026.

[153] C. Richardson, “Pattern: Microservice architecture,” 2025, accessed: 2026-03-01. [Online]. Available: https://microservices.io/patterns/microservices.html

[154] ——, “Pattern: Api gateway / backends for frontends,” 2025, accessed: 2026-03-01. [Online]. Available: https://microservices.io/patterns/apigateway.html

[155] ——, “Microservice architecture patterns,” 2025, accessed: 2026-03-01. [Online]. Available: https://microservices.io/patterns/

[156] D. A. Craft, “Api gateway pattern in a microservices architecture,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.digitalapi.ai/blogs/api-gateway-pattern-in-a-microservices-architecture

[157] GeeksforGeeks, “Microservices communication patterns,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/system-design/microservices-communication-patterns/

[158] ——, “Inter service communication in microservices,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/system-design/inter-service-communication-in-microservices/

[159] Cerbos, “Inter-service communication in microservices,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.cerbos.dev/blog/inter-service-communication-microservices

[160] GeeksforGeeks, “Synchronous vs asynchronous communication system design,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/system-design/synchronous-vs-asynchronous-communication-system-design/

[161] T. S. Side, “Synchronous vs asynchronous microservices communication patterns,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.theserverside.com/answer/Synchronous-vs-asynchronous-microservices-communication-patterns

[162] Solo.io, “Api gateway pattern,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.solo.io/topics/api-gateway/api-gateway-pattern

[163] C. Richardson, “Microservices authentication and authorization,” 2025, accessed: 2026-03-01. [Online]. Available: https://microservices.io/post/architecture/2025/05/28/microservices-authn-authz-part-2-authentication.html

[164] S. O. Community, “Synchronous vs asynchronous in microservice pattern,” 2021, accessed: 2026-03-01. [Online]. Available: https://stackoverflow.com/questions/67343585/synchronous-vs-asynchronous-in-microservice-pattern

[165] C. Richardson, “Api gateway pattern,” 2014, accessed: 2026-03-01. [Online]. Available: https://microservices.io/microservices/news/2014/09/08/apigateway-pattern.html

[166] GeeksforGeeks, “Different ways to establish communication between spring microservices,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/java/different-ways-to-establish-communication-between-spring-microservices/

[167] M. Learn, “Direct client-to-microservice communication versus the api gateway pattern,” 2024, accessed: 2026-03-01. [Online]. Available: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/direct-client-to-microservice-communication-versus-the-api-gateway-pattern

[168] MetaDesign Solutions, “Supabase vs firebase: When to choose open-source postgres for your apps scalability,” 2024, accessed: 2026-03-01. [Online]. Available: https://metadesignsolutions.com/supabase-vs-firebase-when-to-choose-open-source-postgres-for-your-apps-scalability/

[169] Supabase, “Supabase alternatives,” 2024, accessed: 2026-03-01. [Online]. Available: https://supabase.com/alternatives

[170] Elestio Blog, “Self-hosting supabase,” 2024, accessed: 2026-03-01. [Online]. Available: https://elest.io/blog/self-hosting-supabase

[171] Milvus, “What are the limitations of relational databases?” 2024, accessed: 2026-03-01. [Online]. Available: https://milvus.io/ai-quick-reference/what-are-the-limitations-of-relational-databases

[172] TigerData, “Time-series database vs relational database,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.tigerdata.com/learn/time-series-database-vs-relational-database

[173] ——, “The best time-series databases compared,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.tigerdata.com/learn/the-best-time-series-databases-compared

[174] ——, “Best practices for hypertables,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.tigerdata.com/learn/best-practices-for-hypertables

[175] ——, “Timescaledb alternatives,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.tigerdata.com/learn/timescaledb-alternatives

[176] ——, “Continuous aggregates and retention policies in timescaledb,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.tigerdata.com/learn/continuous-aggregates-and-retention-policies-in-timescaledb

[177] Supabase, “Supabase storage is s3 compatible,” 2024, accessed: 2026-03-01. [Online]. Available: https://supabase.com/blog/s3-compatible-storage

[178] ——, “Supabase storage documentation,” 2024, accessed: 2026-03-01. [Online]. Available: https://supabase.com/docs/guides/storage

[179] ——, “Supabase storage repository,” 2024, accessed: 2026-03-01. [Online]. Available: https://github.com/supabase/storage

[180] DeepWiki, “Supabase core concepts,” 2024, accessed: 2026-03-01. [Online]. Available: https://deepwiki.io/supabase-core

[181] IoT Class, “Edge, fog, and cloud architectures overview,” 2026, accessed: 2026-03-01. [Online]. Available: https://iotclass.org/content/architectures/distributed-specialized/edge-fog-cloud-overview.html

[182] DigitalOcean, “Edge computing vs. cloud computing,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.digitalocean.com/resources/articles/edge-computing-vs-cloud-computing

[183] NSF Public Access Repository, “Understanding edge vs. cloud trade-offs,” 2024, accessed: 2026-03-01. [Online]. Available: https://par.nsf.gov/servlets/purl/10184999

[184] GeeksforGeeks, “Difference between edge computing and cloud computing,” 2024, accessed: 2026-03-01. [Online]. Available: https://www.geeksforgeeks.org/cloud-computing/difference-between-edge-computing-and-cloud-computing/

[185] Scale Computing, “What is the difference between edge computing and cloud computing?” 2024, accessed: 2026-03-01. [Online]. Available: https://www.scalecomputing.com/resources/what-is-the-difference-between-edge-computing-and-cloud-computing

[186] ——, “What edge devices do i need?” 2024, accessed: 2026-03-01. [Online]. Available: https://www.scalecomputing.com/resources/what-edge-devices-do-i-need

[187] Elestio, “Supabase on elestio: Self-host your backend with auth, realtime subscriptions, and edge functions,” https://blog.elest.io/supabase-on-elestio-self-host-your-backend-with-auth-realtime-subscriptions-and-edge-func 2024, accessed: 2026-03-07.

[188] TigerData, “Time-series data: Why and how to use a relational database instead of nosql,” https://www.tigerdata.com/blog/time-series-data-why-and-how-to-use-a-relational-database-instead-of-nosql-d0cd6975e87c 2024, accessed: 2026-03-07.

[189] ——, “Data retention with continuous aggregates,” https://www.tigerdata.com/docs/use-timescale/latest/data-retention/data-retention-with-continuous-aggregates, 2024, accessed: 2026-03-07.

[190] DeepWiki, “3.2 core backend services,” https://deepwiki.com/supabase/supabase/3.2-core-backend-services, 2024, accessed: 2026-03-07.

[191] IEEE, “Hybrid edge-cloud architecture concepts for real-time iot processing,” *IEEE Xplore*, 2024, accessed: 2026-03-07.

[192] JATIT, “Edge-cloud architecture for smart systems,” *Journal of Theoretical and Applied Information Technology*, 2024, accessed: 2026-03-07.

[193] IJFMR, “Analysis of real-time processing on iot edges,” *International Journal for Multidisciplinary Research*, 2024, accessed: 2026-03-07.

[194] B. Infosys, “React native supabase auth: Your ultimate guide,” https://ftp.broadwayinfosys.com/blog/react-native-supabase-auth-your-ultimate-guide-1764800567, 2024, accessed: 2026-03-07.

[195] Supabase, “React native quickstart,” https://supabase.com/docs/guides/auth/quickstarts/react-native, 2024, accessed: 2026-03-07.

[196] R. Community, “Firestore, supabase, backend auth and ai for react,” https://www.reddit.com/r/reactnative/comments/1ot9v80/firestore_supabase_backend_auth_and_ai_for_react/, 2024, accessed: 2026-03-07.

[197] TigerData, “Timescaledb vs influxdb for time-series data,” https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series-data-timescale-influx-sql-nosql-36489299 2024, accessed: 2026-03-07.

[198] Tinybird, “Clickhouse vs timescaledb,” https://www.tinybird.co/blog/clickhouse-vs-timescaledb, 2024, accessed: 2026-03-07.

[199] B. Tech, “Redis vs timescaledb for realtime data performance,” https://bix-tech.com/redis-vs-timescaledb-for-realtime-data-performance-architecture-and-when-to-use-each 2024, accessed: 2026-03-07.

[200] Zeetim, “Thick client vs thin client,” https://zeetim.com/thick-client-vs-thin-client/, 2024, accessed: 2026-03-07.

[201] O. Accelerator, “Thin client vs thick client,” https://www.outsourceaccelerator.com/articles/thin-client-vs-thick-client/, 2024, accessed: 2026-03-07.

[202] IEEE, “Ieee document 10492690,” *IEEE Xplore*, 2024, accessed: 2026-03-07.

[203] ——, “Ieee document 8847093,” *IEEE Xplore*, 2024, accessed: 2026-03-07.

[204] ——, “Ieee document 11038522,” *IEEE Xplore*, 2024, accessed: 2026-03-07.

[205] Bohrium, “Edge-cloud architectures for hybrid energy management systems: A comprehensive review,” *Bohrium Paper Details*, 2024, accessed: 2026-03-07.

[206] IoT World Forum, “Iot world forum reference model,” https://www.researchgate.net/figure/oT-World-Forum-Reference-Model_fig2_323525875, 2014, accessed: 2026-03-08.

[207] Supabase, “Realtime architecture flow in modern baas,” https://supabase.com/docs/guides/realtime/architecture, 2024, accessed: 2026-03-08.